SOLARINET · MAINTAINER & INTEGRATOR REFERENCE

API & Expansion Manual

The interfaces of the SolariNet monitoring and system-of-record platform: REST, control plane, change-data capture, SoR, status panel and provisioning. Also covers how to extend the platform, how it is maintained, and where it is heading.

As of main **a28bdf2** · 2026-11-03 · live release rc-20261103-c39f565-io
Supersedes *SolariNet_Inventory_API.html* in substance. Operator: Jason C. Kay. Internal — no secrets are reproduced; credentials are referenced by path only.

Contents

0 About this manual

0.1 Scope and baseline
0.2 Conventions
0.3 How to use it

1 System surfaces and hosts

1.1 Hosts and what they serve
1.2 Interface inventory
1.3 Two databases, one direction of truth

2 REST fundamentals

2.1 Base URL, transport and dispatch
2.2 Authentication
2.3 Authorization policy
2.4 CSRF
2.5 Response envelope
2.6 Common errors
2.7 Request bodies and headers
2.8 Pagination
2.9 Optimistic locking
2.10 Queued control operations
2.11 Handler implementation rules

3 Interface catalogue

3.1 Overview
3.2 Authentication & session
3.3 Monitoring & overview
3.4 Status panel
3.5 Fleet control, discovery & enrollment
3.6 DNS & network allocation
3.7 Identity, PKI & Forgejo
3.8 Provisioning
3.9 Inventory
3.10 RackWire

4 Control socket & SCP wire protocol

4.1 solariCtl: socket and framing
4.2 The privileged-work queue
4.3 Verb table
4.4 SCP transport
4.5 SCP frame format

5 Change data capture & messaging

5.1 Outbox
5.2 Emitter (sor_emitd)
5.3 Appliers
5.4 Notification bus

6 The SoR as a read surface

6.1 Access rules
6.2 Data model at a glance
6.3 IP addresses: the INET6 rule
6.4 Useful read queries
6.5 The monitoring database (separate)

7 Status panel interface

7.1 The chain
7.2 REST contract
7.3 Wire framing
7.4 Daemon configuration
7.5 Changing the panel contract

8 Provisioning & Ansible hooks

8.1 Catalogue API and validation
8.2 The three enqueue verbs
8.3 The Ansible tree
8.4 Extending provisioning

9 Notifications

9.1 Message contract
9.2 Dispatch and delivery semantics
9.3 Senders
9.4 alertbridge: bridge and dead-man's switch
9.5 notify.conf reference

10 Configuration reference

10.1 C daemons (server, client, monitor)
10.2 Dashboard (PHP-FPM environment)
10.3 Python services

11 Extension recipes

11.1 Add a dashboard screen
11.2 Add an API route
11.3 Add a SoR migration
11.4 Add a CDC applier
11.5 Add a client metric (C)
11.6 Add an SNMP or discovery source
11.7 Add an inventory product kind or location level
11.8 Add a notification channel
11.9 Drive the API from an external tool

12 Maintenance

12.1 Dependencies
12.2 Versioning and release
12.3 Migrations and rollback
12.4 Backups
12.5 Deprecation policy PROPOSED
12.6 Testing matrix

13 Roadmap

13.1 Inventory phases I1–I5
13.2 Deprecations
13.3 Interface and operations backlog

A Error code index

A.1 Raised before a handler runs
A.2 Raised by route handlers
A.3 Retry guidance

B Route index

B.1 All routes

0 About this manual

This manual is the reference for every interface SolariNet exposes. It is also the procedure book for extending the platform without breaking it. It is written for maintainers, integrators and anyone adding a screen, route, migration, applier, probe or channel.

0.1 Scope and baseline

Source baseline	`main` at `a28bdf2` (2026-11-03). Every route, verb and table named here was read from that tree.
Live baseline	Release `rc-20261103-c39f565-io` under `/opt/solarinet/releases/` on xenon. The SoR ledger head on cesium is `037_inventory_items_closeout` (applied 2026-11-03 21:18 UTC), checked on 2026-11-03. Passages marked **Since 037** were verified against the merged 037 source on `main`.
Covers	The HTTP API (194 routes), the C control socket and SCP wire protocol, CDC over RabbitMQ, the SoR as a read surface, the status-panel contract, provisioning and Ansible hooks, notification, and configuration names. It also covers extension recipes, maintenance discipline and the roadmap.
Supersedes	`docs/SolariNet_Inventory_API.html` (rev. 1.4). Its pagination, compatibility, product-catalogue and location content is folded into §3 Inventory and §2 Pagination. That file is left unedited.
Companions	The *User Manual* covers operating the dashboard. The *Developer Manual* covers the build and internals. This manual covers the contracts between components and how to add to them.

0.2 Conventions

- **Verified vs inferred.** Statements are verified against code or the live system unless they are tagged `INFERRED` or placed in a grey *Inference* box.
- **Since 037.** A note headed **Since 037** describes inventory behaviour introduced by migration 037 (live 2026-11-03).
- **Roles.** `public` means no session. `session` means any signed-in principal, including `viewer`. `operator` means role `operator` or `admin`. `admin` is admin only.
- **Secrets.** No credential, token, key or option-file content appears here. Where a secret is needed, the manual names the file or environment variable that holds it.
- **Source references** are `path:line` against the baseline commit. Line numbers drift; the symbol names do not.
- Hostnames follow the periodic-table convention: xenon, cesium, benzene, lithium, radium, chlorine, steel.

0.3 How to use it

If you are…	Start at
Calling the API from a script or tool	§2 REST fundamentals, then §11.9 Integrating an external tool
Looking up one endpoint	Type into the filter box (press /), or use Appendix B
Adding a feature	§11 Extension recipes and the extension-point map
Changing the SoR schema	§12.3 Migration discipline
Consuming change events	§5 CDC and messaging
Planning work	§13 Roadmap

1 System surfaces and hosts

SolariNet is a C monitoring tier (server, clients, monitors), a PHP dashboard and API, a MariaDB System of Record (SoR), and a set of Python services joined by RabbitMQ. Each has its own interface. This chapter places them on hosts so the later chapters can refer to them by name.

1.1 Hosts and what they serve

Host	Role in SolariNet	Interfaces it exposes
xenon	Primary server. Runs Apache and PHP 8.5.4 (dashboard + API), `solariServer` and a local client and monitor, and the monitoring DB (MariaDB 11.8.6). Also runs `sor-emit`, `sor-apply-dns`, `sor-reconcile`, `unifipolld`, `notify`, `alertbridge` and the BIND primary for `akoria.net`.	HTTPS `dashboard.akoria.net`; SCP TLS on 7701/7702/7703; ctl socket `/run/solari/solariCtl.sock`
cesium	SoR host (MariaDB 12.3.2, database `solari`). Also runs the Forgejo git server and the nightly SoR dump.	MariaDB 3306 (grant-scoped); Forgejo SSH 2222
benzene	RabbitMQ broker (10.5.2.50:5672, vhost `/solari`), the provisioning store and the Ansible tree.	AMQP exchanges `sor.events` and `notify.events`; `/srv/solari-provision/{store,ansible}`
lithium	Status-panel host. The `solariPanel` daemon drives the RP2350 LED panel over USB-CDC.	Calls `/api/panel*`; serial protocol to the panel
radium	Samba AD DC (BIND9_DLZ for `akoria.org`) and Keycloak (`sso.akoria.org:8443`, realm `akoria`).	OIDC issuer; AD DNS, written by `sor_apply_ad` `INFERRED`. That unit is not installed on xenon, and its unit file runs as root from `/opt/solari-sorsync`.
chlorine	Internal CA (step-ca) at 10.7.0.10, and an authoritative BIND secondary (`[dns.secondary.chlorine]`).	ACME / X.509 issuance used by the PKI routes; DNS
steel	Authoritative BIND secondary (`[dns.secondary.steel]`), applied through `STACK_APPLY`.	DNS; one of the `/api/dns/query` targets

`sor-apply-pihole` is installed on xenon but disabled. Versions and units were read from the live hosts on 2026-11-03: `SELECT VERSION()` on both databases, `php -v` on xenon, the release `RELEASE.json`, and `systemctl list-unit-files` on xenon.

1.2 Interface inventory

These are the interfaces an integrator or extender can build against. They are listed by stability: earlier rows are safer contracts.

Interface	Transport	Consumers	Contract owner	Chapter
Dashboard REST API	HTTPS + JSON envelope, cookie session	SPA, panel daemon, scripts	`dashboard/api/` + `docs/authz/AUTHZ_MANIFEST.json`	2, 3
CDC events	AMQP topic `sor.events`	DNS, AD and Pi-hole appliers	migrations 022–030 + `deploy/sorsync/`	5
Notifications	AMQP topic `notify.events`	notifyd senders	`deploy/notify/`	9
SoR (read)	MariaDB SQL	reports, reconcilers	`netdb/sor/`	6
Panel HTTP + serial	HTTPS + USB-CDC frames	lithium daemon, RP2350 firmware	`status-panel/CONTRACT*.md`	7
ctl socket	AF_UNIX, line protocol	PHP API, `solariCtlClient`	`src/server/solariCtl.c`	4
SCP	NNG over mutual TLS, binary TLV	solariClient, solariMonitor	`include/solari/solariFrame.h`	4
Provisioning helpers	queued ctl verbs → Python → Ansible	dashboard Provisioning screens	`deploy/fleet/`	8

> **RULE** Integrations go through the REST API, or through read-only SoR SELECTs under a dedicated grant. Writing SoR tables directly bypasses validation, audit stamping and the CDC guarantees. Calling the ctl socket directly bypasses the HTTP policy layer. Neither is a supported integration path.

1.3 Two databases, one direction of truth

The **monitoring DB** (xenon, MariaDB 11.8.6, migrations in `db/migrations/`) holds telemetry and fleet state: `hostCurrent` / `hostHistory`, `probeCurrent`, `alertEvent`, `discovered`, enrolment and the ctl request queue. It is written by `solariServer` and the API.

The **SoR** (cesium, MariaDB 12.3.2, `netdb/sor/schema.sql` plus `netdb/sor/migrations/`) holds intended state: entities, addressing, DNS, provisioning catalogue, inventory and RackWire plans. Changes to its CDC tables leave an outbox row in the same transaction, and downstream systems (DNS, AD) are rendered from it. Discovery flows the other way, through `sor_reconcile_discovery`, which promotes adopted monitoring rows into the SoR.

The two databases have **independent migration numbering**. Monitoring `033` and SoR `033` are unrelated files. Always say which stream you mean.

2 REST fundamentals

The dashboard API is one PHP front controller under `/api/`. Every route returns the same JSON envelope and is guarded by the same default-deny policy. Learn the rules in this chapter once and they apply to all 194 routes.

2.1 Base URL, transport and dispatch

Base URL	`https://dashboard.akoria.net/api/` (Apache 2.4 on xenon, HTTPS only, HSTS)
Front controller	`dashboard/api/index.php`. It registers the auth routes, then `routes.php` (reads), then `routes_mutations.php` (writes). Each file loads `routes/*.php` group files from its `$groups` list.
Session gate	Every path except `/api/auth/*`, `POST /api/enrollments` and `POST /api/enrollments/ack` needs a session before dispatch.
Router	`dashboard/api/lib/Router.php`. Patterns use `{name}` segments. An unknown path returns `404 not_found`. A known path with the wrong method returns `405 method_not_allowed`.
Guard	`Policy::enforce` runs before every handler (`dashboard/api/lib/Policy.php`).

Every JSON response carries `Content-Type: application/json; charset=utf-8`, `Cache-Control: no-store` and `X-Content-Type-Options: nosniff`. Apache adds `X-Frame-Options: SAMEORIGIN`, `Referrer-Policy: no-referrer` and HSTS; these were observed live on 2026-11-03.

2.2 Authentication

A session is an opaque PHP session cookie. Credentials are never held client-side. There are two ways to obtain a session.

Keycloak OIDC (primary)

The dashboard is a confidential client using the authorization-code flow. Live, the client is `solarinet-dashboard` in realm `akoria` at `https://sso.akoria.org:8443/realms/akoria`, with scopes `openid profile email`. Keycloak federates to the Samba AD on radium.

1. `GET /api/auth/oidc/login` stores a fresh `state` and `nonce` in the session (valid for 600 s), then redirects with 302 to the Keycloak authorization endpoint.
2. Keycloak authenticates the user and redirects to `GET /api/auth/oidc/callback?code=…&state=…`.
3. The API checks `state` and exchanges the code at the token endpoint. It then validates the ID token: signature via JWKS (RS256/384/512), `iss`, `aud`, `exp` and `nonce`. TLS to the IdP is verified against `oidc.caBundle`.
4. Groups map to a role. `oidc.adminGroups` (default `["domain admins"]`) gives `admin`. `oidc.operatorGroups` gives `operator`. Everyone else gets `oidc.defaultRole` (default `viewer`). The session ID is regenerated and the browser is sent to `postLoginRedirect`.
5. On later requests the role is refreshed from HTTPS UserInfo at most every 60 s. If the UserInfo subject changes, the session is revoked.

Local password login (break-glass)

`POST /api/auth/login` with `{"username","password"}`. It is enabled only when `SOLARI_LOCAL_LOGIN=1` *and* `SOLARI_AUTH_RATE_DIR` names an absolute, non-symlink, writable directory with mode `0700` or tighter. Attempts are limited to 5 per account and source in a rolling 15 minutes. Accounts live in the credential file at `SOLARI_AUTH_FILE` (default: project-root `solari-auth.json`). Hashes are verified with `password_verify`. Use `dashboard/api/tools/solari-passwd.php` to set one. On every request the local account is reloaded from that file, so a deleted or downgraded account loses access at once.

Session properties

Property	Value	Source
Cookie	`solari_sess`; `HttpOnly`, `SameSite=Lax`, `Path=/`, `Secure` when served over TLS, browser-session lifetime	`Auth::bootSession`
Idle timeout	1800 s	`Auth::IDLE_MAX_SEC`
Absolute lifetime	28 800 s (8 h) local; 3600 s (1 h) OIDC	`ABSOLUTE_MAX_SEC`, `OIDC_MAX_SEC`
Roles	`viewer` < `operator` < `admin`; any other value ends the session	`Auth::current`
Fixation	Session ID regenerated on every login	`session_regenerate_id(true)`
Principal	`GET /api/auth/whoami` → `{operator, role, displayName, source, directoryEnabled, oidcEnabled, csrfToken}`	`routes/auth.php`

> **DIRECTORY BIND** `Auth::directoryAuthenticate` (direct LDAP/AD password bind) is a non-functional stub and always returns null. AD users sign in through Keycloak.

2.3 Authorization policy

`Policy::enforce` is default-deny. A route may declare a policy as the fourth argument to `$router->get/post/add`:

```
$router->post('/api/example', static function (): void { … }, ['auth' => 'operator']);
$router->add('PUT', '/api/example/{id}', $handler, Policy::INVENTORY_CATALOG_WRITE);
$router->post('/api/panel/state', $handler, ['auth' => 'session', 'csrf' => 'panel-service']);
```

Declared `auth`	Meaning	Live count
(undeclared)	`GET` / `HEAD` / `OPTIONS` → `session`; every other method → `operator`	—

Declared `auth`	Meaning	Live count
`public`	No session. Login, auth config, the two OIDC legs and the two enrolment-bearer routes. The bearer token is validated by the C bridge.	6
`session`	Any principal, including `viewer`	86
`operator`	`operator` or `admin`, else `403 forbidden`	102
`admin`	`admin` only (no route currently declares it)	0
anything else	Fails closed with `403 forbidden` ("Unrecognized policy")	—

The declared policy for every route is mirrored in `docs/authz/AUTHZ_MANIFEST.json` (`phpRoutes[]`: `method`, `pattern`, `csrf`, `roleRequired`, optional `servicePrincipal`). CI test `tests/dashboard/test_authz_manifest.php` fails when code and manifest disagree.

2.4 CSRF

Every state-changing request authenticated by cookie must pass two checks, unless the route uses the panel-service bridge. Failing either returns `403 forbidden`.

1. **Origin.** The `Origin` header (or, failing that, the origin of `Referer`) must equal this request's own `scheme://host` or an entry in `SOLARI_ALLOWED_ORIGINS` (comma-separated). A request carrying neither header is rejected.
2. **Token.** `X-Solari-CSRF` must equal the session's synchronizer token. Get the token from `csrfToken` in the `POST /api/auth/login` or `GET /api/auth/whoami` response. It lasts as long as the session.

Panel-service bridge. On routes declared `'csrf' => 'panel-service'` (only `POST /api/panel/state` and `POST /api/panel/config`), CSRF is skipped only when both conditions hold:

- the principal is the local account `panel` with role `viewer`;
- the request has no `Origin`, `Referer` or `Sec-Fetch-Site` header, so it cannot come from a browser.

Use this pattern for any future headless service principal; do not loosen it.

> **SCRIPTS** A non-browser client making writes must send an `Origin` header matching the dashboard origin, as well as the cookie and `X-Solari-CSRF`. See §11.9 for a complete curl session.

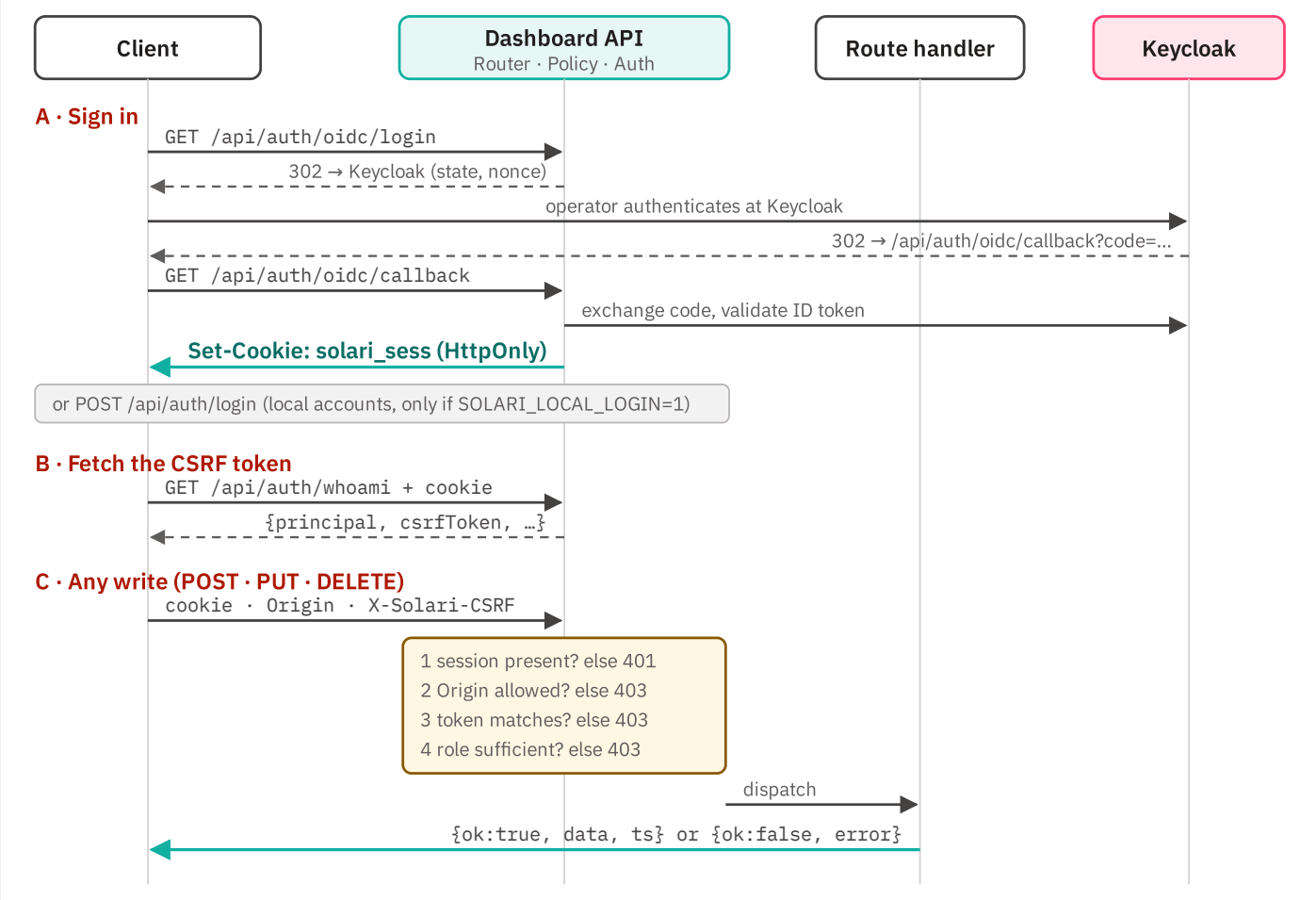


Figure 1. Sign-in, CSRF-token fetch and a write. The checks in step C run in `Policy::enforce` in this order. Public routes skip them all, and reads skip the Origin and token checks.

2.5 Response envelope

```
// success (HTTP 2xx)
{"ok": true,
 "data": { … },
 "ts": "2026-11-03T21:19:10Z"}
```

```
// failure (HTTP 4xx/5xx)
{"ok": false,
 "error": {"code": "conflict",
           "message": "Location changed since it was read."}}
```

- `ts` is server time, ISO-8601 UTC with a `Z` suffix (`Response::nowIso8601Utc`).
- Stored datetimes are emitted through `Coerce::iso()` as ISO-8601 `Z`. Large identifiers go through `Coerce::id()` and may arrive as **strings**; treat IDs as opaque.
- Field names in `data` are camelCase on monitoring routes. Many SoR-backed routes (inventory, provisioning, RackWire) return SoR column names in snake_case. Each route card in §3 gives the actual shape.
- Branch on `error.code`, which is stable. `error.message` is for humans and may change.
- Two routes return non-JSON bodies on success: `GET /api/inventory/codes/{id}/label` (PNG or PDF bytes) and the OIDC redirects (302).

2.6 Common errors

These apply to every route and are not repeated on each route card. Route-specific codes are listed on the cards and indexed in Appendix A.

HTTP	error.code	Raised when	Origin
400	`bad_request`	Body is not a JSON object or array; a path ID is malformed; a field fails validation	`solari_json_body`, route validators
401	`unauthorized`	No session, or the session expired (idle, absolute, revoked)	session gate, `Auth::requireSession`
403	`forbidden`	Role too low; CSRF origin or token failed; ctl peer refused a verb	`Policy`, `Operator`, `SolariCtl`
404	`not_found`	No such route, or no such row	Router, handlers
405	`method_not_allowed`	Path exists, method does not	Router
409	`conflict`	Optimistic-lock mismatch; unique/FK violation (SQLSTATE 23000); SoR trigger rejection (SQLSTATE 45000, trigger message passed through)	`Inv::exec`, `ProvCatalog::dbFail`, `rw_write`
413	`payload_too_large`	Body exceeds a route's declared limit	`solari_json_body($max)`
415	`unsupported_media_type`	Non-empty body whose type is not `application/json` (MIME parameters allowed)	`solari_json_body`
500	`internal_error`	Uncaught exception, including an unmapped DB error. It is logged server-side.	`bootstrap.php`
502	`control_unavailable` / `control_error`	ctl socket unreachable, or the bridge returned no request ID	`SolariCtl`
503	`service_unavailable` / `busy`	SoR unreachable or not configured; lock wait or deadlock on a provisioning or RackWire write	`Sor::db`, `ProvCatalog`, `rw_*`

SINCE 037 Inventory writes that hit a MariaDB deadlock return **HTTP 409** with a message asking the client to **try again**. They no longer surface as an unmapped 500. Retry the whole request with a fresh read; it is safe because the failed transaction rolled back. Provisioning and RackWire keep their existing `503 busy` mapping.

2.7 Request bodies and headers

- **JSON only.** `solari_json_body($maxBytes = null)`. An empty body is treated as `{}`. A non-empty body must be `application/json` and decode to an object or array.
- **Size limits** apply only where a route passes one. Provisioning catalogue writes use `ProvCatalog::body`, which allows 262 144 bytes by default or 16 KiB on smaller routes. It also **rejects unknown keys**.
- **Path IDs** must be positive decimal integers. Inventory validates them with `Inv::idParam`; provisioning accepts up to 4 294 967 295.
- `Idempotency-Key` (optional) is accepted on queued control routes. When present it must be 8–128 characters from `[A-Za-z0-9._-]`, otherwise `400`. It is passed to `REQUEST_SUBMIT`, so a retried request maps to the same queued job.
- **Methods.** Many fleet mutations are `POST …/{id}/delete` or `POST …/{id}/remove` rather than `DELETE`. Inventory and provisioning use `PUT` and `DELETE`. Use exactly the method in the catalogue.

2.8 Pagination

There are three pagination styles; each route card says which applies.

Style	Routes	Parameters	Response
Page envelope	Inventory lists: components, models, receipts, projects, allocations, `locations?flat=1`	`limit` 1–5000, `offset` 0–2 147 483 647. Supplying either enables paging; the other then defaults to `limit=500` or `offset=0`.	`data = {items:[…], page:{limit, offset, total, hasMore, nextOffset}}`

Style	Routes	Parameters	Response
Catalogue page	`GET /api/inventory/product-types`	`limit` 1–500 (default 100), `offset`	Same `{items, page}` shape
Clamped limit	Node and gear history; code lists and history	`limit` clamped: history default 1000, max 5000. Code list default 500, max 5000. Code history default 100, max 1000.	Plain array

Compatibility mode. If you omit both `limit` and `offset` on a page-envelope route, `data` is a bare array, exactly as before paging existed. Components still cap that array at 500 rows; the other lists are unbounded in this mode. New integrations should always send `?limit=500&offset=0` explicitly and loop while `page.hasMore`, using `page.nextOffset`.

```
GET /api/inventory/models?hw_type=server&limit=100&offset=200
→ {"ok":true,"data":{"items":[{"id":123,"make":"Example","model":"Unit"}],
   "page":{"limit":100,"offset":200,"total":237,"hasMore":false,"nextOffset":null}},
   "ts":"2026-10-21T00:00:00Z"}
```

- `total` counts rows matching the filters, before paging. Order within each list is deterministic.
- Offset pages are **not** a snapshot; inserts and deletes between requests shift them. Treat a changed `total` or a duplicate ID across pages as a failed read and retry. The dashboard's browser adapter does this. It reads 1000-row pages and refuses collections over 100 000 rows. Same-count field edits between pages are not detected.
- The combined `GET /api/inventory` returns `{models, units, locations, receipts, projects, allocations}` only while every collection has 500 rows or fewer. Above that it returns `413 inventory_page_required`. It takes no page parameters, so use the per-collection routes instead.

2.9 Optimistic locking

Routes that edit rows people may edit at the same time use compare-on-write. Read the row, keep its version marker, and send the marker back with the edit.

Surface	Marker	Required?	On mismatch
`PUT /api/inventory/locations/{id}`	`updated_at` in the body: the raw DB value, or the ISO `updatedAt` from the GET	Yes	`409 conflict` "Location changed since it was read."
`PUT /api/inventory/container-types/{id}`	`updated_at`	Yes	`409 conflict`
`PUT /api/inventory/product-types/{id}`	`updated_at`	Optional; checked only when sent	`409 conflict` "Product type changed since it was read."
`PUT` / `DELETE /api/inventory/components/{id}`	*none*	—	Last writer wins. Adding a check is on the backlog (§13).
RackWire plan update and restore	`expectedHash`: a hash over name, thumbnail, notes and plan JSON	Yes	`409 plan_conflict`, unless the requested state is already applied
Location merge	Preview `fingerprint` (SHA-256) from `merge-preview`	Yes	`409 merge_conflict`
Provisioning catalogue PUTs	*none*. These use `SELECT … FOR UPDATE` inside a transaction, so the last writer wins.	—	—

`loc_same_timestamp(current, expected)` accepts either form of the timestamp, so clients may echo exactly what they read. On a 409, re-read, re-apply the user's change, and resubmit. Never retry blindly with the new marker.

> **SINCE 037** Edit-conflict timestamps carry **microseconds**. Two edits within the same second are no longer mistaken for one version. Clients must echo `updated_at` / `updatedAt` *exactly as received*; truncating to whole seconds will cause spurious 409s.

2.10 Queued control operations

Privileged fleet actions (provision, decommission, deploy, discovery scans, pool and asset deletes, config pushes, provisioning helpers) cannot be executed by the web server's UID. The route enqueues them on the C server's queue (§4.2) and waits briefly.

- `SolariCtl::callQueued(verb, args, operator, idem, waitSec = 4.0)` submits the job, then polls every 100 ms until `waitSec` runs out. `POST /api/nodes/{id}/config` waits 15 s.
- If the job finishes in time, the response is the route's normal success.
- If not, the route returns success with `"status":"queued"` and `"request":"<id>"`. The route's own status field (for example `"status":"provisioning"`) may be replaced or accompanied by `request`; see the card.
- The only generic HTTP poll today is `GET /api/discovery/scan-requests/{requestId}`, which is scoped to discovery. For other queued jobs, re-read the affected resource.
- Errors from the inner verb map to `403 forbidden`, `502 control_unavailable` or `400 control_error`.

2.11 Handler implementation rules

Anyone writing or reviewing a route must know these server-side rules.

- **Native prepares.** Both `Db` (monitoring) and `Sor` (SoR) connect with `PDO::ATTR_EMULATE_PREPARES => false`. A named placeholder **must not appear twice** in one statement. MariaDB raises `HY093`, which surfaces as `500 internal_error`. Use `:from1` / `:from2`, or positional `?`. CI test `tests/dashboard/test_pdo_placeholders.php` scans every SQL literal under `dashboard/api` for this.

- **Prepared statements only.** `Db::rows/row/scalar/exec` and the `Inv::`, `ProvCatalog::` and `rw_` wrappers all prepare and bind. Identifiers such as metric columns are chosen from whitelists, never interpolated.
- **SoR provenance.** SoR writes stamp `source_id` (`Sor::dashboardSourceId()`, or the `rackwire` source), `asserted_kind='human'` and `asserted_at=NOW()`. Deletes are soft (`deleted_at`), and every read filters `deleted_at IS NULL`.
- **Monitoring writes.** These go through the ctl socket, not direct SQL. The exceptions are alert acknowledgement, maintenance windows, push subscriptions and panel state, which write their own tables.

3 Interface catalogue

This chapter lists every HTTP route the dashboard API registers: 194 routes at `a28bdf2`, grouped by domain. Each card is generated from the router source and the authorization manifest. Type in the search box, or use the chips below, to narrow the list.

3.1 Overview

Domain	Routes	Read	Write
Authentication & session	6	4	2
Monitoring & overview	24	18	6
Status panel	4	1	3
Fleet control, discovery & enrollment	40	12	28
DNS & network allocation	8	7	1
Identity, PKI & Forgejo	3	3	0
Provisioning	35	12	23
Inventory	62	26	36
RackWire	12	6	6
Total	**194**	**89**	**105**

How to read a card.

- The header shows the method, the path, and the role required. A `CSRF` badge means a state change needs `Origin` plus `X-Solari-CSRF` (§2.4).
- **Query** and **Body** list accepted fields; ***** marks a required field.
- **Errors** lists route-specific failures. The common errors in §2.6 are not repeated.
- **ctl verb** names the privileged C-server verb the route invokes (§4).
- **Tables** lists the monitoring or SoR tables the route reads or writes.

Paths, roles and CSRF flags were extracted mechanically and cross-checked against `docs/authz/AUTHZ_MANIFEST.json`; all 194 match. Body, response and error details were read from handler source by a second extraction pass, then spot-checked by hand. Small omissions are possible; the handler named in each card's source line is authoritative.

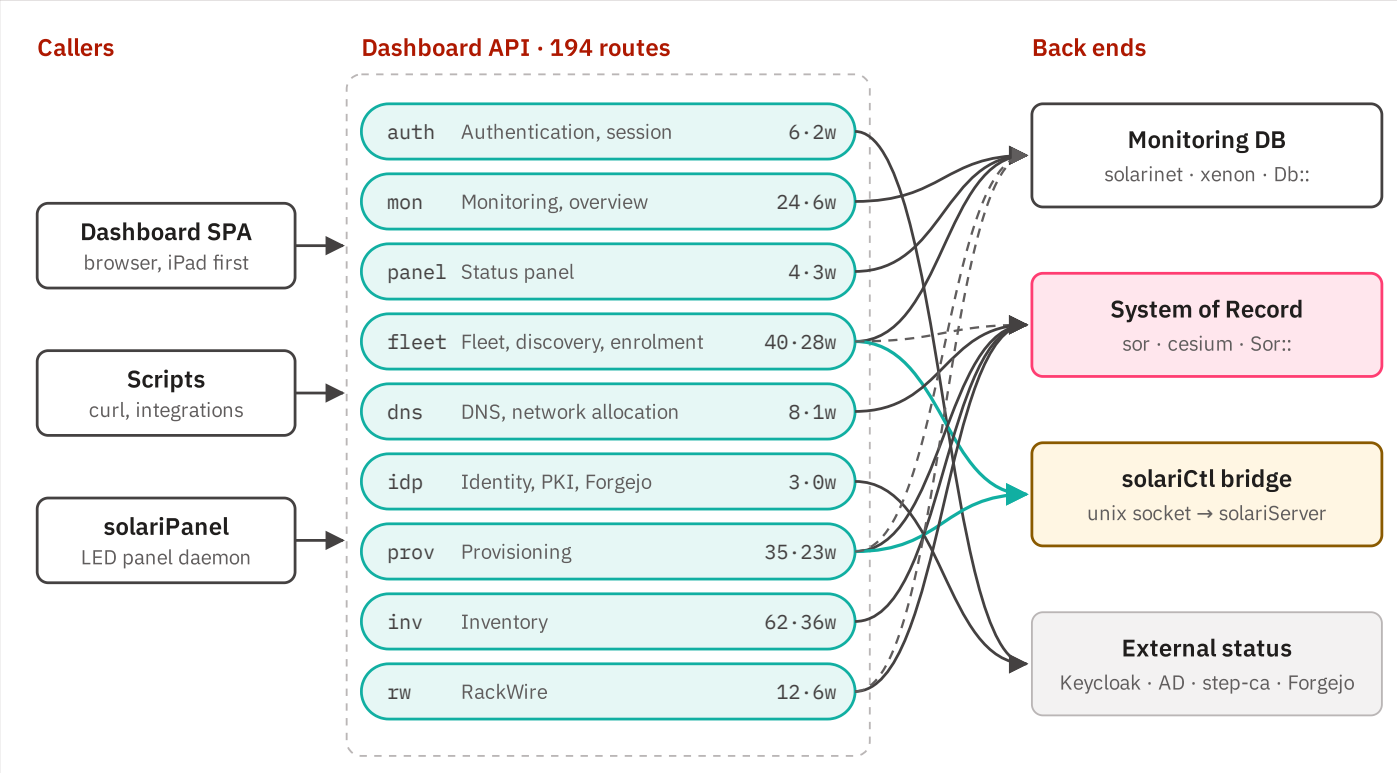


Figure 2. The nine route domains (route count · writes) and the back ends their handlers use. Solid lines are a domain's main store, teal lines are privileged solariCtl verbs, and dashed lines are secondary reads or writes: fleet adoption upserting SoR hosts, provisioning reading node versions, and RackWire reading monitoring interfaces. Derived from `Sor::`, `Db::` and `SolariCtl::` calls in `dashboard/api/routes/`.

3.2 Authentication & session · 6 routes

These six routes establish and inspect a session. `/api/auth/config` tells a client which login methods are enabled. `/api/auth/whoami` returns the principal and the session's `csrfToken`, which every write must echo (§2.4). The OIDC pair runs the Keycloak authorization-code flow; local login is gated by `SOLARI_LOCAL_LOGIN` (§2.2).

POST /api/auth/login public

Authenticates a local username and password and returns the principal and CSRF token.

Body — JSON object
- `username` string — Defaults to empty string.
- `password` string — Defaults to empty string.

Success — `200` · {operator,role,displayName,source,csrfToken}

Errors
- `403 local_login_disabled` — Local login is disabled.
- `401 invalid_credentials` — Credentials do not authenticate.
- `415 unsupported_media_type` — Nonempty body is not application/json.
- `400 bad_request` — Nonempty body is not a JSON object.

Notes — No explicit body-size limit. Authentication details are delegated to Auth.

```
POST /api/auth/login {"username":"operator","password":"fake-password"}
→ {"ok":true,"data":{"operator":"operator","role":"operator","displayName":"Operator","source":"local","csrfToken":"fake"},"ts":"2026-11-03T00:00:00Z"}
```

Source: `dashboard/api/routes/auth.php:16`

POST /api/auth/logout session CSRF: Origin + X-Solari-CSRF

Ends the current authenticated session.

Success — `200` · {loggedOut:true}

Errors — Common errors only (§2.6).

```
POST /api/auth/logout
→ {"ok":true,"data":{"loggedOut":true},"ts":"2026-11-03T00:00:00Z"}
```

Source: `dashboard/api/routes/auth.php:40`

GET /api/auth/config public

Returns public sign-in options and nonsecret OIDC and directory identity details.

Success — `200` · {localEnabled,oidcEnabled,oidcLabel,oidcLoginUrl,oidcProvider,oidcIssuer,oidcClientId,oidcScopes,directory:{enabled,type?,realm?,domain?,host?}}

Errors — Common errors only (§2.6).

Notes — Provider label is inferred from issuer text; secret and CA bundle fields are excluded.

```
GET /api/auth/config
→ {"ok":true,"data":{"localEnabled":true,"oidcEnabled":false,"oidcLabel":"Sign in","oidcLoginUrl":"/api/auth/oidc/login","oidcProvider":"OpenID
Connect","oidcIssuer":"","oidcClientId":"","oidcScopes":"","directory":{"enabled":false}},"ts":"2026-11-03T00:00:00Z"}
```

Source: `dashboard/api/routes/auth.php:49`

GET /api/auth/oidc/login public

Begins the OIDC authorization-code flow and redirects the browser.

Success — `302` · No JSON payload; redirects and exits.

Errors — Common errors only (§2.6).

Notes — Exact redirect and failure behavior is implemented by Oidc::beginLogin().

```
GET /api/auth/oidc/login
→ HTTP redirect to identity provider
```

Source: `dashboard/api/routes/auth.php:97`

GET /api/auth/oidc/callback public

Handles the identity-provider callback, establishes a session, and redirects into the app.

Success — `302` · No JSON payload; redirects and exits.

Errors — Common errors only (§2.6).

Notes — Callback inputs are passed from `$_GET`; exact validation and redirect outcomes are implemented by Oidc::handleCallback().

```
GET /api/auth/oidc/callback?code=fake&state=fake
→ HTTP redirect into application
```

Source: `dashboard/api/routes/auth.php:103`

GET /api/auth/whoami session

Returns the current principal, authentication configuration flags, and session CSRF token.

Success — `200` · {operator,role,displayName,source,directoryEnabled,oidcEnabled,csrfToken}

Errors
- `401 unauthorized` — Auth::current() returns no principal.

```
GET /api/auth/whoami
→ {"ok":true,"data":
{"operator":"operator","role":"operator","displayName":"Operator","source":"local","directoryEnabled":false,"oidcEnabled":false,"csrfToken":"fake"},"ts":"2026-11-
03T00:00:00Z"}
```

Source: `dashboard/api/routes/auth.php:108`

3.3 Monitoring & overview · 24 routes

These are the read surface of the monitoring database: nodes, network gear, probes, alerts, rules, segments, topology and the summary roll-up. Writes are limited to alert acknowledgement, maintenance windows and Web Push subscriptions. History routes clamp `limit` (§2.8). `GET /api/stream` is a retired event-stream endpoint that now answers `204` at once. Poll the summary instead.

SUMMARY · 1

GET `/api/summary` session

Returns fleet counts, node-state totals, active alert count, and server lease state.

Success	`200` · {counts:{nodes,clients,monitors,servers,probes,applications,segments,activeAlerts},stateRoll:{up,degraded,down,unknown,retired},lease:{leaseHolder,leaseEpoch,expiresAt}|null}
Errors	Common errors only (§2.6).
Tables	`node`, `probeTarget`, `procCurrent`, `alertEvent`, `segment`, `serverLease`

```
GET /api/summary
→ {"ok":true,"data":{"counts":{"nodes":3,"clients":1,"monitors":1,"servers":1,"probes":2,"applications":1,"segments":1,"activeAlerts":0},"stateRoll":
{"up":3,"degraded":0,"down":0,"unknown":0,"retired":0},"lease":null},"ts":"2026-11-03T00:00:00Z"}
```

Source: `dashboard/api/routes/summary.php:15`

NODES · 3

GET `/api/nodes` session

Lists nodes with current roster fields and up to 40 recent host-history sparkline samples per node.

Success	`200` · Array<{nodeId,role,hostFqdn,state,lastSeenAt,arch,osName,segId,uplinkGearId,enrolledAt,configEpoch,hist:{cpu,ram,net,disk},criticality?}>
Errors	Common errors only (§2.6).
Tables	`node`, `hostHistory`, `information_schema.COLUMNS`
Notes	`criticality` is omitted before the migration column is present.

```
GET /api/nodes
→ {"ok":true,"data":[{"nodeId":"42","role":"server","hostFqdn":"helium.example","state":"up","hist":{"cpu":[],"ram":[],"net":[],"disk":[]}}],"ts":"2026-11-
03T00:00:00Z"}
```

Source: `dashboard/api/routes/nodes.php:16`

GET `/api/nodes/{id}` session

Returns one node with current host metrics, process rows, and up to 50 recent alert events.

Path	• `id` string — Passed to parameterized node queries.
Success	`200` · {node:{…},host:{sampledAt,cpuLoadMilli,ramUsedKb,ramTotalKb,swapUsedKb,swapTotalKb,disks,ifaces,usbBuses}|null,procs:[{procName,pid,runState,nFiles,nSockets,rssKb,sampledAt}],alerts:[{eventId,ruleId,targetId,firedAt,clearedAt,severity,detail,scope,metric,op,threshold}]}
Errors	• `404 not_found` — No node row matches id.
Tables	`node`, `hostCurrent`, `procCurrent`, `alertEvent`, `alertRule`, `information_schema.COLUMNS`
Notes	`criticality` is omitted before the migration column is present.

```
GET /api/nodes/42
→ {"ok":true,"data":{"node":{"nodeId":"42","role":"server","hostFqdn":"helium.example","state":"up"},"host":null,"procs":[],"alerts":[]},"ts":"2026-11-03T00:00:00Z"}
```

Source: `dashboard/api/routes/nodes.php:76`

GET `/api/nodes/{id}/history` session

Returns one node metric time series in ascending sample time.

Path	• `id` string — Node identifier.
Query	• `metric` string default `cpuAvgMilli` — Allowed: cpuAvgMilli, ramUsedKb, swapUsedKb, diskMinFreePct, netKbps. • `from` datetime string — Parsed by NodeHistoryHelpers::isoToDb(). • `to` datetime string — Parsed by NodeHistoryHelpers::isoToDb(). • `limit` integer default `1000` — Clamped to maximum 5000.
Success	`200` · {nodeId,metric,series:[{ts,value}]}
Errors	• `400 bad_request` — Metric is not whitelisted.
Tables	`hostHistory`
Notes	Invalid date parsing behavior depends on NodeHistoryHelpers::isoToDb().

```
GET /api/nodes/42/history?metric=cpuAvgMilli&limit=2
→ {"ok":true,"data":{"nodeId":"42","metric":"cpuAvgMilli","series":[{"ts":"2026-11-03T00:00:00Z","value":120}]},"ts":"2026-11-03T00:00:00Z"}
```

Source: `dashboard/api/routes/nodes.php:176`

PROBES · 1

GET /api/probes session

Lists probe targets with per-monitor current results and a rolled-up target state.

Success	`200` · Array<{targetId,host,port,proto,replFactor,label,segId,assetId,hostNode,hostName,state,vantages:[{monitorNode,monitorName,outcome,rttMicros,jitterMicros,lossPermille,throughputKbps,serviceMeta,sampledAt}]}>
Errors	Common errors only (§2.6).
Tables	`probeTarget`, `asset`, `probeCurrent`, `node`
Notes	No vantage rows gives state unknown; mixed successful and non-successful vantage outcomes give degraded.

```
GET /api/probes
→ {"ok":true,"data":[{"targetId":"tcp:host:443","host":"host","port":443,"proto":"tcp","state":"unknown","vantages":[]}],"ts":"2026-11-03T00:00:00Z"}
```

Source: `dashboard/api/routes/probes.php:21`

ALERTS & RULES · 4

GET /api/alerts session

Lists alert events with rule and node context, filtered by lifecycle bucket.

Query	• `status` string default `active` — Allowed active, history, all; maximum 500 rows.
Success	`200` · Array<{eventId,ruleId,nodeId,hostFqdn,role,targetId,firedAt,clearedAt,ackedAt,ackedBy,status,severity,detail,scope,metric,op,threshold,opieReportId}>
Errors	• `400 bad_request` — Status is outside allowed values.
Tables	`alertEvent`, `alertRule`, `node`, `opieReport`
Notes	Active filter includes unacked uncleared events and events cleared within 60 minutes; output status is derived as acked, cleared, or active.

```
GET /api/alerts?status=active
→ {"ok":true,"data":[],"ts":"2026-11-03T00:00:00Z"}
```

Source: `dashboard/api/routes/alerts.php:32`

GET /api/rules session

Lists alert rules ordered by rule ID.

Success	`200` · Array<{ruleId,scope,metric,op,threshold,forSeconds,severity,enabled}>
Errors	Common errors only (§2.6).
Tables	`alertRule`

```
GET /api/rules
→ {"ok":true,"data":[],"ts":"2026-11-03T00:00:00Z"}
```

Source: `dashboard/api/routes/alerts.php:98`

POST /api/alerts/ack operator CSRF: Origin + X-Solari-CSRF

Acknowledges one alert event and records the operator and timestamp.

Body	JSON object • `eventId` positive integer **required** — Digits only and nonzero.
Success	`200` · {eventId,ackedBy}
Errors	• `400 bad_request` — eventId missing or not a positive integer. • `404 not_found` — No matching alert event. • `415 unsupported_media_type` — Nonempty body is not application/json.
Tables	`alertEvent`
Notes	Repeated acknowledgement updates the ack timestamp and operator.

```
POST /api/alerts/ack {"eventId":7}
→ {"ok":true,"data":{"eventId":7,"ackedBy":"operator"},"ts":"2026-11-03T00:00:00Z"}
```

Source: `dashboard/api/routes/alerts.php:120`

POST /api/alerts/ack-all operator CSRF: Origin + X-Solari-CSRF

Acknowledges all currently active alert events and returns the affected count.

Success	`200` · {ackedCount,ackedBy}
Errors	Common errors only (§2.6).
Tables	`alertEvent`
Notes	Targets unacked events that are uncleared or were cleared within the past 60 minutes.

```
POST /api/alerts/ack-all
→ {"ok":true,"data":{"ackedCount":2,"ackedBy":"operator"},"ts":"2026-11-03T00:00:00Z"}
```

Source: `dashboard/api/routes/alerts.php:148`

MAINTENANCE WINDOWS · 3

GET /api/maintenance session

Lists maintenance windows filtered by status.

Query
- `status` string default `all` — Allowed active, upcoming, past, all; maximum 500 rows.

Success `200` · Array<{windowId,scope,nodeId,hostFqdn,ipAddr,reason,startsAt,endsAt,status,live,createdBy,createdAt,updatedAt}>

Errors
- `400 bad_request` — Status is outside allowed values.

Tables `maintenanceWindow`

```
GET /api/maintenance?status=active
→ {"ok":true,"data":[],"ts":"2026-11-03T00:00:00Z"}
```

Source: `dashboard/api/routes/maintenance.php:29`

POST /api/maintenance operator CSRF: Origin + X-Solari-CSRF

Schedules a maintenance window for a resolved host or all systems.

Body JSON object
- `host` string — Required unless all is true.
- `all` boolean — Literal true selects all scope.
- `hours` number — Must be positive; schedule from now.
- `from` datetime string — Pair with to if hours is not supplied.
- `to` datetime string — Must be later than start.
- `reason` string — Truncated to 255 characters.

Success `200` · Created window fields including windowId,scope,nodeId,hostFqdn,ipAddr,reason,startsAt,endsAt,status,createdBy,createdAt,updatedAt.

Errors
- `400 bad_request` — Invalid target, time bounds, or mutually exclusive host/all values.
- `415 unsupported_media_type` — Nonempty body is not application/json.

Tables `maintenanceWindow`, `node`, `probeTarget`

Notes Target resolution may consult DNS; date parsing and fractional-hour handling are helper-defined.

```
POST /api/maintenance {"host":"helium","hours":2,"reason":"upgrade"}
→ {"ok":true,"data":{"windowId":4,"scope":"node","status":"scheduled"},"ts":"2026-11-03T00:00:00Z"}
```

Source: `dashboard/api/routes/maintenance.php:77`

POST /api/maintenance/{id}/cancel operator CSRF: Origin + X-Solari-CSRF

Cancels a maintenance window by ID.

Path
- `id` string — Window identifier.

Success `200` · {windowId:int, status:"cancelled"}

Errors
- `400 bad_request` — id is not a positive integer.
- `404 not_found` — No maintenance window with that id.
- `409 bad_request` — Window exists but is not scheduled/active (note: HTTP 409 carrying code bad_request).

Tables `maintenanceWindow (monitoring DB)`

Notes Only scheduled or active windows can be cancelled.

```
POST /api/maintenance/4/cancel
→ {"ok":true,"data":{"windowId":4,"status":"cancelled"},"ts":"2026-11-03T00:00:00Z"}
```

Source: `dashboard/api/routes/maintenance.php:158`

OPIE ANALYSIS · 2

GET /api/opie session

Lists Opie reports, newest first, with optional status filter.

Query
- `status` string default `all` — Allowed all, done, investigating, failed; maximum 500 rows.

Success `200` · Array<{reportId,incidentKey,triggerKind,firstEventId,nodeId,hostFqdn,severity,status,summary,model,durationSec,startedAt,finishedAt}>

Errors
- `400 bad_request` — Status is not allowed.

Tables `opieReport`

```
GET /api/opie?status=done
→ {"ok":true,"data":[],"ts":"2026-11-03T00:00:00Z"}
```

Source: `dashboard/api/routes/opie.php:15`

GET /api/opie/{reportId} session

Returns one Opie report including its analysis markdown.

Path
- `reportId` positive integer string — Must contain digits only and be nonzero.

Success `200` · {reportId,incidentKey,triggerKind,firstEventId,nodeId,hostFqdn,severity,status,summary,analysis,model,durationSec,startedAt,finishedAt}

Errors
- `400 bad_request` — ID is not a positive integer.
- `404 not_found` — No report matches ID.

Tables `opieReport`

```
GET /api/opie/4
→ {"ok":true,"data":{"reportId":4,"incidentKey":"fake","status":"done","analysis":"Report text"},"ts":"2026-11-03T00:00:00Z"}
```

Source: `dashboard/api/routes/opie.php:63`

GET /api/segments session

Lists network segments with node-state rollups.

Success	`200` · Array<{segId,label,cidr,wireless,notes,roll:{up,degraded,down,unknown}}>
Errors	Common errors only (§2.6).
Tables	`segment`, `node`
Notes	Retired nodes count in unknown.

```
GET /api/segments
→ {"ok":true,"data":[],"ts":"2026-11-03T00:00:00Z"}
```

Source: `dashboard/api/routes/topology.php:16`

GET /api/netgear session

Lists managed gear with uplink and attached-node counts.

Success	`200` · Array<{gearId,name,kind,model,segId,ports,uplinkGearId,wireless,mgmtIp,lastSeenAt,attachedNodes}>
Errors	Common errors only (§2.6).
Tables	`networkGear`, `node`, `lldpEdge`
Notes	Attached count is the maximum of uplink-linked node count and distinct LLDP-linked node count.

```
GET /api/netgear
→ {"ok":true,"data":[],"ts":"2026-11-03T00:00:00Z"}
```

Source: `dashboard/api/routes/topology.php:58`

GET /api/topology session

Returns the monitoring or network topology projection.

Query	• `view` string default `monitoring` — Allowed monitoring or network.
Success	`200` · TopologyViews projection with view,nodes,edges keys.
Errors	• `400 bad_request` — View is neither monitoring nor network.
Tables	`node`, `probeTarget`, `probeCurrent`, `networkGear`, `gearInterfaceCurrent`, `lldpEdge`, `segment`
Notes	TopologyViews builders derive health and labels from DB rows; full projection details are extensive.

```
GET /api/topology?view=monitoring
→ {"ok":true,"data":{"view":"monitoring","nodes":[],"edges":[]},"ts":"2026-11-03T00:00:00Z"}
```

Source: `dashboard/api/routes/topology.php:106`

GET /api/gear session

Lists managed gear and current interface rollups.

Success	`200` · Array<{gearId,name,kind,model,segId,ports,uplinkGearId,wireless,mgmtIp,lastSeenAt,ifCount,upCount,totalInRateKbps,totalOutRateKbps}>
Errors	Common errors only (§2.6).
Tables	`networkGear`, `gearInterfaceCurrent`

```
GET /api/gear
→ {"ok":true,"data":[],"ts":"2026-11-03T00:00:00Z"}
```

Source: `dashboard/api/routes/gear.php:22`

GET /api/gear/{id}/interfaces session

Lists current interface metrics for one gear device.

Path	• `id` string — Gear identifier.
Success	`200` · {gearId,interfaces:[{ifIndex,ifName,operStatus,inRateKbps,outRateKbps,inOctets,outOctets,sampledAt}]}
Errors	Common errors only (§2.6).
Tables	`gearInterfaceCurrent`
Notes	An unknown ID returns an empty interfaces array.

```
GET /api/gear/3/interfaces
→ {"ok":true,"data":{"gearId":"3","interfaces":[]},"ts":"2026-11-03T00:00:00Z"}
```

Source: `dashboard/api/routes/gear.php:60`

GET /api/gear/{id}/history session

Returns one gear interface metric time series.

Path
- `id` string — Gear identifier.

Query
- `ifIndex` integer — Applied only when numeric.
- `metric` string default `inRateKbps` — Allowed inRateKbps or outRateKbps.
- `from` datetime string — Parsed through NodeHistoryHelpers.
- `to` datetime string — Parsed through NodeHistoryHelpers.
- `limit` integer default `1000` — Maximum 5000.

Success `200` · {gearId,ifIndex,metric,series:[{ts,value}]}

Errors
- `400 bad_request` — Metric is not whitelisted.

Tables `gearInterfaceHistory`

Notes Nonnumeric ifIndex is ignored and returned as null.

```
GET /api/gear/3/history?ifIndex=1&metric=inRateKbps
→ {"ok":true,"data":{"gearId":"3","ifIndex":1,"metric":"inRateKbps","series":[]},"ts":"2026-11-03T00:00:00Z"}
```

Source: `dashboard/api/routes/gear.php:93`

WEB PUSH · 3

GET /api/push/vapid session

Returns the public application-server VAPID key.

Success `200` · {publicKey}

Errors Common errors only (§2.6).

Notes The source contains a literal public key.

```
GET /api/push/vapid
→ {"ok":true,"data":{"publicKey":"fake-public-key"},"ts":"2026-11-03T00:00:00Z"}
```

Source: `dashboard/api/routes/push.php:14`

POST /api/push/subscribe operator CSRF: Origin + X-Solari-CSRF

Upserts a push subscription for the authenticated operator.

Body JSON object
- `endpoint` URL string **required** — Trimmed, valid URL, max 512 bytes.
- `keys.p256dh` string **required** — Trimmed, max 255 bytes.
- `keys.auth` string **required** — Trimmed, max 255 bytes.

Success `200` · {subscribed:true}

Errors
- `400 bad_request` — Endpoint or keys are missing or invalid.
- `415 unsupported_media_type` — Nonempty body is not application/json.

Tables `push_subscriptions`

Notes Upsert refreshes keys, user agent, and lastSeenAt; user agent is truncated to 255 bytes.

```
POST /api/push/subscribe {"endpoint":"https://push.example/fake","keys":{"p256dh":"fake","auth":"fake"}}
→ {"ok":true,"data":{"subscribed":true},"ts":"2026-11-03T00:00:00Z"}
```

Source: `dashboard/api/routes/push.php:19`

DELETE /api/push/subscribe operator CSRF: Origin + X-Solari-CSRF

Removes a push subscription by endpoint.

Body JSON object
- `endpoint` string **required** — Trimmed and must be nonempty.

Success `200` · {unsubscribed:true}

Errors
- `400 bad_request` — Endpoint is empty.
- `415 unsupported_media_type` — Nonempty body is not application/json.

Tables `push_subscriptions`

Notes Deletes all rows matching endpoint.

```
DELETE /api/push/subscribe {"endpoint":"https://push.example/fake"}
→ {"ok":true,"data":{"unsubscribed":true},"ts":"2026-11-03T00:00:00Z"}
```

Source: `dashboard/api/routes/push.php:48`

RETIRED EVENT STREAM · 1

GET /api/stream session

Immediately ends the legacy event-stream request with HTTP 204.

Success `204` · No body; exits after setting no-store cache header.

Errors Common errors only (§2.6).

Notes Old EventSource clients are told to stop reconnecting; dashboard state uses polling.

```
GET /api/stream
→ HTTP 204 No Content
```

Source: `dashboard/api/routes/stream.php:11`

3.4 Status panel · 4 routes

These four routes form the contract between the dashboard and the LED status panel daemon. The panel principal is a dedicated local account with deliberately narrow rights (§7.2). The binary frame format the daemon sends to the device is in §7.3.

GET /api/panel session

Builds the panel dashboard snapshot, including systems, pools, alerts, panel state, commands, and screen configuration where available.

Success	`200` · Panel snapshot payload assembled from system, pool, alert, probe, gear, panel-state, command, and screen-config queries.
Errors	Common errors only (§2.6).
Tables	`node`, `asset`, `hostCurrent`, `probeTarget`, `probeCurrent`, `alertEvent`, `alertRule`, `pool`, `panelState`, `panelCommand`, `networkGear`, `gearInterfaceCurrent`, `panelScreenConfig`
Notes	This is a read-only consistent-snapshot transaction except that service principals trigger expiry of pending commands older than 120 seconds. Full response includes additional keys defined in the handler.

```
GET /api/panel
→ {"ok":true,"data":{"systems":[],"pools":[],"alerts":{"info":0,"warn":0,"crit":0}},"ts":"2026-11-03T00:00:00Z"}
```

Source: `dashboard/api/routes/panel.php:308`

POST /api/panel/command operator CSRF: Origin + X-Solari-CSRF

Queues a validated physical-panel command for the panel service to consume.

Body	JSON object • `kind` integer 1–9 **required** — 1 SETTHEME, 2 SETSCREEN, 3 BRIGHTNESS, 4 AUTOBRIGHT, 5 ACKALARM, 6 SETDWELL, 7 SLEEP, 8 SCREENEN, 9 SCREENWT. • `arg` integer 0–255 **required** — Per kind: 1 ≤3; 2 ≤2; 3 ≤100; 4/5 ignored; 6 ∈ {3,6,30}; 7 ≤1; 8 (screenIdx<<1)|enabled ≤23; 9 (screenIdx<<3)|weightCode, screenIdx ≤11, weightCode ≤5.
Success	`200` · {cmdId}
Errors	• `403 forbidden` — Principal is panel service or lacks operator/admin role. • `409 queue_full` — 16 or more pending commands remain. • `415 unsupported_media_type` — Nonempty body is not application/json. • `400 bad_request` — JSON/body or panel command validation fails.
Tables	`panelCommand`
Notes	Pending entries older than 120 seconds are expired before cap check; insertion is transaction-protected.

```
POST /api/panel/command {"kind":1,"arg":0}
→ {"ok":true,"data":{"cmdId":12},"ts":"2026-11-03T00:00:00Z"}
```

Source: `dashboard/api/routes/panel.php:792`

POST /api/panel/state session panel-service bridge

Stores the panel service state report and marks commands through its reported command ID as applied.

Body	JSON object • `theme` integer **required** — Range 0–3. • `screen` integer **required** — Range 0–2. • `brightnessPct` integer **required** — Range 0–100. • `autoBright` integer **required** — Range 0/1. • `sleeping` integer **required** — Range 0/1. • `alarmArmed` integer **required** — Range 0/1. • `alarmAcked` integer **required** — Range 0/1. • `dwellSec` integer **required** — Range 0–255. • `ackedEpisodeId` integer **required** — Range 0–4294967295. • `lastCmdId` integer **required** — Range 0–4294967295.
Success	`200` · {panelState:{theme,screen,brightnessPct,autoBright,sleeping,alarmArmed,alarmAcked,dwellSec,ackedEpisodeId,lastCmdId,reportedAt}}
Errors	• `403 forbidden` — Principal is not the local panel service. • `415 unsupported_media_type` — Nonempty body is not application/json. • `400 bad_request` — JSON/body or panel state validation fails.
Tables	`panelState`, `panelCommand`
Notes	Late command confirmation can mark recently expired commands applied for up to 15 seconds.

```
POST /api/panel/state
{"theme":1,"screen":0,"brightnessPct":50,"autoBright":1,"sleeping":0,"alarmArmed":1,"alarmAcked":0,"dwellSec":6,"ackedEpisodeId":0,"lastCmdId":12}
→ {"ok":true,"data":{"panelState":
{"theme":"dark","screen":1,"brightnessPct":50,"autoBright":false,"sleeping":false,"alarmArmed":false,"alarmAcked":false,"dwellSec":30,"ackedEpisodeId":0,"lastCmdId":12
,"reportedAt":"2026-11-03T00:00:00Z"}},"ts":"2026-11-03T00:00:00Z"}
```

Source: `dashboard/api/routes/panel.php:846`

POST `/api/panel/config` session panel-service bridge

Upserts the panel service screen-enable and weight configuration report when its table exists.

Body	JSON object • `screenCfg` array[12] of {enabled:0|1, weightCode:0–5} **required** — Exactly 12 entries (screenIdx = theme×3 + slot). • `flags` integer **required** — 0–255.
Success	`200` · {panelScreenConfig:{screenCfg,flags,reportedAt}|null}
Errors	• `403 forbidden` — Principal is not the local panel service. • `415 unsupported_media_type` — Nonempty body is not application/json. • `400 bad_request` — JSON/body or panel config validation fails.
Tables	`panelScreenConfig`
Notes	Returns a successful null payload when the optional table is absent.

```
POST /api/panel/config {"screenCfg":[{"enabled":1,"weightCode":2}, … 12 entries],"flags":0}
→ {"ok":true,"data":{"panelScreenConfig":null},"ts":"2026-11-03T00:00:00Z"}
```

Source: `dashboard/api/routes/panel.php:909`

3.5 Fleet control, discovery & enrollment · 40 routes

Fleet routes cover the asset registry, discovery and adoption, enrolment approval, node configuration, and the control actions under `/api/control/`. Deploy, provision, fleet-provision, fleet-image, decommission and retire go through the queued `REQUEST_SUBMIT` path (§4.2). They answer `200` with a status, a log handle and, if the call outlasts the synchronous wait, a `request` id to poll. Survey and criticality changes are direct, synchronous verbs.

DISCOVERY · 7

GET `/api/discovery` session

Lists discovered devices for a selected status and decorates each with a best-effort topology neighbor and, when available, an inactive-asset tombstone.

Query	• `status` string default `new` — Allowed values: new, ignored, adopting, adopted, all; anything else returns bad_request (400).
Success	`200` · [{discId,host,ip,kind,via,services,segId,arch,mac,vendor,osName,deviceRole,sysDescr,enrichedAt,mdnsName,mdnsServices,osGuess,openPorts,banners,nmapEnrichedAt,seenCount,firstSeen
Errors	• `400 bad_request` — status is outside the listed values.
Tables	`discovered`, `networkGear`, `lldpEdge`, `node`, `asset`, `information_schema.COLUMNS`
Notes	Ordered by lastSeenAt descending; tombstone is omitted unless a matching inactive asset exists, and neighbor is null when no host cross-reference exists.

```
GET /api/discovery?status=new
→ {"ok":true,"data":[{"discId":"41","host":"sensor-1","ip":"10.0.0.41","status":"new","services":[],"mdnsServices":[],"openPorts":[],"neighbor":null}],"ts":"2026-11-03T12:00:00Z"}
```

Source: `dashboard/api/routes/discovery.php:22`

POST `/api/discovery/scan` operator CSRF: Origin + X-Solari-CSRF

Submits an operator-requested bounded IPv4 discovery scan and returns either an admission-pending request id or a queued scan id.

Body	JSON object • `cidr` string **required** — Must be an IPv4 CIDR wholly inside RFC1918, loopback, or link-local space, with prefix 1–32 and at most 4096 usable hosts. • `ports` string — Comma-separated 1–64 unique decimal TCP ports, 1–65535; empty string uses the server default port set.
Success	`202` · {cidr,status:'admission_pending',requestId} or {cidr,scanId,hostsTotal,status:'queued'}
Errors	• `400 bad_request` — Invalid CIDR, ports, or Idempotency-Key. • `400 control_bad_request` — Framed operator request exceeds the bridge request limit. • `502 control_error` — Bridge omits a request id or scan id, or a queued admission result is malformed. • `502 control_unavailable` — Operator bridge transport is unreachable, incomplete, or cannot accept the full request. • `403 forbidden` — The operator bridge denies authorization for the queued operation.
ctl verb	`DISCOVER via SolariCtl::callQueued (REQUEST_SUBMIT, then REQUEST_GET polling)`
Notes	Optional Idempotency-Key header must be 8–128 characters from [A-Za-z0-9._-]. A request still pending after four seconds returns requestId for asynchronous polling; hostsTotal can be null.

```
POST /api/discovery/scan with {"cidr":"10.0.0.0/24","ports":"22,443"}
→ {"ok":true,"data":{"cidr":"10.0.0.0/24","scanId":58,"hostsTotal":254,"status":"queued"},"ts":"2026-11-03T12:00:00Z"}
```

Source: `dashboard/api/routes/discovery_mut.php:24`

GET `/api/discovery/scan-requests/{requestId}` operator

Polls an operator's queued discovery admission request and returns its state and terminal scan details when available.

Path	• `requestId` positive integer — Must match 1–19 digits and cannot start with zero.
Success	`200` · {requestId,state,scanId?,hostsTotal?,error?}
Errors	• `400 bad_request` — requestId is invalid. • `502 control_error` — A done request contains a malformed discovery admission result. • `502 control_error` — Bridge omits request data or returns an invalid result. • `502 control_unavailable` — Operator bridge transport fails. • `403 forbidden` — The operator bridge denies request polling.
ctl verb	`REQUEST_GET via SolariCtl::poll`
Notes	A done state includes scanId and hostsTotal after strict result parsing; a failed state includes error. Other states return requestId and state only.

```
GET /api/discovery/scan-requests/91
→ {"ok":true,"data":{"requestId":91,"state":"pending"},"ts":"2026-11-03T12:00:00Z"}
```

Source: `dashboard/api/routes/discovery_mut.php:62`

GET /api/discovery/scan-jobs operator

Lists the ten newest durable discovery scan jobs requested by the authenticated operator.

Success `200` · Array<{scanId,cidr,ports,status,hostsTotal,hostsDone,hostsFound,error,createdAt}>

Errors
- `503 service_unavailable` — The local database connection is unavailable.

Tables `discoveryScanJob`

Notes Ten newest jobs for the authenticated requester (requestedBy), scanId descending. Database failures surface as 500 internal_error.

```
GET /api/discovery/scan-jobs
→ {"ok":true,"data":[{"scanId":58,"cidr":"10.0.0.0/24","ports":"22,443","status":"running","hostsTotal":254,"hostsDone":23,"hostsFound":4,"error":"","createdAt":"2026-11-03 12:00:00"}],"ts":"2026-11-03T12:00:00Z"}
```

Source: `dashboard/api/routes/discovery_mut.php:84`

GET /api/discovery/scans/{scanId} operator

Fetches status and host counts for a discovery scan through the control bridge.

Path
- `scanId` positive integer — Must match 1–19 digits with no leading zero; invalid values return 400 bad_request.

Success `200` · {scanId,status,hostsDone,hostsTotal,hostsFound,errorCode,error}

Errors
- `400 bad_request` — scanId is invalid
- `400 control_error` — bridge rejects the status request
- `403 forbidden` — bridge refuses authorization
- `502 control_unavailable` — bridge is unreachable or transport fails

ctl verb `DISCOVER_STATUS via SolariCtl::call`

Notes Requires operator authorization per route registration. Bridge omissions default to unknown status, zero counts/code, and empty error.

```
GET /api/discovery/scans/17
→ {"ok":true,"data":{"scanId":17,"status":"complete","hostsDone":12,"hostsTotal":12,"hostsFound":3,"errorCode":0,"error":""},"ts":"2026-11-03T12:00:00Z"}
```

Source: `dashboard/api/routes/discovery_mut.php:107`

POST /api/discovery/{discId}/adopt operator CSRF: Origin + X-Solari-CSRF

Adopts a discovered candidate through the bridge and mirrors its IP/name into SoR when the discovery row exists.

Path
- `discId` positive integer — Digits only and not zero; invalid values return 400 bad_request.

Body JSON object
- `displayName` string — Nonempty strings are truncated to 120 bytes and sent as name.
- `class` string — Accepted values: server, appliance, network, iot, host, other; other values are ignored.
- `poolId` integer — Integer or digit string; 0 means no pool, and a nonzero missing pool returns 400 bad_request.
- `notes` string — Nonempty strings are truncated to 240 bytes.
- `tags` array — Array is JSON-encoded and forwarded.
- `heartbeat` boolean — Defaults true; any falsey value disables it.
- `services` array — Accepts integer/digit-string ports or objects with port; forwards valid 1–65535 ports when any remain.

Success `200` · {discId,assetId,status}

Errors
- `400 bad_request` — discId invalid or nonzero poolId does not exist
- `400 unsupported_media_type` — body is nonempty and not application/json
- `400 control_error` — bridge rejects ADOPT
- `403 forbidden` — bridge refuses authorization
- `502 control_unavailable` — bridge transport fails

ctl verb `ADOPT via SolariCtl::call`

Tables `pool (read)`, `discovered (read)`, `entities (possible write via Sor::upsertHost)`, `ip_addresses (possible write via Sor::upsertHost)`

Notes Bridge performs adoption; PHP mirrors SoR only if discovered row is found. SoR mirror failure is logged and does not fail the response.

```
POST /api/discovery/42/adopt {"displayName":"edge-1","heartbeat":true}
→ {"ok":true,"data":{"discId":"42","assetId":81,"status":"adopted"},"ts":"2026-11-03T12:00:00Z"}
```

Source: `dashboard/api/routes/discovery_mut.php:127`

POST /api/discovery/{discId}/ignore operator CSRF: Origin + X-Solari-CSRF

Marks a discovered candidate ignored through the control bridge.

Path
- `discId` positive integer — Digits only and not zero; invalid values return 400 bad_request.

Success `200` · {discId,status}

Errors
- `400 bad_request` — discId invalid
- `400 control_error` — bridge rejects IGNORE
- `403 forbidden` — bridge refuses authorization
- `502 control_unavailable` — bridge transport fails

ctl verb `IGNORE via SolariCtl::call`

Notes Optional operator name is forwarded as op when available.

```
POST /api/discovery/42/ignore
→ {"ok":true,"data":{"discId":"42","status":"ignored"},"ts":"2026-11-03T12:00:00Z"}
```

Source: `dashboard/api/routes/discovery_mut.php:182`

GET /api/enrollments session

Lists enrollment records, optionally filtered by status, newest request first.

Query	• `status` string default `all statuses` — Allowed: token, pending, approved, rejected, expired; other nonempty strings return bad_request.
Success	`200` · Array of {enrId,host,ip,role,certFp,status,requestedAt,decidedAt,decidedBy}; csrPem is omitted.
Errors	• `400 bad_request` — Nonempty status is outside the allowed values.
Tables	`enrollment`
Notes	No pagination or write side effects are shown.

```
GET /api/enrollments?status=pending
→ {"ok":true,"data":[{"enrId":12,"host":"node1","ip":"192.0.2.10","role":"server","certFp":"ab12","status":"pending","requestedAt":"2026-01-
01T00:00:00Z","decidedAt":null,"decidedBy":null}],"ts":"2026-01-01T00:00:00Z"}
```

Source: `dashboard/api/routes/provisioning.php:45`

POST /api/enrollments public

Submits a first-boot enrollment token, node identity, certificate fingerprint, and CSR to the bridge.

Body	JSON object • `token` string **required** — Exactly 64 hexadecimal characters. • `role` string **required** — One of client, monitor, server. • `host` string **required** — Nonempty; maximum 253 bytes. • `certFp` string **required** — Exactly 32 colon-separated hexadecimal byte pairs. • `csrPem` string **required** — Nonempty; maximum 4096 bytes.
Success	`200` · {status} or {status,nodeId,cert,ca} when status is approved
Errors	• `413 payload_too_large` — declared Content-Length exceeds 16 KiB • `400 bad_request` — enrollment fields fail validation • `415 unsupported_media_type` — nonempty body is not application/json • `400 control_error` — bridge rejects submission • `403 forbidden` — bridge refuses authorization • `502 control_unavailable` — bridge transport fails • `502 control_error` — approved reply lacks nodeId, cert, or ca
ctl verb	`ENROLL_SUBMIT via SolariCtl::call`
Notes	Approved response includes certificate material. PHP does not persist or sign the CSR.

```
POST /api/enrollments {"token":"aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa","role":"client","host":"edge-
1","certFp":"aa:aa:aa:aa:aa:aa:aa:aa:aa:aa:aa:aa:aa:aa:aa:aa:aa:aa:aa:aa","csrPem":"-----BEGIN CERTIFICATE REQUEST-----…"}
→ {"ok":true,"data":{"status":"pending"},"ts":"2026-11-03T12:00:00Z"}
```

Source: `dashboard/api/routes/enrollments_mut.php:20`

POST /api/enrollments/ack public

Acknowledges enrollment certificate installation using the enrollment token.

Body	JSON object • `token` string **required** — Exactly 64 hexadecimal characters.
Success	`200` · {status}
Errors	• `400 bad_request` — token invalid • `415 unsupported_media_type` — nonempty body is not application/json • `400 control_error` — bridge rejects acknowledgement • `403 forbidden` — bridge refuses authorization • `502 control_unavailable` — bridge transport fails
ctl verb	`ENROLL_ACK via SolariCtl::call`
Notes	The code comments specify the same token can repeat the exchange until acknowledgement succeeds.

```
POST /api/enrollments/ack {"token":"aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa"}
→ {"ok":true,"data":{"status":"acknowledged"},"ts":"2026-11-03T12:00:00Z"}
```

Source: `dashboard/api/routes/enrollments_mut.php:56`

POST /api/enrollments/{enrId}/approve operator CSRF: Origin + X-Solari-CSRF

Approves a pending enrollment after explicit confirmation and queues certificate signing.

Path	• `enrId` positive integer — Digits only and not zero; invalid values return 400 bad_request.
Body	JSON object • `confirm` boolean **required** — Must be exactly true; otherwise 409 confirm_required.
Success	`200` · {enrId,status,decidedBy} or {enrId,status,decidedBy,request} when queued
Errors	• `400 bad_request` — enrId invalid or bridge rejects action • `409 confirm_required` — confirm is not true • `403 forbidden` — bridge refuses authorization • `502 control_unavailable` — bridge transport fails
ctl verb	`APPROVE via SolariCtl::callQueued (REQUEST_SUBMIT, polling)`
Notes	Operator required. Queued response carries request ID; otherwise response reports approved.

```
POST /api/enrollments/73/approve {"confirm":true}
→ {"ok":true,"data":{"enrId":"73","status":"approved","decidedBy":"operator"},"ts":"2026-11-03T12:00:00Z"}
```

Source: `dashboard/api/routes/enrollments_mut.php:67`

POST /api/enrollments/{enrId}/reject

operator | CSRF: Origin + X-Solari-CSRF

Rejects a pending enrollment through the privileged request queue.

Path

- `enrId` positive integer — Digits only and not zero; invalid values return 400 bad_request.

Success

`200` · {enrId,status,decidedBy} or {enrId,status,decidedBy,request} when queued

Errors

- `400 bad_request` — enrId invalid or bridge rejects action
- `401 unauthorized` — no named session user
- `403 forbidden` — bridge refuses authorization
- `502 control_unavailable` — bridge transport fails

ctl verb

`REJECT via SolariCtl::callQueued (REQUEST_SUBMIT, polling)`

Notes

A named session user is required; no operator-role check appears in this handler.

```
POST /api/enrollments/73/reject
→ {"ok":true,"data":{"enrId":"73","status":"rejected","decidedBy":"operator"},"ts":"2026-11-03T12:00:00Z"}
```

Source: `dashboard/api/routes/enrollments_mut.php:91`

CONTROL ACTIONS · 10

POST /api/control/provision

operator | CSRF: Origin + X-Solari-CSRF

Queues a node provisioning operation and returns its current status.

Body

JSON object

- `nodeId` positive integer **required** — Integer or digit string, not zero.
- `buildId` integer — Integer or digit string; invalid values are ignored.
- `configEpoch` integer — Integer or digit string; invalid values are ignored.
- `configBlob` any JSON value — String forwarded directly; other values JSON-encoded when encoding succeeds.

Success

`200` · {nodeId,status} or {nodeId,status,request} when queued

Errors

- `400 bad_request` — nodeId invalid, idempotency key invalid, or bridge rejects queue submission
- `403 forbidden` — bridge refuses authorization
- `502 control_unavailable` — bridge transport fails

ctl verb

`PROVISION via SolariCtl::callQueued (REQUEST_SUBMIT, polling)`

Notes

Uses optional Idempotency-Key. Queued response contains request for later polling.

```
POST /api/control/provision {"nodeId":12,"buildId":4}
→ {"ok":true,"data":{"nodeId":"12","status":"provisioning"},"ts":"2026-11-03T12:00:00Z"}
```

Source: `dashboard/api/routes/control.php:21`

POST /api/control/decommission

operator | CSRF: Origin + X-Solari-CSRF

Requests a confirmed decommission with a nonempty wipe scope through the privileged queue.

Body

JSON object

- `nodeId` positive integer **required** — Integer or digit string, not zero.
- `confirm` boolean **required** — Must be exactly true; otherwise 409 confirm_required.
- `wipeScope` array of strings **required** — Nonempty subset of config, certs, spool, logs, unit; unknown values return 400 bad_request.

Success

`200` · {nodeId,wipeScope,decidedBy,status} and request when queued

Errors

- `400 bad_request` — nodeId, wipeScope, or idempotency key invalid
- `409 confirm_required` — confirm is not true
- `503 control_unavailable` — PING does not report decommissionInline=1
- `400 control_error` — bridge rejects queue submission
- `403 forbidden` — bridge refuses authorization
- `502 control_unavailable` — bridge transport fails

ctl verb

`DECOMMISSION via SolariCtl::callQueued (REQUEST_SUBMIT, polling); capability check via SolariCtl::call PING`

Notes

Scope is encoded as a bitmask for the bridge. Queue consumer handles the one-time confirmation token in memory.

```
POST /api/control/decommission {"nodeId":12,"confirm":true,"wipeScope":["config","certs"]}
→ {"ok":true,"data":{"nodeId":"12","wipeScope":["config","certs"],"decidedBy":"operator","status":"awaiting_node"},"ts":"2026-11-03T12:00:00Z"}
```

Source: `dashboard/api/routes/control.php:60`

POST /api/nodes/{nodeId}/retire

operator | CSRF: Origin + X-Solari-CSRF

Checks that a node exists, then requests confirmed retirement through the privileged queue.

Path

- `nodeId` positive integer — Integer or digit string, not zero.

Body

JSON object

- `confirm` boolean **required** — Must be exactly true; otherwise 409 confirm_required.

Success

`200` · {nodeId,decidedBy,status} and request when queued

Errors

- `400 bad_request` — nodeId invalid, idempotency key invalid, or bridge rejects queue submission
- `404 not_found` — node row does not exist
- `409 confirm_required` — confirm is not true
- `403 forbidden` — bridge refuses authorization
- `502 control_unavailable` — bridge transport fails

ctl verb

`RETIRE via SolariCtl::callQueued (REQUEST_SUBMIT, polling)`

Tables

`node (read)`

Notes

Operator required. Uses optional Idempotency-Key; queued response avoids claiming retirement is complete.

```
POST /api/nodes/12/retire {"confirm":true}
→ {"ok":true,"data":{"nodeId":"12","decidedBy":"operator","status":"retired"},"ts":"2026-11-03T12:00:00Z"}
```

Source: `dashboard/api/routes/control.php:101`

POST /api/nodes/{nodeId}/criticality

operator | CSRF: Origin + X-Solari-CSRF

Sets a node's criticality tier through the control bridge.

Path

- `nodeId` positive integer — Integer or digit string, not zero.

Body

JSON object

- `tier` integer **required** — JSON integer from 0 through 4 inclusive.

Success

`200` · {nodeId,criticality}

Errors

- `400 bad_request` — nodeId or tier invalid, or bridge rejects action
- `404 not_found` — node row does not exist
- `403 forbidden` — bridge refuses authorization
- `502 control_unavailable` — bridge transport fails

ctl verb

`CRIT_SET via SolariCtl::call`

Tables

`node (read)`

Notes

Operator required; bridge arguments use node, tier, and op.

```
POST /api/nodes/12/criticality {"tier":3}
→ {"ok":true,"data":{"nodeId":"12","criticality":3},"ts":"2026-11-03T12:00:00Z"}
```

Source: `dashboard/api/routes/control.php:125`

POST /api/control/survey

operator | CSRF: Origin + X-Solari-CSRF

Broadcasts a fleet survey request through the control bridge.

Body

JSON object

- `scope` string — Comment documents informational scope=all; handler does not validate or use it.

Success

`200` · {survey}

Errors

- `415 unsupported_media_type` — nonempty body is not application/json
- `400 control_error` — bridge rejects SURVEY
- `403 forbidden` — bridge refuses authorization
- `502 control_unavailable` — bridge transport fails

ctl verb

`SURVEY via SolariCtl::call`

Notes

Handler parses but ignores the JSON body; optional operator name is forwarded as op when available.

```
POST /api/control/survey {"scope":"all"}
→ {"ok":true,"data":{"survey":"sent"},"ts":"2026-11-03T12:00:00Z"}
```

Source: `dashboard/api/routes/control.php:143`

POST /api/control/deploy

operator | CSRF: Origin + X-Solari-CSRF

Queues a remote client-agent deployment for a host and returns deployment status details.

Body JSON object

- `host` string **required** — Pattern [A-Za-z0-9._@-], 1–120 chars.
- `server` string — Forwarded only if it matches tls+tcp:// followed by [A-Za-z0-9._:-]+.
- `arch` string — Forwarded only for arm64, arm32, amd64, or x86_64.
- `fqdn` string — Forwarded only if [A-Za-z0-9._-], 1–160 chars.

Success `200` · {host,status:"deploying",log,request?}; request is present when the bridge call remains queued past its wait budget.

Errors

- `400 bad_request` — host is invalid, Idempotency-Key is malformed, or bridge request exceeds its size limit.
- `415 unsupported_media_type` — non-empty body is not application/json.
- `502 control_unavailable` — bridge cannot be reached or does not provide a complete reply.
- `400 control_error` — bridge rejects DEPLOY or queued submission.
- `403 forbidden` — bridge reports an operator/authorization refusal.
- `503 control_error` — bridge reports an active-server/lease failure.
- `500 internal_error` — uncaught exception occurs.

ctl verb `DEPLOY via SolariCtl::callQueued (which uses REQUEST_SUBMIT);`

Notes Effective role operator; CSRF origin+session-token. Handler dashboard/api/routes/control.php:160-183. Deployment is privileged and asynchronous-capable; optional server/arch/fqdn values failing their patterns are omitted.

```
POST /api/control/deploy
→ {"ok":true,"data":{"host":"ops@node-7","status":"deploying","log":"/run/deploy-node-7.log","request":"42"},"ts":"2026-11-03T12:00:00Z"}
```

Source: `dashboard/api/routes/control.php:160`

GET /api/control/deploy

session

Reads the deployment log for a host and reports whether deployment appears running, done, or failed.

Query

- `host` string **required** — Pattern [A-Za-z0-9._@-], 1–120 chars.

Success `200` · {host,log,running,done,failed}

Errors

- `400 bad_request` — host is absent or invalid.
- `500 internal_error` — uncaught exception occurs.

Notes Effective role session; CSRF none. Handler dashboard/api/routes/control.php:187-205. Reads deploy-<sanitized-host>.log beside SOLARI_CTL_SOCK, defaulting to /run/solari; unreadable/missing log is treated as empty.

```
GET /api/control/deploy?host=ops%40node-7
→ {"ok":true,"data":{"host":"ops@node-7","log":"[deploy] done.","running":false,"done":true,"failed":false},"ts":"2026-11-03T12:00:00Z"}
```

Source: `dashboard/api/routes/control.php:187`

POST /api/control/fleet-provision

operator | CSRF: Origin + X-Solari-CSRF

Queues bare-metal fleet provisioning for a target using the requested distro, architecture, and hostname.

Body JSON object

- `target` string **required** — Pattern [A-Za-z0-9._:@-], 1–120 chars.
- `distro` string **required** — One of debian, ubuntu, opensuse, raspios.
- `arch` string **required** — One of x86_64, arm64, arm32.
- `hostname` string **required** — Pattern [A-Za-z0-9._-], 1–160 chars.
- `profile` string — Forwarded if pattern [A-Za-z0-9._-], 1–48 chars.
- `role` string — Forwarded if pattern [A-Za-z0-9._-], 1–32 chars.
- `server` string — Forwarded if it matches tls+tcp:// followed by [A-Za-z0-9._:-]+.
- `ip` string — Forwarded only if PHP validates it as an IP address.

Success `200` · {hostname,status:"provisioning",log,request?}; request is present when the bridge call remains queued past its wait budget.

Errors

- `400 bad_request` — target, distro, arch, hostname, or Idempotency-Key fails validation, or bridge request exceeds its size limit.
- `415 unsupported_media_type` — non-empty body is not application/json.
- `502 control_unavailable` — bridge cannot be reached or does not provide a complete reply.
- `400 control_error` — bridge rejects FLEET_PROVISION or queued submission.
- `403 forbidden` — bridge reports an operator/authorization refusal.
- `503 control_error` — bridge reports an active-server/lease failure.
- `500 internal_error` — uncaught exception occurs.

ctl verb `FLEET_PROVISION via SolariCtl::callQueued (which uses REQUEST_SUBMIT);`

Notes Effective role operator; CSRF origin+session-token. Handler dashboard/api/routes/control.php:212-250. Optional profile/role/server/ip values that fail their filters are omitted.

```
POST /api/control/fleet-provision
→ {"ok":true,"data":{"hostname":"pi-4","status":"provisioning","log":"/run/provision-pi-4.log","request":"43"},"ts":"2026-11-03T12:00:00Z"}
```

Source: `dashboard/api/routes/control.php:212`

POST /api/control/fleet-image

operator | CSRF: Origin + X-Solari-CSRF

Queues a bootable fleet image build for an ARM host.

Body — JSON object

- `hostname` string **required** — Pattern [A-Za-z0-9._-], 1–160 chars.
- `arch` string **required** — One of arm64 or arm32.
- `distro` string — Forwarded if pattern [A-Za-z0-9._-], 1–24 chars; no enum check.
- `profile` string — Forwarded if pattern [A-Za-z0-9._-], 1–48 chars.
- `role` string — Forwarded if pattern [A-Za-z0-9._-], 1–32 chars.
- `server` string — Forwarded if it matches tls+tcp:// followed by [A-Za-z0-9._:-]+.
- `ip` string — Forwarded only if PHP validates it as an IP address.

Success — `200` · {hostname,status:"imaging",log,request?}; request is present when the bridge call remains queued past its wait budget.

Errors

- `400 bad_request` — hostname, arch, or Idempotency-Key fails validation, or bridge request exceeds its size limit.
- `415 unsupported_media_type` — non-empty body is not application/json.
- `502 control_unavailable` — bridge cannot be reached or does not provide a complete reply.
- `400 control_error` — bridge rejects FLEET_IMAGE or queued submission.
- `403 forbidden` — bridge reports an operator/authorization refusal.
- `503 control_error` — bridge reports an active-server/lease failure.
- `500 internal_error` — uncaught exception occurs.

ctl verb — `FLEET_IMAGE via SolariCtl::callQueued (which uses REQUEST_SUBMIT);`

Notes — Effective role operator; CSRF origin+session-token. Handler dashboard/api/routes/control.php:255-284.

```
POST /api/control/fleet-image
→ {"ok":true,"data":{"hostname":"pi-4","status":"imaging","log":"/run/image-pi-4.log","request":"44"},"ts":"2026-11-03T12:00:00Z"}
```

Source: `dashboard/api/routes/control.php:255`

GET /api/control/fleet-provision

session

Reads a provisioning or image-build log for a hostname and reports its apparent state.

Query

- `hostname` string **required** — Pattern [A-Za-z0-9._-], 1–160 chars.
- `image` string default `absent/false selects provision; truthy selects image` — Any truthy query value selects image; no narrower validation.

Success — `200` · {hostname,kind,log,running,done,failed}

Errors

- `400 bad_request` — hostname is absent or invalid.
- `500 internal_error` — uncaught exception occurs.

Notes — Effective role session; CSRF none. Handler dashboard/api/routes/control.php:288-311. Reads image- or provision-<hostname>.log beside SOLARI_CTL_SOCK; missing/unreadable log is empty.

```
GET /api/control/fleet-provision?hostname=pi-4&image=1
→ {"ok":true,"data":{"hostname":"pi-4","kind":"image","log":"[fleet] done.","running":false,"done":true,"failed":false},"ts":"2026-11-03T12:00:00Z"}
```

Source: `dashboard/api/routes/control.php:288`

CONFIGURATION & RULES · 6

GET /api/config

session

Returns the current global configuration view assembled from defaults and persisted control-plane state.

Success — `200` · {schedule:{sampleIntervalSec,watchdogIntervalSec},probe:{roundIntervalSec,probesPerRound,replFactor,gossipIntervalSec},retention:{historyDays,partitionByMonth},lease:{renewSec,ttlSec,epoch?},ingest:{ports:{ingest,survey,pub},tls},ca:{issuer,enroll,certTtlDays},autoDiscover,autoEnroll,epoch?}

Errors

- `500 internal_error` — uncaught exception occurs outside the best-effort globalConfig/serverLease reads.

Tables — `globalConfig`, `serverLease`

Notes — Effective role session; CSRF none. Handler dashboard/api/routes/config.php:26-28. Missing tables/rows or caught DB exceptions preserve defaults.

```
GET /api/config
→ {"ok":true,"data":{"schedule":{"sampleIntervalSec":15,"watchdogIntervalSec":5},"probe":
{"roundIntervalSec":30,"probesPerRound":5,"replFactor":2,"gossipIntervalSec":20},"retention":{"historyDays":90,"partitionByMonth":true},"lease":
{"renewSec":5,"ttlSec":15},"epoch":3},"ts":"2026-11-03T12:00:00Z"}
```

Source: `dashboard/api/routes/config.php:26`

POST /api/config

operator | CSRF: Origin + X-Solari-CSRF

Deep-merges and validates a partial global config, then queues the fleet-wide CONFIG_SET update.

Body

JSON object

- `config fields` object — Partial or full config keys are merged without a key whitelist; numeric settings checked: sampleIntervalSec 5–120, watchdogIntervalSec 1–30, roundIntervalSec 10–120, probesPerRound 1–20, replFactor 1–5, gossipIntervalSec 5–60, historyDays 7–365, renewSec 1–30, ttlSec 5–60. Non-numeric/out-of-range returns bad_request.

Success

`200` · Merged config document with derived epoch key (integer from bridge reply, or null when absent).

Errors

- `400 bad_request` — config validation fails, Idempotency-Key is malformed, or bridge request exceeds its size limit.
- `415 unsupported_media_type` — non-empty body is not application/json.
- `502 control_unavailable` — bridge cannot be reached or does not provide a complete reply.
- `400 control_error` — bridge rejects CONFIG_SET or queued submission.
- `403 forbidden` — bridge reports an operator/authorization refusal.
- `503 control_error` — bridge reports an active-server/lease failure.
- `500 internal_error` — uncaught exception occurs.

ctl verb

`CONFIG_SET via SolariCtl::callQueued (which uses REQUEST_SUBMIT);`

Tables

`globalConfig`, `serverLease`

Notes

Effective role operator; CSRF origin+session-token. Handler dashboard/api/routes/config.php:34-45. PHP does not persist the global document; bridge performs persistence. Although the route does not call ctrl_idem_key directly, it passes that helper to callQueued.

```
POST /api/config
→ {"ok":true,"data":{"schedule":{"sampleIntervalSec":20,"watchdogIntervalSec":5},"epoch":4},"ts":"2026-11-03T12:00:00Z"}
```

Source: `dashboard/api/routes/config.php:34`

GET /api/nodes/{id}/config

session

Returns a node's desired and applied configuration epochs and last directive result.

Path

- `id` string — Must contain a positive decimal integer; otherwise 400 bad_request.

Success

`200` · {nodeId,config,targetEpoch,appliedEpoch,converged,lastResult,lastDirectiveAt}; absent nodeConfig row gives config:null, epochs 0, converged:true, lastResult:null and omits lastDirectiveAt.

Errors

- `400 bad_request` — id is not a positive decimal integer.
- `500 internal_error` — uncaught database or JSON-related exception occurs.

Tables

`nodeConfig`

Notes

Effective role session; CSRF none. Handler dashboard/api/routes/config.php:49-70.

```
GET /api/nodes/12/config
→ {"ok":true,"data":{"nodeId":"12","config":{"sampleIntervalSec":20},"targetEpoch":4,"appliedEpoch":3,"converged":false,"lastResult":"pending","lastDirectiveAt":"2026-11-03T11:59:00Z"},"ts":"2026-11-03T12:00:00Z"}
```

Source: `dashboard/api/routes/config.php:49`

POST /api/nodes/{id}/config

operator | CSRF: Origin + X-Solari-CSRF

Sets a node's desired configuration at the next epoch above both its current target and the global config floor, then queues a PROVISION directive.

Path

- `id` string — Must contain a positive decimal integer; otherwise 400 bad_request.

Body

JSON object

- `config` object/array **required** — Must decode to an array; no inner schema validation is performed.

Success

`200` · {nodeId,config,targetEpoch,status:"pushed"}

Errors

- `400 bad_request` — id is invalid, config is absent/not an array, body content type/JSON is invalid, bridge request exceeds size limit, or bridge rejects PROVISION.
- `404 not_found` — node row does not exist.
- `415 unsupported_media_type` — non-empty body is not application/json.
- `502 control_unavailable` — bridge cannot be reached, fails to reply, or returns no request ID.
- `403 forbidden` — bridge reports an operator/authorization refusal.
- `503 control_error` — bridge reports an active-server/lease failure.
- `500 internal_error` — uncaught exception occurs.

ctl verb

`PROVISION via SolariCtl::callQueued (which uses REQUEST_SUBMIT);`

Tables

`node`, `nodeConfig`, `globalConfig`

Notes

Effective role operator; CSRF origin+session-token. Handler dashboard/api/routes/config.php:74-96. Idempotency-Key is not supplied by this handler. Queued calls may return before completion, but the queued sentinel is ignored in this route.

```
POST /api/nodes/12/config
→ {"ok":true,"data":{"nodeId":"12","config":{"sampleIntervalSec":20},"targetEpoch":5,"status":"pushed"},"ts":"2026-11-03T12:00:00Z"}
```

Source: `dashboard/api/routes/config.php:74`

POST /api/rules/{ruleId} operator CSRF: Origin + X-Solari-CSRF

Validates and applies editable alert-rule fields through the bridge, then returns the stored rule fields.

Path
- `ruleId` string — Positive decimal integer; zero or non-digits return 400 bad_request.

Body
JSON object
- `enabled` boolean — Presence converts truthiness to bridge value 1 or 0.
- `threshold` number/string — Must be numeric; forwarded as float string.
- `forSeconds` integer/string — Must be integer or digit string; no range limit checked.
- `op` string — One of gt, lt, eq, transition.
- `severity` string — One of info, warn, crit.
- `metric` string — 1–64 bytes.
- `scope` string — One of host, probe.

Success
`200` · {ruleId,scope,metric,op,threshold,forSeconds,severity,enabled}

Errors
- `400 bad_request` — ruleId invalid, no editable fields supplied, a field fails validation, body content invalid, bridge request exceeds size limit, or bridge rejects RULE_SET.
- `404 not_found` — alertRule row does not exist.
- `415 unsupported_media_type` — non-empty body is not application/json.
- `502 control_unavailable` — bridge cannot be reached or does not provide a complete reply.
- `403 forbidden` — bridge reports an operator/authorization refusal.
- `503 control_error` — bridge reports an active-server/lease failure.
- `500 internal_error` — uncaught exception occurs.

ctl verb
`RULE_SET via SolariCtl::call;`

Tables
`alertRule`

Notes
Effective role operator; CSRF origin+session-token. Handler dashboard/api/routes/config.php:101-177. PHP reads alertRule before and after the bridge update; bridge performs the update.

```
POST /api/rules/5
→ {"ok":true,"data":{"ruleId":5,"scope":"host","metric":"cpu","op":"gt","threshold":90,"forSeconds":60,"severity":"warn","enabled":true},"ts":"2026-11-03T12:00:00Z"}
```

Source: `dashboard/api/routes/config.php:101`

POST /api/rules/{ruleId}/delete operator CSRF: Origin + X-Solari-CSRF

Deletes an alert rule after explicit confirmation and queues the bridge deletion.

Path
- `ruleId` string — Positive decimal integer; zero or non-digits return 400 bad_request.

Body
JSON object
- `confirm` boolean **required** — Must be literal JSON true; otherwise 409 confirm_required.

Success
`200` · {ruleId,status:"deleted"}

Errors
- `400 bad_request` — ruleId invalid, body content invalid, bridge request exceeds size limit, or bridge rejects queued submission.
- `404 not_found` — alertRule row does not exist.
- `409 confirm_required` — confirm is not literal true.
- `415 unsupported_media_type` — non-empty body is not application/json.
- `502 control_unavailable` — bridge cannot be reached, returns no request ID, or does not provide a complete reply.
- `403 forbidden` — bridge reports an operator/authorization refusal.
- `503 control_error` — bridge reports an active-server/lease failure.
- `500 internal_error` — uncaught exception occurs.

ctl verb
`RULE_DEL via SolariCtl::callQueued (which uses REQUEST_SUBMIT);`

Tables
`alertRule`

Notes
Effective role operator; CSRF origin+session-token. Handler dashboard/api/routes/config.php:179-196. Confirmation is permanent delete acknowledgement; queue sentinel is not inspected by this handler.

```
POST /api/rules/5/delete
→ {"ok":true,"data":{"ruleId":5,"status":"deleted"},"ts":"2026-11-03T12:00:00Z"}
```

Source: `dashboard/api/routes/config.php:179`

POOLS · 4

GET /api/pools session

Lists pools with asset counts and health-state counts derived from current probe outcomes.

Success
`200` · Array of {poolId,name,description,color,assetCount,health:{up,degraded,down,unknown}}.

Errors
- `500 internal_error` — uncaught database exception occurs.

Tables
`probeCurrent`, `probeTarget`, `asset`, `pool`

Notes
Effective role session; CSRF none. Handler dashboard/api/routes/pools.php:17-41. Pools are ordered by name; assets with null poolId contribute to pool id 0, and probe states without current results count as unknown.

```
GET /api/pools
→ {"ok":true,"data":[{"poolId":1,"name":"Unassigned","description":"","color":"#888888","assetCount":2,"health":
{"up":1,"degraded":0,"down":0,"unknown":1}}],"ts":"2026-11-03T12:00:00Z"}
```

Source: `dashboard/api/routes/pools.php:17`

POST /api/pools operator CSRF: Origin + X-Solari-CSRF

Creates a pool through the POOL_NEW control verb.

Body JSON object

- `name` string **required** — Trimmed; 1–64 bytes after trimming.
- `description` string — If nonempty, truncated to 240 bytes.
- `color` string — If nonempty, truncated to 15 bytes.

Success `201` · {poolId:number|null, name:string}

Errors

- `400 bad_request` — name is empty or longer than 64 bytes after trim
- `400 bad_request` — nonempty body is not JSON or JSON does not decode to an array
- `415 unsupported_media_type` — nonempty body lacks application/json content type
- `400 control_error` — ctl rejects POOL_NEW
- `403 forbidden` — ctl refuses POOL_NEW
- `502 control_unavailable` — ctl bridge unavailable

ctl verb `POOL_NEW via SolariCtl::call`

Notes Writes are delegated to ctl; description and color are passed as desc and color.

```
POST /api/pools {"name":"Compute","description":"Servers"}
→ {"ok":true,"data":{"poolId":7,"name":"Compute"},"ts":"2026-01-01T00:00:00Z"}
```

Source: `dashboard/api/routes/pools.php:43`

POST /api/pools/{poolId} operator CSRF: Origin + X-Solari-CSRF

Renames or edits the description and color of an existing pool.

Path

- `poolId` positive integer — Must contain decimal digits and cannot be 0.

Body JSON object

- `name` string — If supplied, trimmed and must be 1–64 bytes.
- `description` string — If supplied, truncated to 240 bytes.
- `color` string — If supplied, truncated to 15 bytes.

Success `200` · {poolId:number}

Errors

- `400 bad_request` — invalid poolId, invalid name, or no editable fields
- `404 not_found` — pool does not exist
- `400 bad_request` — nonempty body is not JSON or JSON does not decode to an array
- `415 unsupported_media_type` — nonempty body lacks application/json content type
- `400 control_error` — ctl rejects POOL_SET
- `403 forbidden` — ctl refuses POOL_SET
- `502 control_unavailable` — ctl bridge unavailable

ctl verb `POOL_SET via SolariCtl::call`

Notes The pool existence check reads the pool table before ctl mutation.

```
POST /api/pools/7 {"name":"Compute"}
→ {"ok":true,"data":{"poolId":7},"ts":"2026-01-01T00:00:00Z"}
```

Source: `dashboard/api/routes/pools.php:56`

POST /api/pools/{poolId}/delete operator CSRF: Origin + X-Solari-CSRF

Deletes a pool and reassigns its assets to Unassigned after explicit confirmation.

Path

- `poolId` positive integer — Decimal digits, nonzero; pool 1 (Unassigned) cannot be deleted.

Body JSON object

- `confirm` boolean **required** — Must be literal JSON true; otherwise confirm_required (409).

Success `200` · {poolId:number, status:"deleted", reassigned:number|null}

Errors

- `400 bad_request` — poolId invalid or equals 1
- `404 not_found` — pool does not exist
- `409 confirm_required` — confirm is not literal true
- `400 bad_request` — nonempty body is not JSON or JSON does not decode to an array
- `415 unsupported_media_type` — nonempty body lacks application/json content type
- `400 bad_request` — supplied Idempotency-Key fails validation
- `400 control_error` — queued POOL_DEL fails
- `403 forbidden` — ctl refuses queued request
- `502 control_unavailable` — ctl bridge unavailable or request id missing

ctl verb `POOL_DEL via SolariCtl::callQueued (submits REQUEST_SUBMIT)`

Notes Queued call waits up to 15 seconds; an unsettled call returns {request,status:"queued"} rather than the normal deletion data.

```
POST /api/pools/7/delete {"confirm":true}
→ {"ok":true,"data":{"poolId":7,"status":"deleted","reassigned":3},"ts":"2026-01-01T00:00:00Z"}
```

Source: `dashboard/api/routes/pools.php:77`

GET /api/assets session

Lists assets with pool, probe target count, health state, tags, and available lifecycle fields.

Query
- `poolId` integer default `all pools` — Applied only if the query value consists of digits; other values are ignored.

Success `200` · [{assetId,ip,host,displayName,class,poolId,poolName,tags,notes,monitorHost,targetCount,state,lifecycle?,criticality?}]

Errors Common errors only (§2.6).

Notes lifecycle and criticality keys are included only when their columns exist; assets with null poolId are not included by a numeric poolId filter.

```
GET /api/assets?poolId=2
→ {"ok":true,"data":[{"assetId":12,"ip":"192.0.2.4","host":"node.example","displayName":"node.example","class":"server","poolId":2,"poolName":"Compute","tags":
[],"notes":"","monitorHost":true,"targetCount":1,"state":"up","lifecycle":"active","criticality":2}],"ts":"2026-01-01T00:00:00Z"}
```

Source: `dashboard/api/routes/assets.php:17`

GET /api/assets/{assetId} session

Returns one asset with its targets and each target's current vantage count and rolled-up state.

Path
- `assetId` positive integer — Decimal digits and nonzero.

Success `200` · {assetId,ip,host,displayName,class,poolId,poolName,tags,notes,monitorHost,targetCount,state,lifecycle?,criticality?,targets:[{targetId,host,port,proto,label,state,vantages}]}

Errors
- `400 bad_request` — assetId is not a positive decimal integer
- `404 not_found` — asset does not exist

Notes Lifecycle fields are conditional on schema columns; state derives from current probeCurrent rows.

```
GET /api/assets/12
→ {"ok":true,"data":{"assetId":12,"ip":"192.0.2.4","host":"node.example","displayName":"node.example","class":"server","poolId":2,"poolName":"Compute","tags":
[],"notes":"","monitorHost":true,"targetCount":1,"state":"up","targets":[{"targetId":"t-
1","host":"node.example","port":443,"proto":"tcp","label":"HTTPS","state":"up","vantages":1}]},"ts":"2026-01-01T00:00:00Z"}
```

Source: `dashboard/api/routes/assets.php:41`

POST /api/assets/{assetId} operator CSRF: Origin + X-Solari-CSRF

Merges supplied asset metadata into the current record and updates the SoR host and ctl asset mirror.

Path
- `assetId` positive integer — Decimal digits and nonzero.

Body JSON object
- `displayName` string — String-cast and truncated to 120 bytes for ctl; the untruncated value is passed to Sor::upsertHost.
- `class` string — One of server, appliance, network, iot, host, other.
- `poolId` integer|null — Null maps to pool 0; nonzero values must identify an existing pool.
- `notes` string — String-cast and truncated to 240 bytes.
- `monitorHost` boolean — Boolean-cast by PHP.
- `tags` array — Used only if an array; values are passed as JSON without item validation.

Success `200` · {assetId:number, status:"updated"}

Errors
- `400 bad_request` — invalid assetId, invalid class, or nonzero poolId does not exist
- `404 not_found` — asset does not exist
- `503 service_unavailable` — SoR upsert fails
- `400 bad_request` — nonempty body is not JSON or JSON does not decode to an array
- `415 unsupported_media_type` — nonempty body lacks application/json content type
- `400 control_error` — ctl rejects ASSET_SET
- `403 forbidden` — ctl refuses ASSET_SET
- `502 control_unavailable` — ctl bridge unavailable

ctl verb `ASSET_SET via SolariCtl::call`

Tables `sources`, `ip_addresses`, `entities`

Notes SoR upsert is a no-op success if SoR is unconfigured or IP empty; otherwise it is keyed by IP and may create or rename a host. ctl receives the full merged desired state.

```
POST /api/assets/12 {"displayName":"node.example","class":"server","poolId":2,"tags":["prod"]}
→ {"ok":true,"data":{"assetId":12,"status":"updated"},"ts":"2026-01-01T00:00:00Z"}
```

Source: `dashboard/api/routes/assets.php:77`

POST /api/assets/{assetId}/remove operator CSRF: Origin + X-Solari-CSRF

Removes the asset from SoR and queues deletion of the asset and its probe targets/history.

Path
- `assetId` positive integer — Decimal digits and nonzero.

Body
JSON object
- `confirm` boolean **required** — Must be literal JSON true; otherwise confirm_required (409).

Success
`200` · {assetId:number,status:"removed",removed:number|null}

Errors
- `400 bad_request` — invalid assetId or supplied Idempotency-Key
- `404 not_found` — asset does not exist
- `409 confirm_required` — confirm is not literal true
- `503 service_unavailable` — SoR removal fails
- `400 bad_request` — nonempty body is not JSON or JSON does not decode to an array
- `415 unsupported_media_type` — nonempty body lacks application/json content type
- `400 control_error` — queued ASSET_REMOVE fails
- `403 forbidden` — ctl refuses queued request
- `502 control_unavailable` — ctl bridge unavailable or request id missing

ctl verb
`ASSET_REMOVE via SolariCtl::callQueued (submits REQUEST_SUBMIT)`

Tables
`sources`, `ip_addresses`, `entities`

Notes
SoR removal is a soft delete and no-op success when unconfigured or IP empty; queued call waits 15 seconds and may return {request,status:"queued"}.

```
POST /api/assets/12/remove {"confirm":true}
→ {"ok":true,"data":{"assetId":12,"status":"removed","removed":1},"ts":"2026-01-01T00:00:00Z"}
```

Source: `dashboard/api/routes/assets.php:124`

POST /api/assets/{assetId}/lifecycle operator CSRF: Origin + X-Solari-CSRF

Queues a lifecycle transition for an existing asset.

Path
- `assetId` positive integer — Decimal digits and nonzero.

Body
JSON object
- `confirm` boolean **required** — Must be literal JSON true.
- `action` string **required** — decommission maps to decommissioned; delete to deleted; restore to active.

Success
`200` · {assetId:number,status:"decommissioned"|"deleted"|"active"}

Errors
- `400 bad_request` — invalid assetId, action, or supplied Idempotency-Key
- `404 not_found` — asset does not exist
- `409 confirm_required` — confirm is not literal true
- `400 bad_request` — nonempty body is not JSON or JSON does not decode to an array
- `415 unsupported_media_type` — nonempty body lacks application/json content type
- `400 control_error` — queued LIFECYCLE_SET fails
- `403 forbidden` — ctl refuses queued request
- `502 control_unavailable` — ctl bridge unavailable or request id missing

ctl verb
`LIFECYCLE_SET via SolariCtl::callQueued (submits REQUEST_SUBMIT)`

Notes
Queued call waits up to 15 seconds and may return {request,status:"queued"} if unsettled.

```
POST /api/assets/12/lifecycle {"confirm":true,"action":"decommission"}
→ {"ok":true,"data":{"assetId":12,"status":"decommissioned"},"ts":"2026-01-01T00:00:00Z"}
```

Source: `dashboard/api/routes/assets.php:151`

POST /api/assets/{assetId}/purge operator CSRF: Origin + X-Solari-CSRF

Queues an administrator-confirmed permanent purge of an existing asset.

Path
- `assetId` positive integer — Decimal digits and nonzero.

Body
JSON object
- `confirmName` string **required** — Must exactly match displayName, falling back to host then IP.

Success
`200` · {assetId:number,status:"purged"}

Errors
- `400 bad_request` — invalid assetId or supplied Idempotency-Key
- `404 not_found` — asset does not exist
- `409 confirm_required` — confirmName is absent, empty, non-string, or mismatched
- `400 bad_request` — nonempty body is not JSON or JSON does not decode to an array
- `415 unsupported_media_type` — nonempty body lacks application/json content type
- `400 control_error` — queued ASSET_PURGE fails
- `403 forbidden` — ctl refuses queued request
- `502 control_unavailable` — ctl bridge unavailable or request id missing

ctl verb
`ASSET_PURGE via SolariCtl::callQueued (submits REQUEST_SUBMIT)`

Notes
Requires admin role. Queued call waits up to 15 seconds; the confirmation chain is displayName, host, then IP.

```
POST /api/assets/12/purge {"confirmName":"node.example"}
→ {"ok":true,"data":{"assetId":12,"status":"purged"},"ts":"2026-01-01T00:00:00Z"}
```

Source: `dashboard/api/routes/assets.php:171`

POST `/api/assets/{assetId}/criticality` operator CSRF: Origin + X-Solari-CSRF

Sets an existing asset's criticality tier through ctl.

Path
- `assetId` positive integer — Decimal digits and nonzero.

Body
JSON object
- `tier` integer **required** — Must be a JSON integer from 0 through 4.

Success
`200` · {assetId:number,criticality:number}

Errors
- `400 bad_request` — invalid assetId or tier
- `404 not_found` — asset does not exist
- `400 bad_request` — nonempty body is not JSON or JSON does not decode to an array
- `415 unsupported_media_type` — nonempty body lacks application/json content type
- `400 control_error` — ctl rejects CRIT_SET
- `403 forbidden` — ctl refuses CRIT_SET
- `502 control_unavailable` — ctl bridge unavailable

ctl verb
`CRIT_SET via SolariCtl::call`

Notes
Passes the authenticated operator name to ctl as op.

```
POST /api/assets/12/criticality {"tier":3}
→ {"ok":true,"data":{"assetId":12,"criticality":3},"ts":"2026-01-01T00:00:00Z"}
```

Source: `dashboard/api/routes/assets.php:194`

POST `/api/assets/{assetId}/targets/{targetId}/remove` operator CSRF: Origin + X-Solari-CSRF

Queues removal of a target after checking it belongs to the specified asset.

Path
- `assetId` positive integer — Decimal digits and nonzero.
- `targetId` string — Must be nonempty and no more than 128 bytes.

Body
JSON object
- `confirm` boolean **required** — Must be literal JSON true; otherwise confirm_required (409).

Success
`200` · {assetId:number,targetId:string,status:"removed"}

Errors
- `400 bad_request` — invalid assetId, targetId, or supplied Idempotency-Key
- `404 not_found` — target does not belong to asset
- `409 confirm_required` — confirm is not literal true
- `400 bad_request` — nonempty body is not JSON or JSON does not decode to an array
- `415 unsupported_media_type` — nonempty body lacks application/json content type
- `400 control_error` — queued TARGET_REMOVE fails
- `403 forbidden` — ctl refuses queued request
- `502 control_unavailable` — ctl bridge unavailable or request id missing

ctl verb
`TARGET_REMOVE via SolariCtl::callQueued (submits REQUEST_SUBMIT)`

Notes
Queued call waits up to 15 seconds and may return {request,status:"queued"}; target lookup is scoped to assetId.

```
POST /api/assets/12/targets/t-1/remove {"confirm":true}
→ {"ok":true,"data":{"assetId":12,"targetId":"t-1","status":"removed"},"ts":"2026-01-01T00:00:00Z"}
```

Source: `dashboard/api/routes/assets.php:210`

3.6 DNS & network allocation · 8 routes

These routes are read-only views of the DNS release state (zones, records, hosts, RPZ, health, and a query probe). The one write approves or revokes a network's static-address allocator eligibility in the SoR. DNS content itself changes only through the SoR and CDC (§5). There is no API for editing records directly.

GET `/api/networks/allocator-eligibility` session

Lists live SoR networks with their live subnets, DHCP dynamic pools, IPv4 subnet counts, and overlaps against other networks.

Success
`200` · [{id,name,purpose,enabled,management,is_default,allocator_eligible,allocator_approved_by,allocator_approved_at,vlan_tag,vlan_name,subnets:[{id,network_id,cidr,gateway_ip,dhcp_pools,ipv4}],ipv4_subnet_count,overlapping_networks,ipv4_overlap_count}]

Errors
- `503 service_unavailable` — A network or subnet read throws.
- `503 service_unavailable` — SoR is unconfigured or unreachable.

Tables
`networks`, `vlans`, `subnets`, `dhcp_ranges`

Notes
Only networks and subnets with deleted_at IS NULL are selected; overlapping_networks contains distinct other-network name/CIDR strings.

```
GET /api/networks/allocator-eligibility
→ {"ok":true,"data":[{"id":7,"name":"Servers","enabled":true,"allocator_eligible":true,"subnets":
[{"id":12,"network_id":7,"cidr":"10.0.0.0/24","gateway_ip":"10.0.0.1","dhcp_pools":"10.0.0.100–10.0.0.180","ipv4":true}],"ipv4_subnet_count":1,"overlapping_networks":
[],"ipv4_overlap_count":0}],"ts":"2026-11-03T12:00:00Z"}
```

Source: `dashboard/api/routes/network_allocator.php:12`

POST /api/networks/{id}/allocator-eligibility operator CSRF: Origin + X-Solari-CSRF

Approves or revokes a live network's static-address allocator eligibility in the SoR.

Path
- `id` positive integer — Must be a digit string representing an integer greater than zero.

Body

JSON object
- `eligible` boolean **required** — Must be a JSON boolean.
- `confirm` boolean **required** — Must be true; otherwise confirmation_required (400).

Success

`200` · {id,allocator_eligible}

Errors
- `400 bad_request` — Invalid id or eligible is missing/not boolean.
- `400 confirmation_required` — confirm is not exactly true.
- `404 not_found` — The network is absent or deleted.
- `409 conflict` — Approval is requested for a disabled network, a network without exactly one live IPv4 subnet, or an overlapping live IPv4 prefix.
- `503 service_unavailable` — SoR is unconfigured/unreachable or the transactional write fails.
- `413 payload_too_large` — A configured JSON body limit is exceeded.
- `415 unsupported_media_type` — A non-empty body is not application/json.

Tables

`networks`, `subnets`

Locking

Pessimistic row lock: SELECT … FOR UPDATE on networks.id; no optimistic-lock field or 409 version code.

Notes

Approval writes allocator_approved_by and allocator_approved_at; revocation clears both. Body read with solari_json_body() and no explicit byte cap (only the PHP/Apache request limits apply).

```
POST /api/networks/7/allocator-eligibility with {"eligible":true,"confirm":true}
→ {"ok":true,"data":{"id":7,"allocator_eligible":true},"ts":"2026-11-03T12:00:00Z"}
```

Source: `dashboard/api/routes/network_allocator.php:84`

GET /api/dns/query session

Queries the requested DNS name and record type concurrently against selected configured resolvers.

Query
- `name` hostname/FQDN **required** — Must match the route's 1–253 character hostname pattern.
- `type` string default `A` — Case-insensitive; accepted types are A, AAAA, CNAME, PTR, SRV, TXT, MX, NS, SOA, CAA, ANY.
- `servers` comma-separated server ids default `steel,xenon,radium,external` — Blank also selects all; each nonblank id must be configured.

Success

`200` · {name,type,ts,results:[{server:{id,label,ip,kind},status,flags,answers,authority,queryTimeMs,rpzBlocked,error}]}

Errors
- `400 bad_request` — Invalid name, type, or server id.

Notes

Resolver IPs come from the server-side registry; dig failures are represented in each result as status/error rather than route errors.

```
GET /api/dns/query?name=host.akoria.net&type=A&servers=xenon
→ {"ok":true,"data":{"name":"host.akoria.net","type":"A","results":[{"server":
{"id":"xenon","label":"xenon","ip":"10.0.0.20","kind":"internal"},"status":"NOERROR","flags":{"aa":false,"ra":true},"answers":[],"authority":
[],"queryTimeMs":3,"rpzBlocked":false,"error":null}]},"ts":"2026-11-03T12:00:00Z"}
```

Source: `dashboard/api/routes/dns.php:48`

GET /api/dns/health session

Checks configured DNS zones' SOA serial parity and resolver reachability, then includes RPZ parity and local zone-file statistics.

Success

`200` · {sorAvailable,servers:[{id,label,ip,reachable}],zones:[{name,kind,authority,expectedServers,soa:{serverId:{serial,status,queryTimeMs}},parity}],rpz:{zone,serials,parity,blockedCount,lastRefresh,sourceUrl}}

Errors

Common errors only (§2.6).

Tables

`dns_zones`

Notes

Server reachability is false only when no dig result for that server avoided TIMEOUT; zone parity requires every expected server to return NOERROR with a parseable matching serial. RPZ local file defaults to /etc/bind/zones/db.rpz.akoria or SOLARI_RPZ_ZONE_FILE and degrades file statistics to null.

```
GET /api/dns/health
→ {"ok":true,"data":{"sorAvailable":true,"servers":[{"id":"xenon","label":"xenon","ip":"10.0.0.20","reachable":true}],"zones":
[{"name":"akoria.net","kind":"forward","authority":"sor_rendered","expectedServers":["xenon","steel"],"soa":{"xenon":
{"serial":2026110301,"status":"NOERROR","queryTimeMs":2}},"parity":true}],"rpz":{"zone":"rpz.akoria","serials":{},"parity":true,"blockedCount":12,"lastRefresh":"2026-
11-03T11:00:00Z","sourceUrl":"https://raw.githubusercontent.com/StevenBlack/hosts/master/hosts"}},"ts":"2026-11-03T12:00:00Z"}
```

Source: `dashboard/api/routes/dns.php:91`

GET /api/dns/zones session

Lists live SoR DNS zones with primary nameserver and generated versus explicit record counts.

Success

`200` · [{id,name,kind,authority,primaryNs,defaultTtl,hostmaster,recordCount,generatedCount,explicitCount,updatedAt,notes}]

Errors
- `503 service_unavailable` — SoR is unconfigured or unreachable.

Tables

`dns_zones`, `entities`, `dns_records`

Notes

Ordered by zone name; record counts exclude deleted records.

```
GET /api/dns/zones
→ {"ok":true,"data":
[{"id":"2","name":"akoria.net","kind":"forward","authority":"sor_rendered","primaryNs":"xenon","defaultTtl":3600,"hostmaster":"hostmaster.akoria.net","recordCount":9,"
generatedCount":7,"explicitCount":2,"updatedAt":"2026-11-03T10:00:00Z","notes":null}],"ts":"2026-11-03T12:00:00Z"}
```

Source: `dashboard/api/routes/dns.php:175`

GET `/api/dns/zones/{id}/records` session

Returns ordered live records from one live SoR DNS zone, with optional type, text, and generated filters.

Path	• `id` numeric string — Must contain only decimal digits.
Query	• `type` string — Optional; validated against the DNS route type whitelist and uppercased. • `q` string — Optional trimmed substring matched against name or rdata using LIKE. • `generated` 0|1 — Filters is_generated; any other nonempty value yields bad_request (400). • `limit` integer default `500` — Decimal digits required; clamped to 1–2000, with omitted/blank defaulting to 500.
Success	`200` · {zone:{id,name},records:[{id,name,rrtype,rdata,ttl,priority,isGenerated,ipAddressId,targetEntityId,assertedKind,updatedAt}],total,truncated}
Errors	• `400 bad_request` — Invalid zone id, record type, generated filter, or limit. • `404 not_found` — No live zone has the requested id. • `503 service_unavailable` — SoR is unconfigured or unreachable.
Tables	`dns_zones`, `dns_records`
Notes	Page size is controlled by limit only; the route does not return an offset or cursor.

```
GET /api/dns/zones/2/records?type=A&limit=100
→ {"ok":true,"data":{"zone":{"id":"2","name":"akoria.net"},"records":
[{"id":"83","name":"host.akoria.net","rrtype":"A","rdata":"10.0.0.41","ttl":300,"priority":null,"isGenerated":true,"ipAddressId":"25","targetEntityId":null,"assertedKi
nd":"system","updatedAt":"2026-11-03T10:00:00Z"}],"total":1,"truncated":false},"ts":"2026-11-03T12:00:00Z"}
```

Source: `dashboard/api/routes/dns.php:218`

GET `/api/dns/hosts` session

Returns DNS host, interface, and CNAME preview rows from SoR views.

Success	`200` · {hosts:[{host,ip,role,lifecycle}],ifaces:[{label,ip}],cnames:[{alias,targetHost}]}
Errors	• `503 service_unavailable` — SoR is unconfigured or unreachable.
Tables	`v_dns_hosts`, `v_dns_ifaces`, `v_dns_cnames`
Notes	Read-only preview of SoR DNS views.

```
GET /api/dns/hosts
→ {"ok":true,"data":{"hosts":[{"host":"host.akoria.net","ip":"10.0.0.41","role":"server","lifecycle":"active"}],"ifaces":[],"cnames":[]},"ts":"2026-11-03T12:00:00Z"}
```

Source: `dashboard/api/routes/dns.php:299`

GET `/api/dns/rpz` session

Returns RPZ SOA parity for xenon and steel plus local RPZ zone-file statistics.

Success	`200` · {zone,serials:{xenon:{serial,status},steel:{serial,status}},parity,blockedCount,lastRefresh,sourceUrl}
Errors	Common errors only (§2.6).
Notes	Local file statistics use SOLARI_RPZ_ZONE_FILE or /etc/bind/zones/db.rpz.akoria; missing/unreadable file returns null blockedCount and lastRefresh.

```
GET /api/dns/rpz
→ {"ok":true,"data":{"zone":"rpz.akoria","serials":{"xenon":{"serial":2026110301,"status":"NOERROR"},"steel":
{"serial":2026110301,"status":"NOERROR"}},"parity":true,"blockedCount":12,"lastRefresh":"2026-11-
03T11:00:00Z","sourceUrl":"https://raw.githubusercontent.com/StevenBlack/hosts/master/hosts"},"ts":"2026-11-03T12:00:00Z"}
```

Source: `dashboard/api/routes/dns.php:325`

3.7 Identity, PKI & Forgejo · 3 routes

These three read-only status views cover the internal CA (chlorine), Forgejo, and the identity stack (Samba AD and Keycloak). None of them mutates anything. Certificate issuance and account management happen in those systems, not through this API.

GET `/api/forgejo` session

Returns the Forgejo summary, degrading unavailable service state into the payload.

Success	`200` · Forgejo::summary() payload, including availability and summary fields.
Errors	Common errors only (§2.6).
Notes	Implementation is fail-soft; exact keys depend on Forgejo helper result.

```
GET /api/forgejo
→ {"ok":true,"data":{"available":false,"reason":"unavailable"},"ts":"2026-11-03T00:00:00Z"}
```

Source: `dashboard/api/routes/forgejo.php:18`

GET `/api/ca` session

Returns CA health, certificate inventory, and node certificate summaries.

Success	`200` · Ca::summary() payload including stepCa, roots, provisioners, nodeCerts, and warnDays.
Errors	Common errors only (§2.6).
Tables	`node`
Notes	CA service and node inventory failures degrade their corresponding payload sections.

```
GET /api/ca
→ {"ok":true,"data":{"stepCa":{},"roots":[],"provisioners":[],"nodeCerts":[],"warnDays":30},"ts":"2026-11-03T00:00:00Z"}
```

Source: `dashboard/api/routes/ca.php:18`

GET /api/identity session

Returns Keycloak users with enrolled factor and group information, or an unavailable result.

Success	`200` · {available,reason,realmUrl?,users:[{id,username,email,enabled,emailVerified,createdAt,factors,groups}]}
Errors	Common errors only (§2.6).
Notes	At most 200 users are requested from Keycloak; helper failures return unavailable data with HTTP 200.

```
GET /api/identity
→ {"ok":true,"data":{"available":false,"reason":"Keycloak is not configured","users":[]},"ts":"2026-11-03T00:00:00Z"}
```

Source: `dashboard/api/routes/identity.php:19`

3.8 Provisioning · 35 routes

Provisioning covers the image, software and stack catalogue, its relationship sub-resources, the run history, and the three queued enqueue verbs. The catalogue is described in §8.1. Enqueue bodies, run tracking and log retrieval are in §8.2.

BUILDS · 1

GET /api/builds session

Lists build artifacts and counts non-retired nodes by architecture and configured version.

Success	`200` · {builds:[{buildId,arch,os,version,channel,sha256,sizeBytes,artifactUri,publishedAt,nodesOnThisVersion}],nodeVersionDist:{arch:{version:count}}}; null node versions use the empty-string key.
Errors	Common errors only (§2.6).
Tables	`buildArtifact`, `node`, `nodeConfig`
Notes	Builds order by arch, OS, then publishedAt descending; node distribution counts rows with `state <> 'retired'`.

```
GET /api/builds
→ {"ok":true,"data":{"builds":
[{"buildId":3,"arch":"x86_64","os":"linux","version":"1.2","channel":"stable","sha256":"abc","sizeBytes":1024,"artifactUri":"https://example.invalid/a","publishedAt":"
2026-01-01T00:00:00Z","nodesOnThisVersion":2}],"nodeVersionDist":{"x86_64":{"1.2":2}}},"ts":"2026-01-01T00:00:00Z"}
```

Source: `dashboard/api/routes/provisioning.php:85`

OS IMAGES · 5

GET /api/provisioning/images session

Lists live provisioning images ordered by distro, release, architecture, boot method, and ID.

Success	`200` · Array of image objects {id,distro,family,release_name,release_label,arch,boot_method,source_url,checksum_url,version,sha256,fetched_at,fetched,install_template,refresh_policy,netboot_verified,notes,updated_
Errors	• `400 bad_request` — A catalog database constraint or field-range error occurs. • `409 conflict` — A catalog rule or referenced-row constraint rejects the query. • `503 busy` — Catalog lock timeout or deadlock occurs. • `500 db_error` — Other provisioning database error occurs.
Tables	`prov_images`
Notes	Only rows with deleted_at IS NULL are returned.

```
GET /api/provisioning/images
→ {"ok":true,"data":
[{"id":1,"distro":"debian","family":"debian","release_name":"stable","release_label":"Stable","arch":"x86_64","boot_method":"ipxe","source_url":null,"checksum_url":nul
l,"version":null,"sha256":null,"fetched_at":null,"fetched":false,"install_template":null,"refresh_policy":"manual","netboot_verified":false,"notes":null,"updated_at":"
2026-01-01T00:00:00Z"}],"ts":"2026-01-01T00:00:00Z"}
```

Source: `dashboard/api/routes/provisioning.php:233`

GET /api/provisioning/images/{id} session

Returns one live provisioning image by numeric ID.

Path	• `id` positive integer — 1..4294967295; otherwise bad_request.
Success	`200` · Image object with the same keys as GET /api/provisioning/images.
Errors	• `400 bad_request` — ID is not a positive integer up to 4294967295. • `404 not_found` — No live image has that ID. • `400 bad_request` — A catalog database constraint or field-range error occurs. • `409 conflict` — A catalog rule or referenced-row constraint rejects the query. • `503 busy` — Catalog lock timeout or deadlock occurs. • `500 db_error` — Other provisioning database error occurs.
Tables	`prov_images`
Notes	Soft-deleted rows are treated as not found.

```
GET /api/provisioning/images/1
→ {"ok":true,"data":
{"id":1,"distro":"debian","family":"debian","release_name":"stable","release_label":null,"arch":"x86_64","boot_method":"ipxe","source_url":null,"checksum_url":null,"ve
rsion":null,"sha256":null,"fetched_at":null,"fetched":false,"install_template":null,"refresh_policy":"manual","netboot_verified":false,"notes":null,"updated_at":"2026-
01-01T00:00:00Z"},"ts":"2026-01-01T00:00:00Z"}
```

Source: `dashboard/api/routes/provisioning.php:239`

POST /api/provisioning/images

operator | CSRF: Origin + X-Solari-CSRF

Creates a provisioning image and returns the created row.

Body JSON object

- `distro` string **required** — Lowercase slug, up to 32 characters.
- `family` string **required** — One of suse, debian, fedora.
- `release_name` string **required** — Safe release token up to 48 characters.
- `release_label` string|null — UTF-8, at most 96 characters.
- `arch` string **required** — One of x86_64, arm64, armhf, riscv64, i386.
- `boot_method` string **required** — One of ipxe, pi_netboot, media; pi_netboot requires arm64 or armhf.
- `source_url` string|null — HTTP(S) URL, no whitespace/control characters, at most 512 bytes.
- `checksum_url` string|null — Same URL validation as source_url.
- `install_template` string|null — Safe template token, up to 128 characters.
- `refresh_policy` string — If supplied, manual, weekly, or point_release.
- `netboot_verified` boolean|0|1 — Boolean or integer 0/1.
- `notes` string|null — UTF-8, at most 255 characters.

Success `201` · Image object with the standard image keys.

Errors

- `400 bad_request` — Invalid JSON shape, unknown field, missing required field, managed field supplied, or field validation fails.
- `409 conflict` — Duplicate key, FK, or catalog rule rejects insert.
- `503 busy` — Catalog lock timeout or deadlock occurs.
- `500 db_error` — Other DB error or created row cannot be read.

Tables `prov_images`

Notes 201 response; version, sha256, fetched_at are refresh-managed.

```
POST /api/provisioning/images {"distro":"debian","family":"debian","release_name":"stable","arch":"x86_64","boot_method":"ipxe"}
→ {"ok":true,"data":
{"id":1,"distro":"debian","family":"debian","release_name":"stable","release_label":null,"arch":"x86_64","boot_method":"ipxe","source_url":null,"checksum_url":null,"ve
rsion":null,"sha256":null,"fetched_at":null,"fetched":false,"install_template":null,"refresh_policy":null,"netboot_verified":false,"notes":null,"updated_at":"2026-01-
01T00:00:00Z"},"ts":"2026-01-01T00:00:00Z"}
```

Source: `dashboard/api/routes/provisioning.php:247`

PUT /api/provisioning/images/{id}

operator | CSRF: Origin + X-Solari-CSRF

Updates supplied fields on a live provisioning image.

Path

- `id` positive integer — 1..4294967295.

Body JSON object

- `image fields` object **required** — Partial update; allowed fields are distro,family,release_name,release_label,arch,boot_method,source_url,checksum_url,install_template,refresh_policy,netboot_verified,notes; body must be nonempty.

Success `200` · Image object with the standard image keys.

Errors

- `400 bad_request` — Invalid ID/body/field, empty body, or attempt to set version, sha256, fetched_at.
- `404 not_found` — No live image has that ID.
- `409 conflict` — Duplicate key or catalog rule rejects update.
- `503 busy` — Catalog lock timeout or deadlock occurs.
- `500 db_error` — Other DB error.

Tables `prov_images`

Locking Transaction locks the row with FOR UPDATE; no updated_at/expected_updated_at comparison. Changed source fields clear version, sha256, fetched_at.

Notes A partial update; source field changes invalidate the fetched artifact.

```
PUT /api/provisioning/images/1 {"notes":"Reviewed"}
→ {"ok":true,"data":
{"id":1,"distro":"debian","family":"debian","release_name":"stable","release_label":null,"arch":"x86_64","boot_method":"ipxe","source_url":null,"checksum_url":null,"ve
rsion":null,"sha256":null,"fetched_at":null,"fetched":false,"install_template":null,"refresh_policy":null,"netboot_verified":false,"notes":"Reviewed","updated_at":"202
6-01-01T00:00:00Z"},"ts":"2026-01-01T00:00:00Z"}
```

Source: `dashboard/api/routes/provisioning.php:255`

DELETE /api/provisioning/images/{id}

operator | CSRF: Origin + X-Solari-CSRF

Soft-deletes a live provisioning image.

Path

- `id` positive integer — 1..4294967295.

Success `200` · {id,deleted:true}.

Errors

- `400 bad_request` — ID is invalid.
- `404 not_found` — No live image has that ID.
- `409 conflict` — FK or catalog rule rejects deletion.
- `503 busy` — Catalog lock timeout or deadlock occurs.
- `500 db_error` — Other DB error.

Tables `prov_images`

Notes Deletion stamps human provenance; no explicit transaction.

```
DELETE /api/provisioning/images/1
→ {"ok":true,"data":{"id":1,"deleted":true},"ts":"2026-01-01T00:00:00Z"}
```

Source: `dashboard/api/routes/provisioning.php:279`

GET /api/provisioning/software session

Lists live software catalog entries, optionally filtered by item type, with live conflict IDs.

Query
- `type` string default `all item types` — If nonempty, one of pkg, repo, artifact, script, build, role.

Success
`200` · Array of software objects {id,logical_name,item_type,name_suse,name_debian,name_fedora,applies_families,applies_arches,ansible_role,source_url,version_pin,spec,notes,updated_at,conflicts_with}.

Errors
- `400 bad_request` — type is not an allowed item type.
- `400 bad_request` — A catalog database constraint or field-range error occurs.
- `409 conflict` — A catalog rule or referenced-row constraint rejects the query.
- `503 busy` — Catalog lock timeout or deadlock occurs.
- `500 db_error` — Other provisioning database error occurs.

Tables
`prov_software`, `prov_software_conflicts`

Notes
Conflict IDs include live conflict pairs whose two software rows are live.

```
GET /api/provisioning/software?type=pkg
→ {"ok":true,"data":
[{"id":2,"logical_name":"curl","item_type":"pkg","name_suse":"curl","name_debian":"curl","name_fedora":"curl","applies_families":null,"applies_arches":null,"ansible_ro
le":null,"source_url":null,"version_pin":null,"spec":null,"notes":null,"updated_at":"2026-01-01T00:00:00Z","conflicts_with":[]}],"ts":"2026-01-01T00:00:00Z"}
```

Source: `dashboard/api/routes/provisioning.php:293`

GET /api/provisioning/software/{id} session

Returns one live software entry with artifact, requirement, conflict, and stack relationships.

Path
- `id` positive integer — 1..4294967295.

Success
`200` · Software object plus artifacts:[{id,arch,url,sha256}], requires/required_by/conflicts:[{software_id,logical_name,item_type,notes}], stacks:[{id,slug}].

Errors
- `400 bad_request` — ID is invalid.
- `404 not_found` — No live software entry has that ID.
- `400 bad_request` — A catalog database constraint or field-range error occurs.
- `409 conflict` — A catalog rule or referenced-row constraint rejects the query.
- `503 busy` — Catalog lock timeout or deadlock occurs.
- `500 db_error` — Other provisioning database error occurs.

Tables
`prov_software`, `prov_software_artifacts`, `prov_software_requires`, `prov_software_conflicts`, `prov_stack_members`, `prov_stacks`

Notes
Related rows and referenced catalog records are filtered to live rows.

```
GET /api/provisioning/software/2
→ {"ok":true,"data":
{"id":2,"logical_name":"curl","item_type":"pkg","name_suse":"curl","name_debian":"curl","name_fedora":"curl","applies_families":null,"applies_arches":null,"ansible_rol
e":null,"source_url":null,"version_pin":null,"spec":null,"notes":null,"updated_at":"2026-01-01T00:00:00Z","artifacts":[],"requires":[],"required_by":[],"conflicts":
[],"stacks":[]},"ts":"2026-01-01T00:00:00Z"}
```

Source: `dashboard/api/routes/provisioning.php:320`

POST /api/provisioning/software operator CSRF: Origin + X-Solari-CSRF

Creates a software catalog entry and returns its detail representation.

Body
JSON object
- `logical_name` string **required** — Nonblank UTF-8, at most 96 characters, no control characters.
- `item_type` string — pkg, repo, artifact, script, build, or role; defaults to pkg.
- `name_suse/name_debian/name_fedora` string|null — Each at most 128 characters; family-name constraints depend on item_type.
- `applies_families` list|null — Null means all; otherwise nonempty list from suse,debian,fedora.
- `applies_arches` list|null — Null means all; otherwise nonempty list from x86_64,arm64,armhf,riscv64,i386.
- `ansible_role` string|null — Safe role token up to 64; required only for item_type=role.
- `source_url` string|null — HTTP(S), no whitespace/control, at most 512 bytes.
- `version_pin` string|null — Safe token up to 64; required for artifact and build.
- `spec` object|null — Type-specific validation; encoded JSON max 128 KiB.
- `notes` string|null — UTF-8, at most 255 characters.

Success
`201` · Software detail object including artifacts, requires, required_by, conflicts, and stacks arrays.

Errors
- `400 bad_request` — Invalid JSON shape, unknown field, required field absent, or field/type-specific validation fails.
- `409 conflict` — Duplicate key, FK, or catalog rule rejects insert.
- `503 busy` — Catalog lock timeout or deadlock occurs.
- `500 db_error` — Other DB error or created row cannot be read.

Tables
`prov_software`, `prov_software_artifacts`, `prov_software_requires`, `prov_software_conflicts`, `prov_stack_members`, `prov_stacks`

Notes
Only the listed software fields are accepted; an empty object fails because logical_name is required.

```
POST /api/provisioning/software {"logical_name":"curl","name_debian":"curl"}
→ {"ok":true,"data":
{"id":2,"logical_name":"curl","item_type":"pkg","name_suse":null,"name_debian":"curl","name_fedora":null,"applies_families":null,"applies_arches":null,"ansible_role":n
ull,"source_url":null,"version_pin":null,"spec":null,"notes":null,"updated_at":"2026-01-01T00:00:00Z","artifacts":[],"requires":[],"required_by":[],"conflicts":
[],"stacks":[]},"ts":"2026-01-01T00:00:00Z"}
```

Source: `dashboard/api/routes/provisioning.php:328`

PUT /api/provisioning/software/{id} operator CSRF: Origin + X-Solari-CSRF

Updates supplied fields on a live software entry and returns its detail.

Path
- `id` positive integer — 1..4294967295.

Body
JSON object
- `software fields` object **required** — Partial nonempty object; allowed keys are logical_name,item_type,name_suse,name_debian,name_fedora,applies_families,applies_arches,ansible_role,source_url,version_pin,spec,notes.

Success
`200` · Software detail object including artifacts, requires, required_by, conflicts, stacks.

Errors
- `400 bad_request` — Invalid ID/body/field, empty body, or type-specific validation fails.
- `404 not_found` — No live software entry has that ID.
- `409 conflict` — Duplicate/FK/catalog rule failure or changing away from artifact while live artifact rows exist.
- `503 busy` — Catalog lock timeout or deadlock occurs.
- `500 db_error` — Other DB error.

Tables
`prov_software`, `prov_software_artifacts`, `prov_software_requires`, `prov_software_conflicts`, `prov_stack_members`, `prov_stacks`

Locking
Transaction locks the software row with FOR UPDATE; no updated_at/expected_updated_at comparison. Counts live artifacts before validating update.

Notes
Partial updates validate merged type-dependent fields against the stored row.

```
PUT /api/provisioning/software/2 {"notes":"Updated"}
→ {"ok":true,"data":
{"id":2,"logical_name":"curl","item_type":"pkg","name_suse":null,"name_debian":"curl","name_fedora":null,"applies_families":null,"applies_arches":null,"ansible_role":n
ull,"source_url":null,"version_pin":null,"spec":null,"notes":"Updated","updated_at":"2026-01-01T00:00:00Z","artifacts":[],"requires":[],"required_by":[],"conflicts":
[],"stacks":[]},"ts":"2026-01-01T00:00:00Z"}
```

Source: `dashboard/api/routes/provisioning.php:336`

DELETE /api/provisioning/software/{id} operator CSRF: Origin + X-Solari-CSRF

Soft-deletes a software entry and its artifacts, outgoing requirements, and conflict pairs.

Path
- `id` positive integer — 1..4294967295.

Success
`200` · {id,deleted:true}.

Errors
- `400 bad_request` — ID is invalid.
- `404 not_found` — No live software entry has that ID.
- `409 conflict` — Entry is still in a live stack, is required by another live entry, or DB catalog rule refuses deletion.
- `503 busy` — Catalog lock timeout or deadlock occurs.
- `500 db_error` — Other DB error.

Tables
`prov_software`, `prov_stack_members`, `prov_stacks`, `prov_software_requires`, `prov_software_artifacts`, `prov_software_conflicts`

Locking
Transaction locks target software row with FOR UPDATE; no optimistic updated_at comparison.

Notes
The transaction refuses deletion while live stack membership or incoming requirements exist; outgoing requirements and related rows are soft-deleted.

```
DELETE /api/provisioning/software/2
→ {"ok":true,"data":{"id":2,"deleted":true},"ts":"2026-01-01T00:00:00Z"}
```

Source: `dashboard/api/routes/provisioning.php:356`

POST /api/provisioning/software/{id}/artifacts operator CSRF: Origin + X-Solari-CSRF

Adds an architecture-specific URL and SHA-256 artifact to an existing software entry of type artifact.

Path
- `id` positive integer ID, 1..4294967295 — Invalid values return bad_request.

Body
JSON object
- `arch` string **required** — One of any, x86_64, arm64, armhf, riscv64, i386.
- `url` string **required** — HTTP(S), no whitespace or control characters, maximum 512 bytes.
- `sha256` string **required** — Exactly 64 lowercase hexadecimal characters.

Success
`201` · Software detail object with base software fields plus artifacts, requires, required_by, conflicts, and stacks.

Errors
- `400 bad_request` — Invalid path/body field, missing required key, unknown key, non-object body, or self-invalid data.
- `404 not_found` — Software entry is missing or deleted.
- `409 conflict` — Database uniqueness, foreign-key, or catalog rule rejects the insert.
- `413 payload_too_large` — Body exceeds 16384 bytes.
- `415 unsupported_media_type` — Nonempty request body is not application/json.
- `503 busy` — Database lock or deadlock error.
- `500 db_error` — Other database error.

Tables
`prov_software`, `prov_software_artifacts`, `prov_software_requires`, `prov_software_conflicts`, `prov_stack_members`, `prov_stacks`

Notes
The parent must have item_type artifact. Insert runs in a transaction and records human provenance.

```
POST /api/provisioning/software/12/artifacts
{"arch":"x86_64","url":"https://example.invalid/app.tar.gz","sha256":"aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa"}
→ {"ok":true,"data":{"id":12,"logical_name":"demo","item_type":"artifact","artifacts":
[{"id":31,"arch":"x86_64","url":"https://example.invalid/app.tar.gz","sha256":"aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa"}],"requires":
[],"required_by":[],"conflicts":[],"stacks":[]},"ts":"2026-01-01T00:00:00Z"}
```

Source: `dashboard/api/routes/provisioning.php:395`

DELETE /api/provisioning/software/{id}/artifacts/{artifactId}

operator | CSRF: Origin + X-Solari-CSRF

Soft-deletes an artifact belonging to the specified software entry.

Path

- `id` positive integer ID, 1..4294967295 — Invalid values return bad_request.
- `artifactId` positive integer ID, 1..4294967295 — Invalid values return bad_request.

Success

`200` · {id, deleted:true} where id is artifactId.

Errors

- `400 bad_request` — Either path ID is invalid.
- `404 not_found` — No live artifact row matches both IDs.
- `409 conflict` — Database catalog rule rejects the update.
- `503 busy` — Database lock or deadlock error.
- `500 db_error` — Other database error.

Tables

`prov_software_artifacts`

Notes

Sets deleted_at; update is scoped to the parent software ID.

```
DELETE /api/provisioning/software/12/artifacts/31
→ {"ok":true,"data":{"id":31,"deleted":true},"ts":"2026-01-01T00:00:00Z"}
```

Source: `dashboard/api/routes/provisioning.php:417`

POST /api/provisioning/software/{id}/requires

operator | CSRF: Origin + X-Solari-CSRF

Adds a requirement edge from the specified software entry to another software entry.

Path

- `id` positive integer ID, 1..4294967295 — Invalid values return bad_request.

Body

JSON object

- `requires_software_id` integer or digit string ID **required** — Positive ID up to 4294967295.
- `notes` string or null — Maximum 255 UTF-8 characters.

Success

`201` · Software detail object with base software fields plus artifacts, requires, required_by, conflicts, and stacks.

Errors

- `400 bad_request` — Invalid/missing ID, unknown key, invalid notes, or entry requires itself.
- `404 not_found` — Either software entry is missing or deleted.
- `409 conflict` — Database uniqueness, foreign-key, or catalog rule rejects the insert.
- `413 payload_too_large` — Body exceeds 16384 bytes.
- `415 unsupported_media_type` — Nonempty request body is not application/json.
- `503 busy` — Database lock or deadlock error.
- `500 db_error` — Other database error.

Tables

`prov_software`, `prov_software_requires`, `prov_software_artifacts`, `prov_software_conflicts`, `prov_stack_members`, `prov_stacks`

Notes

Self-requirement is rejected. Insert runs in a transaction and records human provenance.

```
POST /api/provisioning/software/12/requires {"requires_software_id":18,"notes":"needed at runtime"}
→ {"ok":true,"data":{"id":12,"logical_name":"demo","item_type":"pkg","artifacts":[],"requires":
[{"software_id":18,"logical_name":"lib","item_type":"pkg","notes":"needed at runtime"}],"required_by":[],"conflicts":[],"stacks":[]},"ts":"2026-01-01T00:00:00Z"}
```

Source: `dashboard/api/routes/provisioning.php:431`

DELETE /api/provisioning/software/{id}/requires/{requiresId}

operator | CSRF: Origin + X-Solari-CSRF

Soft-deletes the requirement edge from the specified software entry to the required entry.

Path

- `id` positive integer ID, 1..4294967295 — Invalid values return bad_request.
- `requiresId` positive integer ID, 1..4294967295 — Invalid values return bad_request.

Success

`200` · {software_id, requires_software_id, deleted:true}.

Errors

- `400 bad_request` — Either path ID is invalid.
- `404 not_found` — No live matching requirement edge exists.
- `409 conflict` — Database catalog rule rejects the update.
- `503 busy` — Database lock or deadlock error.
- `500 db_error` — Other database error.

Tables

`prov_software_requires`

Notes

Sets deleted_at; the edge is directional.

```
DELETE /api/provisioning/software/12/requires/18
→ {"ok":true,"data":{"software_id":12,"requires_software_id":18,"deleted":true},"ts":"2026-01-01T00:00:00Z"}
```

Source: `dashboard/api/routes/provisioning.php:450`

POST /api/provisioning/software/{id}/conflicts

operator · CSRF: Origin + X-Solari-CSRF

Adds a conflict edge between the specified software entry and another software entry.

Path
- `id` positive integer ID, 1..4294967295 — Invalid values return bad_request.

Body
JSON object
- `software_id` integer or digit string ID **required** — Positive ID up to 4294967295.
- `notes` string or null — Maximum 255 UTF-8 characters.

Success
`201` · Software detail object with base software fields plus artifacts, requires, required_by, conflicts, and stacks.

Errors
- `400 bad_request` — Invalid/missing ID, unknown key, invalid notes, or self-conflict.
- `404 not_found` — Either software entry is missing or deleted.
- `409 conflict` — Database uniqueness, foreign-key, or catalog rule rejects the insert.
- `413 payload_too_large` — Body exceeds 16384 bytes.
- `415 unsupported_media_type` — Nonempty request body is not application/json.
- `503 busy` — Database lock or deadlock error.
- `500 db_error` — Other database error.

Tables
`prov_software`, `prov_software_conflicts`, `prov_software_artifacts`, `prov_software_requires`, `prov_stack_members`, `prov_stacks`

Notes
Self-conflicts are rejected; stored IDs are canonicalized low-to-high. Insert runs in a transaction and records human provenance.

```
POST /api/provisioning/software/12/conflicts {"software_id":18,"notes":"mutually exclusive"}
→ {"ok":true,"data":{"id":12,"logical_name":"demo","item_type":"pkg","artifacts":[],"requires":[],"required_by":[],"conflicts":
[{"software_id":18,"logical_name":"other","item_type":"pkg","notes":"mutually exclusive"}],"stacks":[]},"ts":"2026-01-01T00:00:00Z"}
```

Source: `dashboard/api/routes/provisioning.php:464`

DELETE /api/provisioning/software/{id}/conflicts/{otherId}

operator · CSRF: Origin + X-Solari-CSRF

Soft-deletes the conflict edge between the specified software entries.

Path
- `id` positive integer ID, 1..4294967295 — Invalid values return bad_request.
- `otherId` positive integer ID, 1..4294967295 — Invalid values return bad_request.

Success
`200` · {software_id, other_id, deleted:true}.

Errors
- `400 bad_request` — Either path ID is invalid.
- `404 not_found` — No live matching conflict edge exists.
- `409 conflict` — Database catalog rule rejects the update.
- `503 busy` — Database lock or deadlock error.
- `500 db_error` — Other database error.

Tables
`prov_software_conflicts`

Notes
Conflict IDs are canonicalized for matching; sets deleted_at.

```
DELETE /api/provisioning/software/12/conflicts/18
→ {"ok":true,"data":{"software_id":12,"other_id":18,"deleted":true},"ts":"2026-01-01T00:00:00Z"}
```

Source: `dashboard/api/routes/provisioning.php:484`

STACKS · 10

GET /api/provisioning/stacks

session

Lists live stacks with live member counts and live dependency slugs.

Success
`200` · Array of {id, slug, name, description, locked, sort_order, updated_at, member_count, depends_on:[slug]}.

Errors
- `503 busy` — Database lock or deadlock error.
- `500 db_error` — Other database error.

Tables
`prov_stacks`, `prov_stack_members`, `prov_software`, `prov_stack_deps`

Notes
No pagination or query parameters are read; only live stacks, members, software, and dependencies contribute.

```
GET /api/provisioning/stacks
→ {"ok":true,"data":[{"id":4,"slug":"base","name":"Base","description":null,"locked":false,"sort_order":0,"updated_at":"2026-01-
01T00:00:00Z","member_count":2,"depends_on":[]}],"ts":"2026-01-01T00:00:00Z"}
```

Source: `dashboard/api/routes/provisioning.php:499`

GET /api/provisioning/stacks/{id}

session

Returns one live stack with its members, dependencies, and dependents.

Path
- `id` positive integer ID, 1..4294967295 — Invalid values return bad_request.

Success
`200` · {id, slug, name, description, locked, sort_order, updated_at, members:[{software_id, logical_name, item_type, item_order, effective_order, sort_order, notes}], depends_on:[{stack_id, slug}], depended_on_by:[{stack_id, slug}]}

Errors
- `400 bad_request` — Path ID is invalid.
- `404 not_found` — Stack is missing or deleted.
- `503 busy` — Database lock or deadlock error.
- `500 db_error` — Other database error.

Tables
`prov_stacks`, `prov_stack_members`, `prov_software`, `prov_stack_deps`

Notes
Member rows include only live memberships and live software entries.

```
GET /api/provisioning/stacks/4
→ {"ok":true,"data":{"id":4,"slug":"base","name":"Base","description":null,"locked":false,"sort_order":0,"updated_at":"2026-01-01T00:00:00Z","members":[],"depends_on":
[],"depended_on_by":[]},"ts":"2026-01-01T00:00:00Z"}
```

Source: `dashboard/api/routes/provisioning.php:520`

POST /api/provisioning/stacks

operator | CSRF: Origin + X-Solari-CSRF

Creates a stack with the supplied fields.

Body — JSON object

- `slug` string **required** — Pattern [a-z0-9][a-z0-9-]{0,47}.
- `name` string **required** — Nonblank valid UTF-8, maximum 96 characters.
- `description` string or null — Maximum 255 UTF-8 characters.
- `locked` boolean or 0/1 integer — Stored as 0/1.
- `sort_order` integer — Range 0..65535.

Success — `201` · Stack detail object with members, depends_on, and depended_on_by arrays.

Errors

- `400 bad_request` — Missing required field, unknown key, invalid value, empty/non-object body.
- `409 conflict` — Duplicate unique key, foreign-key, or catalog rule rejects insert.
- `413 payload_too_large` — Body exceeds 16384 bytes.
- `415 unsupported_media_type` — Nonempty request body is not application/json.
- `503 busy` — Database lock or deadlock error.
- `500 db_error` — Created row cannot be read or another database error occurs.

Tables — `prov_stacks`, `prov_stack_members`, `prov_stack_deps`

Notes — Omitted optional fields are handled by stackFields defaults; insert stamps human provenance.

```
POST /api/provisioning/stacks {"slug":"base","name":"Base"}
→ {"ok":true,"data":{"id":4,"slug":"base","name":"Base","description":null,"locked":false,"sort_order":0,"updated_at":"2026-01-01T00:00:00Z","members":[],"depends_on":
[],"depended_on_by":[]},"ts":"2026-01-01T00:00:00Z"}
```

Source: `dashboard/api/routes/provisioning.php:528`

PUT /api/provisioning/stacks/{id}

operator | CSRF: Origin + X-Solari-CSRF

Updates supplied fields on an existing live stack.

Path

- `id` positive integer ID, 1..4294967295 — Invalid values return bad_request.

Body — JSON object

- `slug` string — Pattern [a-z0-9][a-z0-9-]{0,47}.
- `name` string — Nonblank valid UTF-8, maximum 96 characters.
- `description` string or null — Maximum 255 UTF-8 characters.
- `locked` boolean or 0/1 integer — Stored as 0/1.
- `sort_order` integer — Range 0..65535.

Success — `200` · Stack detail object with members, depends_on, and depended_on_by arrays.

Errors

- `400 bad_request` — Invalid path/body field, unknown key, or empty update body.
- `404 not_found` — Stack is missing or deleted.
- `409 conflict` — Duplicate unique key, foreign-key, or catalog rule rejects update.
- `413 payload_too_large` — Body exceeds 16384 bytes.
- `415 unsupported_media_type` — Nonempty request body is not application/json.
- `503 busy` — Database lock or deadlock error.
- `500 db_error` — Other database error.

Tables — `prov_stacks`, `prov_stack_members`, `prov_stack_deps`

Notes — The handler locks the stack row with FOR UPDATE inside a transaction; there is no expected_updated_at optimistic lock.

```
PUT /api/provisioning/stacks/4 {"name":"Core Base"}
→ {"ok":true,"data":{"id":4,"slug":"base","name":"Core Base","description":null,"locked":false,"sort_order":0,"updated_at":"2026-01-01T00:00:00Z","members":
[],"depends_on":[],"depended_on_by":[]},"ts":"2026-01-01T00:00:00Z"}
```

Source: `dashboard/api/routes/provisioning.php:535`

DELETE /api/provisioning/stacks/{id}

operator | CSRF: Origin + X-Solari-CSRF

Soft-deletes a stack and its live memberships and outgoing dependencies when deletion is permitted.

Path

- `id` positive integer ID, 1..4294967295 — Invalid values return bad_request.

Success — `200` · {id, deleted:true}.

Errors

- `400 bad_request` — Path ID is invalid.
- `404 not_found` — Stack is missing or deleted.
- `409 conflict` — Stack is locked, is depended on by a live stack, or a database rule rejects deletion.
- `503 busy` — Database lock or deadlock error.
- `500 db_error` — Other database error.

Tables — `prov_stacks`, `prov_stack_deps`, `prov_stack_members`

Notes — Runs in a transaction; refuses locked stacks and stacks referenced by live dependents; soft-deletes memberships and dependencies where the stack is the dependent.

```
DELETE /api/provisioning/stacks/4
→ {"ok":true,"data":{"id":4,"deleted":true},"ts":"2026-01-01T00:00:00Z"}
```

Source: `dashboard/api/routes/provisioning.php:551`

POST /api/provisioning/stacks/{id}/members

operator | CSRF: Origin + X-Solari-CSRF

Adds a software entry to a stack with optional ordering and notes.

Path
- `id` positive integer ID, 1..4294967295 — Invalid values return bad_request.

Body

JSON object
- `software_id` integer or digit string ID **required** — Positive ID up to 4294967295.
- `item_order` integer or null — Range 0..65535, or null.
- `sort_order` integer — Range 0..65535.
- `notes` string or null — Maximum 255 UTF-8 characters.

Success

`201` · Stack detail object with members, depends_on, and depended_on_by arrays.

Errors
- `400 bad_request` — Invalid/missing ID or member field, unknown key, or invalid body.
- `404 not_found` — Stack or software entry is missing or deleted.
- `409 conflict` — Duplicate member, foreign-key, or catalog rule rejects insert.
- `413 payload_too_large` — Body exceeds 16384 bytes.
- `415 unsupported_media_type` — Nonempty request body is not application/json.
- `503 busy` — Database lock or deadlock error.
- `500 db_error` — Other database error.

Tables

`prov_stacks`, `prov_software`, `prov_stack_members`, `prov_stack_deps`

Notes

Runs in a transaction, checks both parent rows with FOR UPDATE, and records human provenance.

```
POST /api/provisioning/stacks/4/members {"software_id":12,"item_order":10,"sort_order":0}
→ {"ok":true,"data":{"id":4,"slug":"base","name":"Base","description":null,"locked":false,"sort_order":0,"updated_at":"2026-01-01T00:00:00Z","members":
[{"software_id":12,"logical_name":"demo","item_type":"pkg","item_order":10,"effective_order":10,"sort_order":0,"notes":null}],"depends_on":[],"depended_on_by":
[]},"ts":"2026-01-01T00:00:00Z"}
```

Source: `dashboard/api/routes/provisioning.php:582`

PUT /api/provisioning/stacks/{id}/members/{softwareId}

operator | CSRF: Origin + X-Solari-CSRF

Updates an existing live stack member's ordering or notes and returns the refreshed stack detail.

Path
- `id` positive integer, 1..4294967295 — Stack ID.
- `softwareId` positive integer, 1..4294967295 — Software entry ID.

Body

JSON object
- `item_order` integer|null — 0..65535 or null.
- `sort_order` integer — 0..65535.
- `notes` string|null — UTF-8, at most 255 characters.

Success

`200` · {id,slug,name,description,locked,sort_order,updated_at,members,depends_on,depended_on_by}; members/detail fields are assembled by stackDetail.

Errors
- `400 bad_request` — Invalid path ID, body, unknown field, empty update, or attempted software_id change.
- `404 not_found` — No live member matches the stack and software IDs, or the stack is not readable.
- `413 payload_too_large` — JSON body exceeds 16384 bytes.
- `415 unsupported_media_type` — A nonempty body is not application/json.
- `409 conflict` — Database uniqueness, FK, or catalog rule rejects the update.
- `503 busy` — Database deadlock or lock timeout.
- `500 db_error` — Other catalog PDO error.

Tables

`prov_stack_members`, `prov_stacks`, `prov_software`, `prov_stack_deps`

Notes

An unchanged update can report zero affected rows; the route checks for a still-live member before deciding it is not found. No transaction is used for this update.

```
PUT /api/provisioning/stacks/12/members/34 {"sort_order":5,"notes":"core"}
→ {"ok":true,"data":{"id":12,"slug":"base","members":[{"software_id":34}],"depends_on":[],"depended_on_by":[]},"ts":"2026-11-03T00:00:00Z"}
```

Source: `dashboard/api/routes/provisioning.php:600`

DELETE /api/provisioning/stacks/{id}/members/{softwareId}

operator | CSRF: Origin + X-Solari-CSRF

Soft-deletes a live member from a stack.

Path
- `id` positive integer, 1..4294967295 — Stack ID.
- `softwareId` positive integer, 1..4294967295 — Software entry ID.

Success

`200` · {stack_id,software_id,deleted}

Errors
- `400 bad_request` — Either path ID is invalid.
- `404 not_found` — No live member matches the supplied IDs.
- `409 conflict` — A database constraint or catalog rule rejects the update.
- `503 busy` — Database deadlock or lock timeout.
- `500 db_error` — Other catalog PDO error.

Tables

`prov_stack_members`

Notes

Sets deleted_at; no provenance fields are updated.

```
DELETE /api/provisioning/stacks/12/members/34
→ {"ok":true,"data":{"stack_id":12,"software_id":34,"deleted":true},"ts":"2026-11-03T00:00:00Z"}
```

Source: `dashboard/api/routes/provisioning.php:622`

POST /api/provisioning/stacks/{id}/deps

operator | CSRF: Origin + X-Solari-CSRF

Adds a stack dependency and returns the refreshed stack detail.

Path
- `id` positive integer, 1..4294967295 — Stack ID.

Body
JSON object
- `depends_on_stack_id` positive integer, 1..4294967295 **required** — Must differ from the path stack ID.

Success
`201` · {id,slug,name,description,locked,sort_order,updated_at,members,depends_on,depended_on_by}; assembled by stackDetail.

Errors
- `400 bad_request` — Invalid IDs, missing/unknown field, invalid body, or self-dependency.
- `404 not_found` — Either stack is missing or soft-deleted.
- `413 payload_too_large` — JSON body exceeds 4096 bytes.
- `415 unsupported_media_type` — A nonempty body is not application/json.
- `409 conflict` — Duplicate dependency, FK, or catalog rule rejects insert.
- `503 busy` — Database deadlock or lock timeout.
- `500 db_error` — Other catalog PDO error.

Tables
`prov_stack_deps`, `prov_stacks`, `prov_stack_members`, `prov_software`

Notes
Insertion and live-stack checks are transactional; insert stamps human provenance.

```
POST /api/provisioning/stacks/12/deps {"depends_on_stack_id":9}
→ {"ok":true,"data":{"id":12,"slug":"base","members":[],"depends_on":[{"id":9}],"depended_on_by":[]},"ts":"2026-11-03T00:00:00Z"}
```

Source: `dashboard/api/routes/provisioning.php:636`

DELETE /api/provisioning/stacks/{id}/deps/{dependsOnId}

operator | CSRF: Origin + X-Solari-CSRF

Soft-deletes a live dependency edge from a stack.

Path
- `id` positive integer, 1..4294967295 — Owning stack ID.
- `dependsOnId` positive integer, 1..4294967295 — Dependency stack ID.

Success
`200` · {stack_id,depends_on_stack_id,deleted}

Errors
- `400 bad_request` — Either path ID is invalid.
- `404 not_found` — No live dependency edge matches the IDs.
- `409 conflict` — A database constraint or catalog rule rejects the update.
- `503 busy` — Database deadlock or lock timeout.
- `500 db_error` — Other catalog PDO error.

Tables
`prov_stack_deps`

Notes
Sets deleted_at; no provenance fields are updated.

```
DELETE /api/provisioning/stacks/12/deps/9
→ {"ok":true,"data":{"stack_id":12,"depends_on_stack_id":9,"deleted":true},"ts":"2026-11-03T00:00:00Z"}
```

Source: `dashboard/api/routes/provisioning.php:653`

TARGETS, RUNS & ACTIONS · 8

GET /api/provisioning/targets

session

Lists eligible live SoR entities with live MAC-bearing interfaces and resolves target architecture.

Success
`200` · [{id,name,entity_type,lifecycle,role,hw_model_id,hw_make,hw_model,hw_type,arch,arch_source,agent_arch,interfaces:[{name,mac,management}]}]

Errors
- `500 db_error` — Catalog query PDO error.

Tables
`entities`, `interfaces`, `hardware_units`, `hardware_models`, `node`

Notes
Hardware model architecture takes precedence over agent architecture; agent lookup matches normalized short hostname and failures are caught as empty agent data. Entities without a live MAC-bearing interface are excluded.

```
GET /api/provisioning/targets
→ {"ok":true,"data":
[{"id":"42","name":"helium","entity_type":"server","lifecycle":"active","role":"compute","hw_model_id":3,"arch":"x86_64","arch_source":"hardware_model","agent_arch":"x
86_64","interfaces":[{"name":"eno1","mac":"aa:bb:cc:dd:ee:ff","management":true}]}],"ts":"2026-11-03T00:00:00Z"}
```

Source: `dashboard/api/routes/provisioning.php:700`

GET /api/provisioning/log

session

Returns the bounded tail of a provisioning helper log.

Query
- `kind` string **required** — One of image-refresh, prov-stage, stack-apply.
- `id` positive integer ID **required** — Must satisfy the positive integer ID contract through 4294967295.

Success
`200` · {kind,id,exists,log,truncated,size,updated_at}

Errors
- `400 bad_request` — kind is invalid/missing or id is invalid/missing.

Notes
Returns at most 65536 tail bytes, may drop the first partial line when truncated, and strips ANSI color escapes. id in the result is an integer.

```
GET /api/provisioning/log?kind=prov-stage&id=7
→ {"ok":true,"data":{"kind":"prov-stage","id":7,"exists":false,"log":"","truncated":false,"size":0,"updated_at":null},"ts":"2026-11-03T00:00:00Z"}
```

Source: `dashboard/api/routes/provisioning.php:751`

GET /api/provisioning/store session

Returns the latest cached provisioning store status or an unknown/stale result.

Success	`200` · {known,store,fstype,source,reachable,mounted,linked,ok,size_bytes,avail_bytes,reason,checked_at,stale}; unknown result is {known:false,stale:true,checked_at:null}.
Errors	Common errors only (§2.6).
Notes	Reads a maximum of 16384 bytes from prov-store.json beside the ctl socket. String values are clipped to 512 characters; numeric byte fields accept nonnegative JSON integers only.

```
GET /api/provisioning/store
→ {"ok":true,"data":{"known":false,"stale":true,"checked_at":null},"ts":"2026-11-03T00:00:00Z"}
```

Source: `dashboard/api/routes/provisioning.php:803`

GET /api/provisioning/runs session

Lists provisioning runs newest first, optionally filtered by entity, image, mode, or status.

Query	• `entity` positive integer ID default `none` — When nonempty, filters r.entity_id. • `image` positive integer ID default `none` — When nonempty, filters r.image_id. • `mode` string default `none` — One of reimage, apply, refresh. • `status` string default `none` — One of queued, running, succeeded, failed, cancelled. • `limit` integer default `50` — Decimal 1..200; malformed or out-of-range values produce bad_request.
Success	`200` · [{id,mode,entity_id,entity_name,image_id,image_label,stack_slugs,requested_by,status,dry_run,detail,requested_at,started_at,finished_at}]
Errors	• `400 bad_request` — Invalid entity/image ID, mode/status enum, or limit. • `500 db_error` — Run query PDO error.
Tables	`prov_runs`, `entities`, `prov_images`
Notes	Rows order by requested_at DESC, then id DESC. stack_slugs decodes from JSON and falls back to an empty list when invalid.

```
GET /api/provisioning/runs?status=queued&limit=10
→ {"ok":true,"data":[{"id":"7","mode":"apply","entity_id":"42","entity_name":"helium","image_id":null,"image_label":null,"stack_slugs":
["base"],"requested_by":"operator","status":"queued","dry_run":true,"detail":null,"requested_at":"2026-11-
03T00:00:00Z","started_at":null,"finished_at":null}],"ts":"2026-11-03T00:00:00Z"}
```

Source: `dashboard/api/routes/provisioning.php:851`

GET /api/provisioning/runs/{id} session

Returns one provisioning run by numeric run ID.

Path	• `id` positive integer run ID — Route accepts 1..18 decimal digits without leading zero.
Success	`200` · {id,mode,entity_id,entity_name,image_id,image_label,stack_slugs,requested_by,status,dry_run,detail,requested_at,started_at,finished_at}
Errors	• `400 bad_request` — ID is not a positive decimal number of at most 18 digits. • `404 not_found` — No matching run exists. • `500 db_error` — Run query PDO error.
Tables	`prov_runs`, `entities`, `prov_images`
Notes	Run id is returned as a string; entity_id is a string or null.

```
GET /api/provisioning/runs/7
→ {"ok":true,"data":{"id":"7","mode":"apply","entity_id":"42","entity_name":"helium","image_id":null,"image_label":null,"stack_slugs":
["base"],"requested_by":"operator","status":"queued","dry_run":true,"detail":null,"requested_at":"2026-11-
03T00:00:00Z","started_at":null,"finished_at":null},"ts":"2026-11-03T00:00:00Z"}
```

Source: `dashboard/api/routes/provisioning.php:883`

POST /api/provisioning/image-refresh operator CSRF: Origin + X-Solari-CSRF

Queues an image refresh request for an existing image.

Body	JSON object • `image` positive integer ID **required** — Integer or digit string through 4294967295.
Success	`200` · {image,status,log,request?}; status is queued or started.
Errors	• `400 bad_request` — Invalid body, image ID, or Idempotency-Key. • `404 not_found` — Image does not exist. • `413 payload_too_large` — JSON body exceeds 4096 bytes. • `415 unsupported_media_type` — A nonempty body is not application/json. • `502 control_error` — Queued ctl request fails or bridge returns no request ID. • `403 forbidden` — Queued ctl rejects operator authorization. • `503 control_error` — Ctl reports active-server/lease availability failure. • `409 conflict` — Catalog constraint rejects a related DB operation. • `503 busy` — Catalog deadlock or lock timeout.
ctl verb	`IMAGE_REFRESH via SolariCtl::callQueued (REQUEST_SUBMIT, then REQUEST_GET polling)`
Tables	`prov_images`
Notes	Optional Idempotency-Key must match [A-Za-z0-9._-]{8,128}; same key resubmission returns the existing request ID. Queue poll budget is four seconds; queued requests include request for later polling.

```
POST /api/provisioning/image-refresh {"image":5}
→ {"ok":true,"data":{"image":5,"status":"queued","log":null,"request":"71"},"ts":"2026-11-03T00:00:00Z"}
```

Source: `dashboard/api/routes/provisioning.php:919`

POST `/api/provisioning/stage` operator CSRF: Origin + X-Solari-CSRF

Queues a netboot staging request after validating the target interface, image and stack slugs.

Body: JSON object
- `entity` positive integer ID **required** — Integer or digit string through 4294967295.
- `image` positive integer ID **required** — Integer or digit string through 4294967295.
- `stacks` array of strings **required** — 1..32 distinct slugs matching [a-z0-9][a-z0-9-]{0,47}; duplicates are deduplicated.
- `mac` string **required** — Normalized to lowercase colon-separated MAC; must match aa:bb:cc:dd:ee:ff form.

Success: `200` · {entity,image,stacks,mac,arch,arch_source,status,log,request?}; status is queued or started.

Errors:
- `400 bad_request` — Invalid JSON/body field, IDs, stack list, MAC, or Idempotency-Key.
- `404 not_found` — Entity, image, or one or more live stack slugs are absent.
- `409 conflict` — No live MAC interface, requested MAC is not live on entity, image is not ipxe/fetched, target architecture is unknown, or image architecture differs.
- `413 payload_too_large` — JSON body exceeds 16384 bytes.
- `415 unsupported_media_type` — A nonempty body is not application/json.
- `502 control_error` — Queued ctl request fails or bridge returns no request ID.
- `403 forbidden` — Queued ctl rejects operator authorization.
- `503 control_error` — Ctl reports active-server/lease availability failure.

ctl verb: `PROV_STAGE via SolariCtl::callQueued (REQUEST_SUBMIT, then REQUEST_GET polling)`

Tables: `entities`, `hardware_units`, `hardware_models`, `interfaces`, `prov_images`, `prov_stacks`

Notes: Client-supplied architecture is not accepted. Hardware-model arch takes precedence over normalized agent node.arch; unknown arch and mismatch are rejected. Queue requests may return request for later polling.

```
POST /api/provisioning/stage {"entity":42,"image":5,"stacks":["base"],"mac":"AA-BB-CC-DD-EE-FF"}
→ {"ok":true,"data":{"entity":42,"image":5,"stacks":
["base"],"mac":"aa:bb:cc:dd:ee:ff","arch":"x86_64","arch_source":"hardware_model","status":"started","log":null},"ts":"2026-11-03T00:00:00Z"}
```

Source: `dashboard/api/routes/provisioning.php:936`

POST `/api/provisioning/apply` operator CSRF: Origin + X-Solari-CSRF

Queues a stack application request for a live entity and selected stacks.

Body: JSON object
- `entity` positive integer ID **required** — Integer or digit string through 4294967295.
- `stacks` array of strings **required** — 1..32 distinct slugs matching [a-z0-9][a-z0-9-]{0,47}; duplicates are deduplicated.
- `dry_run` boolean **required** — Must be a JSON boolean.

Success: `200` · {entity,stacks,dry_run,status,log,request?}; status is queued or started.

Errors:
- `400 bad_request` — Invalid JSON/body, IDs, stack list, dry_run, or Idempotency-Key.
- `404 not_found` — Entity or one or more live stack slugs are absent.
- `413 payload_too_large` — JSON body exceeds 16384 bytes.
- `415 unsupported_media_type` — A nonempty body is not application/json.
- `502 control_error` — Queued ctl request fails or bridge returns no request ID.
- `403 forbidden` — Queued ctl rejects operator authorization.
- `503 control_error` — Ctl reports active-server/lease availability failure.

ctl verb: `STACK_APPLY via SolariCtl::callQueued (REQUEST_SUBMIT, then REQUEST_GET polling)`

Tables: `entities`, `prov_stacks`

Notes: Optional Idempotency-Key must match [A-Za-z0-9._-]{8,128}; same key resubmission returns the existing request ID. Queue poll budget is four seconds.

```
POST /api/provisioning/apply {"entity":42,"stacks":["base"],"dry_run":true}
→ {"ok":true,"data":{"entity":42,"stacks":["base"],"dry_run":true,"status":"queued","log":null,"request":"72"},"ts":"2026-11-03T00:00:00Z"}
```

Source: `dashboard/api/routes/provisioning.php:996`

3.9 Inventory · 62 routes

Inventory is the largest domain. It is backed by the SoR on cesium; a route answers `503` when the SoR is unreachable. It covers product types and barcodes (the catalogue), items (`components`, stored as `hardware_units`), legacy models, locations and container types, receipts, projects and allocations, and system codes and labels. This section folds in and supersedes `docs/SolariNet_Inventory_API.html` rev. 1.4.

Paging — Components, models, receipts, projects, allocations and `locations?flat=1` use the page envelope (§2.8). Omitting both `limit` and `offset` keeps the old bare-array shape.

Legacy aggregate — `GET /api/inventory` keeps its object shape (models, units, locations, receipts, projects, allocations) only while every collection has at most 500 rows. Beyond that it returns `413 inventory_page_required`. It takes no page parameters, so large inventories must use the collection routes.

Catalogue — A product type says *what* something is; every physical or software instance is a separate item. Kinds: hardware, peripheral, component, software, accessory, consumable, other. Names are trimmed and required. Any key inside `specs` that looks like a secret is rejected at any depth: password, token, API key, licence key, credential, private key. Licence data is a non-secret reference only.

Barcodes — Codes are upper-cased with whitespace removed, and unique while live. A declared `gtin` must have a valid length and check digit. `GET /api/inventory/product-barcodes?code=` is an exact lookup that returns a prefill candidate. It never creates anything and never calls an external service. Duplicates return `409 duplicate_barcode`.

Locations — Levels are property → floor → room → section, and a root must be a property. A `storage_unit` may sit under any level, including another storage unit. Live sibling names are unique, including among roots; soft-deleted rows release their name. The database enforces levels and rejects cycles. `PUT` refuses `parent_id`; use `/move`. Merge is two-step: `merge-preview` returns reference counts and a SHA-256 fingerprint, then `merge` takes that fingerprint and rechecks under lock. Repeating a completed merge returns the same result. Bulk storage-unit creation takes 1–500 units atomically.

Location errors — `409` with one of `duplicate_location`, `location_cycle`, `merge_conflict`, `container_not_empty` or `conflict` (stale `updated_at`). A merge fingerprint expires when either endpoint or any referenced row changes.

Reading consistently — Page requests are not one database transaction. A changed `page.total` or a duplicate id across pages means the data moved; restart the read. The dashboard does this at 1000 rows per page and refuses collections over 100 000 rows.

SINCE 037 **Add from the catalogue.** Items can be created directly from a product type: choose the type, a quantity and a location, and the API creates that many items in one step. Such items carry `product_type_id` and may have a null `model_id` (§6). **Software licences.** Software-kind items record seats, expiry, a reference-only pointer to where the licence lives (never a key or licence text), and the machines it is installed on. **Retired types** are hidden from the Components picker. **Location codes** are unique across all live locations, not only among siblings; a duplicate returns `409` .

COMBINED SNAPSHOT · 1

GET **/api/inventory** session

Returns the bounded legacy aggregate of live inventory collections.

Success `200` · {models:[model],units:[component],locations:[location tree],receipts:[receipt],projects:[project],allocations:[allocation]}

Errors
- `413 inventory_page_required` — Any collection exceeds 500 rows.

Tables `hardware_models` , `hardware_units` , `locations` , `receipts` , `projects` , `project_allocations`

Notes Response uses per-collection 500-row cap.

```
GET /api/inventory
→ {"ok":true,"data":{"models":[],"units":[],"locations":[],"receipts":[],"projects":[],"allocations":[]},"ts":"2026-01-01T00:00:00Z"}
```

Source: `dashboard/api/routes/inventory.php:102`

COMPONENTS (HARDWARE UNITS) · 8

GET **/api/inventory/components** session

Lists live hardware units with optional filters and pagination.

Query
- `stock_status` string — Live hardware_units.stock_status enum.
- `location_id` integer — Digits only.
- `model_id` integer — Digits only.
- `product_type_id` integer — Digits only.
- `q` string — Substring match over serial, asset tag, make, and model.
- `limit` integer default `500` — With offset, 1–5000; without offset, clamped to 1–5000.
- `offset` integer default `0` — With limit, 0–2147483647.

Success `200` · array of component rows, or {items,page} when limit or offset is supplied

Errors
- `400 bad_request` — Invalid filter, enum, or pagination value.

Tables `hardware_units` , `hardware_models` , `locations` , `entities` , `entity_hardware_unit_links`

Notes Without pagination parameters, returns at most 500 rows; send `limit` (up to 5000) to page.

```
GET /api/inventory/components?limit=20&offset=0
→ {"ok":true,"data":{"items":[],"page":{"limit":20,"offset":0,"total":0,"hasMore":false,"nextOffset":null}},"ts":"2026-01-01T00:00:00Z"}
```

Source: `dashboard/api/routes/inventory.php:129`

GET **/api/inventory/components/{id}** session

Returns one live component by ID.

Path
- `id` positive integer — Decimal digits, not zero.

Success `200` · component row

Errors
- `400 bad_request` — ID is not a positive integer.
- `404 not_found` — No component has that ID.

Tables `hardware_units` , `hardware_models` , `locations` , `entities` , `entity_hardware_unit_links`

```
GET /api/inventory/components/12
→ {"ok":true,"data":{"id":12,"modelId":3,"make":"Example","model":"Widget","hwType":"other"},"ts":"2026-01-01T00:00:00Z"}
```

Source: `dashboard/api/routes/inventory.php:172`

GET **/api/inventory/components/{id}/entity-link** session

Returns a component's current entity association and up to 200 eligible entity candidates.

Path
- `id` positive integer — Component ID.

Query
- `q` string — Escaped substring match on entity name or role.

Success `200` · {unit:{id,assetTag,serial},entity:null|{id,name,entityType,lifecycle},candidates:[{id,name,entityType,lifecycle}]}

Errors
- `400 bad_request` — ID is not positive.
- `404 not_found` — Component does not exist.

Tables `hardware_units` , `entities`

Notes Candidates are not automatically associated.

```
GET /api/inventory/components/12/entity-link?q=router
→ {"ok":true,"data":{"unit":{"id":12,"assetTag":"A-12","serial":"S-12"},"entity":null,"candidates":[]},"ts":"2026-01-01T00:00:00Z"}
```

Source: `dashboard/api/routes/inventory.php:184`

POST /api/inventory/components/{id}/entity-link

operator | CSRF: Origin + X-Solari-CSRF

Associates a component with a live SoR entity and records an audit entry.

Path
- `id` positive integer — Component ID.

Body
JSON object
- `entity_id` positive integer **required** — Integer or digit string.
- `note` string — Defaults to operator association note; maximum 255 bytes.

Success
`200` · {unitId,entityId,unchanged}

Errors
- `400 bad_request` — Invalid ID, entity_id, or note length.
- `404 not_found` — Component or live entity is absent.
- `409 entity_already_linked` — Entity belongs to another unit.
- `409 unit_already_linked` — Unit belongs to another entity or uniqueness constraint fires.
- `503 entity_link_failed` — PDO failure other than handled duplicate-link case.
- `500 entity_link_failed` — Other throwable during association.

Tables
`hardware_units`, `entities`, `audit_log`

Notes
Idempotent for an already matching association; locks involved rows and writes audit_log.

```
POST /api/inventory/components/12/entity-link {"entity_id":7}
→ {"ok":true,"data":{"unitId":12,"entityId":7,"unchanged":false},"ts":"2026-01-01T00:00:00Z"}
```

Source: `dashboard/api/routes/inventory.php:219`

DELETE /api/inventory/components/{id}/entity-link

operator | CSRF: Origin + X-Solari-CSRF

Removes a component's entity association and records an audit entry.

Path
- `id` positive integer — Component ID.

Body
JSON object
- `note` string — Defaults to operator unlink note; maximum 255 bytes.

Success
`200` · {unitId,entityId:null|integer,unchanged}

Errors
- `400 bad_request` — Invalid ID or note length.
- `404 not_found` — Component is absent.
- `500 entity_unlink_failed` — Throwable during unlink.

Tables
`hardware_units`, `entities`, `audit_log`

Notes
No existing link returns unchanged true.

```
DELETE /api/inventory/components/12/entity-link {}
→ {"ok":true,"data":{"unitId":12,"entityId":7,"unchanged":false},"ts":"2026-01-01T00:00:00Z"}
```

Source: `dashboard/api/routes/inventory.php:291`

POST /api/inventory/components

operator | CSRF: Origin + X-Solari-CSRF

Creates one item, or `count` identical items all-or-none, of a catalogue product type.

Body
JSON object
- `product_type_id` integer **required** — A live (not retired) product type. Required unless `model_id` is sent.
- `model_id` integer|null — Legacy hardware model. Alone, it derives the product type; with `product_type_id`, the two must agree.
- `count` integer — 1–50. Creates that many rows in one transaction. When sent, the response is `{items,count}`.
- `serial` string — Trimmed, maximum 128 bytes. Only with `count` 1; unique per product type.
- `asset_tag` string — Trimmed, maximum 64 bytes. Only with `count` 1.
- `location_id` integer|null — Must identify a live location.
- `acquired_on` date|null — YYYY-MM-DD.
- `quantity` positive integer — Defaults to 1.
- `unit_cost_cents` non-negative integer|null — Integer cents.
- `stock_status` enum — Defaults to in_stock.
- `receipt_id` integer|null — Must identify a live receipt.
- `notes` string|null — Trimmed.
- `licence_seats` positive integer|null — Software items only; 1–4294967295.
- `licence_expires_on` date|null — Software items only; a real YYYY-MM-DD date.
- `licence_reference` string|null — Software items only; where the licence is kept (vault item, order number, URL), at most 1024 bytes. Values that look like a key, password or token are rejected.
- `installed_on_unit_id` integer|null — Software items only; another live, non-software item.

Success
`201` · component row; `{items:[row…],count}` when `count` is sent

Errors
- `400 bad_request` — No product type, retired type, model/type disagreement, count out of range, serial or asset tag with count > 1, licence fields on a non-software item, or any field invalid.
- `409 conflict` — Trigger SIGNAL, duplicate serial, constraint violation, or a deadlock/lock timeout ("try again"; safe to retry).

Tables
`hardware_units`, `product_types`, `hardware_models`, `locations`, `receipts`

Notes
Create stamps dashboard source and human assertion. `model_id` may be null on the stored row.

```
POST /api/inventory/components {"product_type_id":7,"count":4,"location_id":12}
→ {"ok":true,"data":{"items":[{"id":12,"productTypeId":7,"modelId":null,"stockStatus":"in_stock"},…],"count":4},"ts":"2026-01-01T00:00:00Z"}
```

Source: `dashboard/api/routes/inventory.php:349`

PUT /api/inventory/components/{id} operator CSRF: Origin + X-Solari-CSRF

Updates supplied fields on a live component.

Path
- `id` positive integer — Component ID.

Body
JSON object
- `product_type_id` integer — A live product type; cannot be cleared. The current type is accepted even if since retired.
- `model_id` integer|null — Legacy model; null clears it. Must agree with the product type.
- `serial` string|null — Maximum 128 bytes.
- `asset_tag` string|null — Maximum 64 bytes.
- `location_id` integer|null — Live location.
- `acquired_on` date|null — YYYY-MM-DD.
- `quantity` positive integer — Defaults to 1 when supplied empty.
- `unit_cost_cents` non-negative integer|null — Integer cents.
- `stock_status` enum — Live schema enum.
- `receipt_id` integer|null — Live receipt.
- `notes` string|null — Trimmed.
- `licence_seats` positive integer|null — Software items only; 1–4294967295.
- `licence_expires_on` date|null — Software items only; a real YYYY-MM-DD date.
- `licence_reference` string|null — Software items only; where the licence is kept (vault item, order number, URL), at most 1024 bytes. Values that look like a key, password or token are rejected.
- `installed_on_unit_id` integer|null — Software items only; another live, non-software item.

Success
`200` · component row

Errors
- `400 bad_request` — Invalid ID, invalid field, cleared or retired product type, model/type disagreement, licence fields on a non-software item, or no fields.
- `404 not_found` — Component absent.
- `409 conflict` — Trigger SIGNAL, constraint violation, or a deadlock/lock timeout ("try again").

Tables
`hardware_units`, `product_types`, `hardware_models`, `locations`, `receipts`

Notes
No optimistic lock: the update does not check `updated_at`, so the last writer wins (§2.9).

```
PUT /api/inventory/components/12 {"notes":"checked"}
→ {"ok":true,"data":{"id":12,"notes":"checked"},"ts":"2026-01-01T00:00:00Z"}
```

Source: `dashboard/api/routes/inventory.php:413`

DELETE /api/inventory/components/{id} operator CSRF: Origin + X-Solari-CSRF

Soft-deletes a component.

Path
- `id` positive integer — Component ID.

Success
`200` · {id,status:"deleted"}

Errors
- `400 bad_request` — Invalid ID.
- `404 not_found` — Component absent.
- `409 conflict` — Constraint violation.

Tables
`hardware_units`

Notes
Sets deleted_at. No `updated_at` check (§2.9).

```
DELETE /api/inventory/components/12
→ {"ok":true,"data":{"id":12,"status":"deleted"},"ts":"2026-01-01T00:00:00Z"}
```

Source: `dashboard/api/routes/inventory.php:480`

HARDWARE MODELS (LEGACY PROJECTION) · 4

GET /api/inventory/models session

Lists live hardware models with optional filters and pagination.

Query
- `hw_type` string — Live hardware_models enum.
- `q` string — Substring match on make or model.
- `limit` integer default `500` — 1–5000 when pagination used.
- `offset` integer default `0` — 0–2147483647; enables pagination.

Success
`200` · array of model rows, or {items,page}

Errors
- `400 bad_request` — Invalid enum or pagination.

Tables
`hardware_models`

```
GET /api/inventory/models?limit=20
→ {"ok":true,"data":{"items":[],"page":{"limit":20,"offset":0,"total":0,"hasMore":false,"nextOffset":null}},"ts":"2026-01-01T00:00:00Z"}
```

Source: `dashboard/api/routes/inventory.php:435`

GET /api/inventory/models/{id} session

Returns one live hardware model.

Path
- `id` positive integer — Model ID.

Success
`200` · model row {id,make,model,hwType,arch,notes,sourceId,createdAt,updatedAt}

Errors
- `400 bad_request` — Invalid ID.
- `404 not_found` — Model absent.

Tables
`hardware_models`

```
GET /api/inventory/models/3
→ {"ok":true,"data":{"id":3,"make":"Example","model":"Widget","hwType":"other","arch":null},"ts":"2026-01-01T00:00:00Z"}
```

Source: `dashboard/api/routes/inventory.php:460`

POST /api/inventory/models

operator | CSRF: Origin + X-Solari-CSRF

Creates a catalog product type and its hardware-model projection with audit entries.

Body JSON object

- `make` string **required** — Trimmed, maximum 128 bytes.
- `model` string **required** — Trimmed, maximum 128 bytes.
- `hw_type` enum — Defaults to other.
- `arch` enum|null — Absent, null, or empty becomes null.
- `notes` string|null — Trimmed.

Success `201` · model row

Errors

- `400 bad_request` — Invalid required strings, type, or arch.
- `409 conflict` — Constraint violation.

Tables `product_types`, `hardware_models`, `audit_log`

Notes Writes product_types and hardware_models in one transaction.

```
POST /api/inventory/models {"make":"Example","model":"Widget"}
→ {"ok":true,"data":{"id":3,"make":"Example","model":"Widget","hwType":"other","arch":null},"ts":"2026-01-01T00:00:00Z"}
```

Source: `dashboard/api/routes/inventory.php:469`

PUT /api/inventory/models/{id}

operator | CSRF: Origin + X-Solari-CSRF

Updates only a model's architecture assertion.

Path

- `id` positive integer — Model ID.

Body JSON object

- `arch` enum|null **required** — Only permitted property; null or empty clears to unknown.

Success `200` · model row

Errors

- `400 bad_request` — Body contains fields other than arch or invalid arch.
- `404 not_found` — Model absent.
- `409 conflict` — Constraint violation.

Tables `hardware_models`

Notes Reasserts dashboard source and human provenance.

```
PUT /api/inventory/models/3 {"arch":null}
→ {"ok":true,"data":{"id":3,"arch":null},"ts":"2026-01-01T00:00:00Z"}
```

Source: `dashboard/api/routes/inventory.php:509`

LOCATIONS & CONTAINERS · 14

GET /api/inventory/locations

session

Returns a location tree, direct children, or paginated flat list.

Query

- `flat` string — Must equal 1; cannot accompany parent_id.
- `parent_id` integer — Digits only; returns direct children.
- `limit` integer default `500` — Used with flat; 1–5000.
- `offset` integer default `0` — Used with flat; 0–2147483647.

Success `200` · tree roots with children, direct-child array, or flat={items,page}

Errors

- `400 bad_request` — Invalid flat/parent_id combination or pagination.

Tables `locations`

Notes flat=1 defaults to limit 500 and offset 0.

```
GET /api/inventory/locations
→ {"ok":true,"data":[],"ts":"2026-01-01T00:00:00Z"}
```

Source: `dashboard/api/routes/inventory.php:533`

GET /api/inventory/locations/{id}

session

Returns one live location.

Path

- `id` positive integer — Location ID.

Success `200` · location row including parentId, containerTypeId, assignedProjectId

Errors

- `400 bad_request` — Invalid ID.
- `404 not_found` — Location absent.

Tables `locations`

```
GET /api/inventory/locations/5
→ {"ok":true,"data":{"id":5,"parentId":null,"name":"Shelf","locType":"storage_unit"},"ts":"2026-01-01T00:00:00Z"}
```

Source: `dashboard/api/routes/inventory.php:562`

POST /api/inventory/locations operator CSRF: Origin + X-Solari-CSRF

Creates a location after validating hierarchy and sibling-name constraints.

Body JSON object

- `name` string **required** — Trimmed, maximum 128 bytes.
- `loc_type` enum **required** — Live locations enum.
- `parent_id` integer|null — Optional valid location; transition constraints apply.
- `container_type_id` integer|null — Storage-unit metadata rules apply.
- `assigned_project_id` integer|null — Storage-unit metadata rules apply.
- `code` string|null — Maximum 32 bytes.
- `notes` string|null — Trimmed.

Success `201` · location row

Errors

- `400 bad_request` — Invalid fields, hierarchy, or sibling name.

Tables `locations`, `projects`, `location_container_types`, `audit_log`

Notes Writes audit_log transactionally.

```
POST /api/inventory/locations {"name":"Shelf","loc_type":"storage_unit"}
→ {"ok":true,"data":{"id":5,"name":"Shelf","locType":"storage_unit"},"ts":"2026-01-01T00:00:00Z"}
```

Source: `dashboard/api/routes/inventory.php:571`

PUT /api/inventory/locations/{id} operator CSRF: Origin + X-Solari-CSRF

Updates location fields with an updated_at optimistic lock.

Path

- `id` positive integer — Location ID.

Body JSON object

- `updated_at` string **required** — Must match current timestamp.
- `name` string — Cannot be cleared; maximum 128 bytes.
- `loc_type` enum — Live locations enum.
- `code` string|null — Maximum 32 bytes.
- `notes` string|null — Trimmed.
- `container_type_id` integer|null — Allowed only under storage_unit rules.
- `assigned_project_id` integer|null — Allowed only under storage_unit rules.

Success `200` · location row

Errors

- `400 bad_request` — Missing timestamp, parent_id supplied, no editable fields, or invalid field.
- `404 not_found` — Location absent or deleted.
- `409 conflict` — updated_at differs from current value.

Tables `locations`, `projects`, `hardware_units`, `audit_log`

Locking Requires updated_at; mismatch yields 409 conflict.

Notes Reparenting is rejected here; audit write is transactional.

```
PUT /api/inventory/locations/5 {"updated_at":"2026-01-01 00:00:00","name":"Shelf 2"}
→ {"ok":true,"data":{"id":5,"name":"Shelf 2"},"ts":"2026-01-01T00:00:00Z"}
```

Source: `dashboard/api/routes/inventory.php:608`

POST /api/inventory/locations/{id}/move operator CSRF: Origin + X-Solari-CSRF

Moves a live location to a new parent or makes it a root property.

Path

- `id` positive integer — Invalid ID returns bad_request 400.

Body JSON object

- `parent_id` positive integer|null **required** — Null makes a root; root locations must be properties, and parent/level/cycle/sibling rules apply.

Success `200` · {id,parent_id,old_parent_id}

Errors

- `400 bad_request` — parent_id missing, invalid, or hierarchy transition invalid.
- `404 not_found` — Location or parent is absent/deleted.
- `409 location_cycle` — Move creates/traverses a cycle or chain exceeds 10000.
- `409 duplicate_location` — Live sibling name collides.
- `409 conflict` — Related constraint prevents change.

Tables `locations`, `audit_log`

Locking Row locks during transaction; no optimistic timestamp.

Notes Writes an audit row; hierarchy constraints and DB triggers are backstops.

```
POST /api/inventory/locations/42/move {"parent_id":12}
→ {"ok":true,"data":{"id":42,"parent_id":12,"old_parent_id":9},"ts":"2026-11-03T12:00:00Z"}
```

Source: `dashboard/api/routes/location_admin.php:33`

GET /api/inventory/locations/{id}/merge-preview session

Returns live source and target summaries, reference counts, and a fingerprint for a later merge.

Path
- `id` positive integer — Source location.

Query
- `target_id` positive integer **required** — Must differ from source.

Success `200` · {source:{id,parent_id,name,loc_type,code,assigned_project_id},target:{…},counts,fingerprint}

Errors
- `400 bad_request` — Invalid target_id or source equals target.
- `404 not_found` — Either live location is absent.

Tables `locations`, `information_schema`, `barcodes`, `barcode_scans`, `external_refs`

Notes Fingerprint must be echoed to merge; preview reads reference counts and schema FK metadata.

```
GET /api/inventory/locations/42/merge-preview?target_id=51
→ {"ok":true,"data":{"source":{"id":42,"parent_id":9,"name":"Shelf A","loc_type":"storage_unit","code":null,"assigned_project_id":null},"target":
{"id":51,"parent_id":9,"name":"Shelf B","loc_type":"storage_unit","code":null,"assigned_project_id":null},"counts":
{},"fingerprint":"aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa"},"ts":"2026-11-03T12:00:00Z"}
```

Source: `dashboard/api/routes/location_admin.php:66`

POST /api/inventory/locations/{id}/merge operator CSRF: Origin + X-Solari-CSRF

Merges a source location into a target, retargeting supported references and soft-deleting the source.

Path
- `id` positive integer — Source location.

Body JSON object
- `target_id` positive integer **required** — Must differ from source.
- `fingerprint` string **required** — 64 lowercase hexadecimal characters from preview.

Success `200` · {source_id,target_id,merged,idempotent,counts,moved}

Errors
- `400 bad_request` — Target missing/invalid, fingerprint malformed, or IDs equal.
- `404 not_found` — Source or target is unavailable, except completed repeat merge.
- `409 merge_conflict` — Unsupported FK, stale fingerprint, type/project mismatch, or child collision.
- `409 location_cycle` — Target lies in source subtree.
- `409 duplicate_location` — Constraint reports duplicate.
- `409 conflict` — Related database constraint prevents change.

Tables `locations`, `barcodes`, `barcode_scans`, `external_refs`, `audit_log`, `information_schema`

Locking Locks source, target, references, and children in a transaction; no optimistic timestamp.

Notes Retargets supported references and live location barcodes; scan and external-reference history stays pointed at source. A repeat of an already committed merge is an idempotent success.

```
POST /api/inventory/locations/42/merge {"target_id":51,"fingerprint":"aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa"}
→ {"ok":true,"data":{"source_id":42,"target_id":51,"merged":true,"idempotent":false,"counts":{},"moved":{}},"ts":"2026-11-03T12:00:00Z"}
```

Source: `dashboard/api/routes/location_admin.php:94`

POST /api/inventory/locations/bulk-storage-units operator CSRF: Origin + X-Solari-CSRF

Creates 1–500 storage units under a parent, optionally assigning codes, container type, and project.

Body JSON object
- `parent_id` positive integer|null **required** — May be null only where hierarchy rules allow storage-unit root.
- `count` integer **required** — 1–500.
- `names` string[] — Must have exactly count entries; each trimmed name 1–128 characters.
- `prefix` string — Used without names; trimmed nonempty, max 100 characters.
- `start` integer — Required with prefix; non-negative.
- `code_prefix` string|null — Trimmed nonempty, max 20 characters.
- `code_start` integer — Defaults 0; non-negative when code_prefix supplied.
- `container_type_id` positive integer|null — Must identify live container type.
- `project_id` positive integer|null — Must identify live project.

Success `201` · {ids,count}

Errors
- `400 bad_request` — Invalid count, names/prefix/start, code generation, or IDs.
- `404 not_found` — Container type, project, or parent missing.
- `409 duplicate_location` — Names/codes collide in batch or existing data.
- `409 location_cycle` — Hierarchy trigger rejects cycle/depth.
- `409 conflict` — Related constraint prevents change.

Tables `locations`, `container_types`, `projects`, `audit_log`

Locking One transaction; sibling/code checks lock rows; no optimistic timestamp.

Notes All-or-none batch; generated codes are prefix plus increasing integer and must not exceed 32 characters.

```
POST /api/inventory/locations/bulk-storage-units {"parent_id":12,"count":2,"prefix":"Bin ","start":1}
→ {"ok":true,"data":{"ids":[61,62],"count":2},"ts":"2026-11-03T12:00:00Z"}
```

Source: `dashboard/api/routes/location_admin.php:211`

PUT /api/inventory/locations/{id}/project

operator | CSRF: Origin + X-Solari-CSRF

Assigns a project to a storage-unit location or releases its assignment.

Path
- `id` positive integer — Location ID.

Body
JSON object
- `project_id` positive integer|null **required** — Null releases assignment; release requires an empty container.

Success
`200` · {id,project_id}

Errors
- `400 bad_request` — Invalid project_id or location is not a storage_unit.
- `404 not_found` — Location or requested project is missing.
- `409 container_not_empty` — Release attempted while live children or hardware units exist.

Tables
`locations`, `projects`, `hardware_units`, `audit_log`

Locking
Location is locked in transaction; no optimistic timestamp.

Notes
Assignment change is audited.

```
PUT /api/inventory/locations/42/project {"project_id":7}
→ {"ok":true,"data":{"id":42,"project_id":7},"ts":"2026-11-03T12:00:00Z"}
```

Source: `dashboard/api/routes/location_admin.php:313`

GET /api/inventory/container-types

session

Lists container types, including retired rows only when requested.

Query
- `include_deleted` string default `0` — Only 0 or 1.

Success
`200` · [{id,name,created_at,updated_at,deleted_at}]

Errors
- `400 bad_request` — include_deleted is not 0 or 1.

Tables
`container_types`

Notes
Sorted by name then ID; deleted rows excluded by default.

```
GET /api/inventory/container-types
→ {"ok":true,"data":[{"id":3,"name":"Drawer","created_at":"2026-01-01 00:00:00","updated_at":"2026-01-01 00:00:00","deleted_at":null}],"ts":"2026-11-03T12:00:00Z"}
```

Source: `dashboard/api/routes/location_admin.php:346`

POST /api/inventory/container-types

operator | CSRF: Origin + X-Solari-CSRF

Creates a container type.

Body
JSON object
- `name` string **required** — Trimmed, 1–64 characters.

Success
`201` · {id,name,created_at,updated_at,deleted_at}

Errors
- `400 bad_request` — Name is invalid.
- `409 duplicate_location` — Database unique key reports duplicate.

Tables
`container_types`, `audit_log`

Locking
Transactional insert; no optimistic timestamp.

Notes
Audits creation.

```
POST /api/inventory/container-types {"name":"Drawer"}
→ {"ok":true,"data":{"id":3,"name":"Drawer","created_at":"2026-01-01 00:00:00","updated_at":"2026-01-01 00:00:00","deleted_at":null},"ts":"2026-11-03T12:00:00Z"}
```

Source: `dashboard/api/routes/location_admin.php:355`

PUT /api/inventory/container-types/{id}

operator | CSRF: Origin + X-Solari-CSRF

Renames a live container type.

Path
- `id` positive integer — Container type ID.

Body
JSON object
- `name` string **required** — Trimmed, 1–64 characters.
- `updated_at` string **required** — Must match current raw DB timestamp or ISO form.

Success
`200` · {id,name,created_at,updated_at,deleted_at}

Errors
- `400 bad_request` — Name or updated_at invalid.
- `404 not_found` — No live container type.
- `409 conflict` — updated_at differs from locked row.
- `409 duplicate_location` — Database unique key reports duplicate.

Tables
`container_types`, `audit_log`

Locking
Optimistic compare via updated_at; mismatch conflict 409.

Notes
Audit records rename.

```
PUT /api/inventory/container-types/3 {"name":"Cabinet","updated_at":"2026-01-01T00:00:00Z"}
→ {"ok":true,"data":{"id":3,"name":"Cabinet","created_at":"2026-01-01 00:00:00","updated_at":"2026-02-01 00:00:00","deleted_at":null},"ts":"2026-11-03T12:00:00Z"}
```

Source: `dashboard/api/routes/location_admin.php:369`

DELETE /api/inventory/container-types/{id}

operator · CSRF: Origin + X-Solari-CSRF

Soft-deletes a live container type, retaining existing references.

Path
- `id` positive integer — Container type ID.

Success
`200` · {id,status}

Errors
- `404 not_found` — No live container type.
- `409 duplicate_location` — Constraint rejects deletion.
- `409 conflict` — Related database constraint prevents change.

Tables
`container_types`, `audit_log`

Locking
Row locked in transaction; no optimistic timestamp.

Notes
Retired type remains referenced by existing locations.

```
DELETE /api/inventory/container-types/3
→ {"ok":true,"data":{"id":3,"status":"deleted"},"ts":"2026-11-03T12:00:00Z"}
```

Source: `dashboard/api/routes/location_admin.php:396`

GET /api/inventory/locations/{id}/contents

session

Returns a live location, descendant locations, their live hardware units, and aggregate unit value.

Path
- `id` positive integer — Root location ID.

Success
`200` · {location,path,locations,units,unitCount,valueCents}

Errors
- `400 bad_request` — Invalid ID.
- `404 not_found` — No live location.

Tables
`locations`, `hardware_units`, `hardware_models`

Notes
Descendant traversal is limited to 16 levels; unit value sums unitCostCents multiplied by quantity.

```
GET /api/inventory/locations/42/contents
→ {"ok":true,"data":{"location":{"id":42,"name":"Shelf"},"path":"Lab › Shelf","locations":[],"units":[],"unitCount":0,"valueCents":0},"ts":"2026-11-03T12:00:00Z"}
```

Source: `dashboard/api/routes/inv_codes.php:573`

RECEIPTS · 5

GET /api/inventory/receipts

session

Lists live receipts with vendor and text filters.

Query
- `vendor` string — Substring match.
- `q` string — Substring across vendor, order_ref, notes.
- `limit` integer default `500` — 1–5000 when pagination used.
- `offset` integer default `0` — 0–2147483647.

Success
`200` · array of receipt rows, or {items,page}

Errors
- `400 bad_request` — Invalid pagination.

Tables
`receipts`

Notes
Receipt rows expose scan presence and URL, not server file path.

```
GET /api/inventory/receipts?limit=20
→ {"ok":true,"data":{"items":[],"page":{"limit":20,"offset":0,"total":0,"hasMore":false,"nextOffset":null}},"ts":"2026-01-01T00:00:00Z"}
```

Source: `dashboard/api/routes/inventory.php:687`

GET /api/inventory/receipts/{id}

session

Returns one live receipt's metadata.

Path
- `id` positive integer — Receipt ID.

Success
`200` · receipt row {id,vendor,purchasedOn,subtotalCents,currency,orderRef,notes,sourceId,hasScan,scanMime,scanUrl,createdAt,updatedAt}

Errors
- `400 bad_request` — Invalid ID.
- `404 not_found` — Receipt absent.

Tables
`receipts`

```
GET /api/inventory/receipts/9
→ {"ok":true,"data":{"id":9,"vendor":"Example","currency":"USD","hasScan":false,"scanUrl":null},"ts":"2026-01-01T00:00:00Z"}
```

Source: `dashboard/api/routes/inventory.php:712`

POST /api/inventory/receipts operator CSRF: Origin + X-Solari-CSRF

Creates a receipt record.

Body — JSON object
- `vendor` string|null — Trimmed, maximum 128 bytes.
- `purchased_on` date|null — YYYY-MM-DD.
- `subtotal_cents` non-negative integer|null — Integer cents.
- `currency` string — Three letters uppercased; invalid values silently use USD.
- `order_ref` string|null — Maximum 96 bytes.
- `notes` string|null — Trimmed.

Success — `201` · receipt row

Errors
- `400 bad_request` — Invalid date, amount, or string type.
- `409 conflict` — Constraint violation.

Tables — `receipts`

```
POST /api/inventory/receipts {"vendor":"Example"}
→ {"ok":true,"data":{"id":9,"vendor":"Example","currency":"USD","hasScan":false},"ts":"2026-01-01T00:00:00Z"}
```

Source: `dashboard/api/routes/inventory.php:721`

POST /api/inventory/receipts/{id}/scan operator CSRF: Origin + X-Solari-CSRF

Uploads and stores a receipt scan.

Path
- `id` positive integer — Receipt ID.

Body — multipart
- `file` file **required** — Must be non-empty and at most 25 MiB; sniffed MIME must be supported image or PDF.

Success — `200` · updated receipt row

Errors
- `400 bad_request` — Invalid ID, missing/failed upload, size, or MIME.
- `404 not_found` — Receipt absent.
- `409 conflict` — Constraint violation saving metadata.
- `500 internal_error` — Upload directory unavailable/unwritable or move fails.

Tables — `receipts`

Notes — Replaces prior file and best-effort deletes it; upload directory is SOLARI_UPLOAD_DIR/receipts or /var/lib/solarinet-dashboard/uploads/receipts.

```
POST /api/inventory/receipts/9/scan multipart file
→ {"ok":true,"data":{"id":9,"hasScan":true,"scanMime":"application/pdf","scanUrl":"/api/inventory/receipts/9/scan"},"ts":"2026-01-01T00:00:00Z"}
```

Source: `dashboard/api/routes/inventory.php:748`

GET /api/inventory/receipts/{id}/scan session

Streams a stored receipt scan as raw bytes.

Path
- `id` positive integer — Receipt ID.

Success — `200` · Raw file bytes with sniffed stored Content-Type; no JSON envelope.

Errors
- `400 bad_request` — Invalid ID.
- `404 not_found` — Scan absent or unreadable.

Tables — `receipts`

Notes — Private no-cache; inline disposition and nosniff header.

```
GET /api/inventory/receipts/9/scan
→ %PDF-...
```

Source: `dashboard/api/routes/inventory.php:812`

PROJECTS, ALLOCATIONS & SHOPPING · 11

GET /api/inventory/projects session

Lists live projects with status and text filters.

Query
- `status` enum — Live projects status enum.
- `q` string — Substring over name or code.
- `limit` integer default `500` — 1–5000 when paging.
- `offset` integer default `0` — 0–2147483647.

Success — `200` · array of project rows, or {items,page}

Errors
- `400 bad_request` — Invalid status or pagination.

Tables — `projects`

```
GET /api/inventory/projects?limit=20
→ {"ok":true,"data":{"items":[],"page":{"limit":20,"offset":0,"total":0,"hasMore":false,"nextOffset":null}},"ts":"2026-01-01T00:00:00Z"}
```

Source: `dashboard/api/routes/inventory.php:841`

GET /api/inventory/projects/{id} session

Returns a live project and its live allocations.

Path
- `id` positive integer — Project ID.

Success
`200` · {project:project row,allocations:[allocation row]}

Errors
- `400 bad_request` — Invalid ID.
- `404 not_found` — Project absent.

Tables
`projects`, `project_allocations`, `hardware_units`, `hardware_models`

```
GET /api/inventory/projects/4
→ {"ok":true,"data":{"project":{"id":4,"name":"Upgrade"},"allocations":[]},"ts":"2026-01-01T00:00:00Z"}
```

Source: `dashboard/api/routes/inventory.php:867`

GET /api/inventory/allocations session

Lists live allocations for live projects.

Query
- `project_id` positive integer — Digits only.
- `allocation` enum — Live project_allocations enum.
- `limit` integer default `500` — 1–5000 when paging.
- `offset` integer default `0` — 0–2147483647.

Success
`200` · array of allocation rows, or {items,page}

Errors
- `400 bad_request` — Invalid project_id, allocation enum, or pagination.

Tables
`project_allocations`, `projects`, `hardware_units`, `hardware_models`

```
GET /api/inventory/allocations?project_id=4
→ {"ok":true,"data":[],"ts":"2026-01-01T00:00:00Z"}
```

Source: `dashboard/api/routes/inventory.php:883`

POST /api/inventory/projects operator CSRF: Origin + X-Solari-CSRF

Creates a project.

Body
JSON object
- `name` string **required** — Trimmed, maximum 160 bytes.
- `code` string|null — Maximum 32 bytes.
- `status` enum — Defaults to planning.
- `tray_location_id` integer|null — Live location.
- `owner_user_id` integer|null — Live user.
- `description` string|null — Trimmed.
- `started_on` date|null — YYYY-MM-DD.
- `target_on` date|null — YYYY-MM-DD.
- `completed_on` date|null — YYYY-MM-DD.

Success
`201` · project row

Errors
- `400 bad_request` — Invalid fields or FK.
- `409 conflict` — Constraint violation.

Tables
`projects`, `locations`, `users`

```
POST /api/inventory/projects {"name":"Upgrade"}
→ {"ok":true,"data":{"id":4,"name":"Upgrade","status":"planning"},"ts":"2026-01-01T00:00:00Z"}
```

Source: `dashboard/api/routes/inventory.php:912`

PUT /api/inventory/projects/{id} operator CSRF: Origin + X-Solari-CSRF

Updates supplied project fields.

Path
- `id` positive integer — Project ID.

Body
JSON object
- `name` string — Cannot clear; maximum 160 bytes.
- `code` string|null — Maximum 32 bytes.
- `status` enum — Live enum.
- `tray_location_id` integer|null — Live location.
- `owner_user_id` integer|null — Live user.
- `description` string|null — Trimmed.
- `started_on` date|null — YYYY-MM-DD.
- `target_on` date|null — YYYY-MM-DD.
- `completed_on` date|null — YYYY-MM-DD.

Success
`200` · project row

Errors
- `400 bad_request` — Invalid ID/body or no editable fields.
- `404 not_found` — Project absent.
- `409 conflict` — Constraint violation.

Tables
`projects`, `locations`, `users`

```
PUT /api/inventory/projects/4 {"status":"active"}
→ {"ok":true,"data":{"id":4,"name":"Upgrade","status":"active"},"ts":"2026-01-01T00:00:00Z"}
```

Source: `dashboard/api/routes/inventory.php:942`

DELETE /api/inventory/projects/{id}

operator | CSRF: Origin + X-Solari-CSRF

Soft-deletes a project and its live allocations transactionally.

Path
- `id` positive integer — Project ID.

Success `200` · {id,status:"deleted"}

Errors
- `400 bad_request` — Invalid ID.
- `404 not_found` — Project absent.
- `409 conflict` — Constraint violation.

Tables `projects`, `project_allocations`

Notes Both rows receive deleted_at.

```
DELETE /api/inventory/projects/4
→ {"ok":true,"data":{"id":4,"status":"deleted"},"ts":"2026-01-01T00:00:00Z"}
```

Source: `dashboard/api/routes/inventory.php:991`

POST /api/inventory/projects/{id}/allocations

operator | CSRF: Origin + X-Solari-CSRF

Creates a project allocation for exactly one unit or model.

Path
- `id` positive integer — Project ID.

Body JSON object
- `hardware_unit_id` integer|null — Exactly one of this and hardware_model_id required.
- `hardware_model_id` integer|null — Exactly one of this and hardware_unit_id required.
- `allocation` enum — Defaults to needed.
- `quantity` positive integer — Defaults to 1.
- `notes` string|null — Trimmed.

Success `201` · allocation row

Errors
- `400 bad_request` — Invalid ID, project missing, XOR/FK/enum/quantity validation.
- `404 not_found` — Project absent.
- `409 conflict` — Double-commit trigger or constraint violation.

Tables `projects`, `project_allocations`, `hardware_units`, `hardware_models`

Notes Allocation invariant is exactly one live unit or model.

```
POST /api/inventory/projects/4/allocations {"hardware_model_id":3,"quantity":2}
→ {"ok":true,"data":{"id":17,"projectId":4,"hardwareModelId":3,"allocation":"needed","quantity":2},"ts":"2026-01-01T00:00:00Z"}
```

Source: `dashboard/api/routes/inventory.php:1008`

PUT /api/inventory/projects/{id}/allocations/{allocId}

operator | CSRF: Origin + X-Solari-CSRF

Updates allocation fields or transitions its allocation state.

Path
- `id` positive integer — Project ID.
- `allocId` positive integer — Allocation ID belonging to project.

Body JSON object
- `hardware_unit_id` integer|null — Merged with existing assignment; XOR invariant remains.
- `hardware_model_id` integer|null — Merged with existing assignment; XOR invariant remains.
- `allocation` enum — Live enum.
- `quantity` positive integer — Defaults to 1 when supplied empty.
- `notes` string|null — Trimmed.

Success `200` · allocation row

Errors
- `400 bad_request` — Invalid ID, invalid body, or no editable fields.
- `404 not_found` — Allocation absent from project.
- `409 conflict` — Double-commit trigger or constraint violation.

Tables `project_allocations`, `hardware_units`, `hardware_models`, `projects`

Notes Committed/installed unit updates stock_status and moves it to project tray when configured.

```
PUT /api/inventory/projects/4/allocations/17 {"allocation":"committed"}
→ {"ok":true,"data":{"id":17,"projectId":4,"allocation":"committed"},"ts":"2026-01-01T00:00:00Z"}
```

Source: `dashboard/api/routes/inventory.php:1040`

DELETE /api/inventory/projects/{id}/allocations/{allocId}

operator | CSRF: Origin + X-Solari-CSRF

Soft-deletes a project allocation.

Path
- `id` positive integer — Project ID.
- `allocId` positive integer — Allocation ID.

Success `200` · {id,status:"deleted"}

Errors
- `400 bad_request` — Invalid IDs.
- `404 not_found` — Allocation absent from project.
- `409 conflict` — Constraint violation.

Tables `project_allocations`

Notes Sets deleted_at.

```
DELETE /api/inventory/projects/4/allocations/17
→ {"ok":true,"data":{"id":17,"status":"deleted"},"ts":"2026-01-01T00:00:00Z"}
```

Source: `dashboard/api/routes/inventory.php:1106`

GET /api/inventory/shopping session

Lists the project shopping view, optionally filtered by project.

Query
- `project_id` integer — Digits only.

Success `200` · array of {projectId,project,hardwareModelId,make,model,hwType,qtyNeeded}

Errors
- `400 bad_request` — project_id is not digits.

Tables `v_project_shopping`

```
GET /api/inventory/shopping?project_id=4
→ {"ok":true,"data":[],"ts":"2026-01-01T00:00:00Z"}
```

Source: `dashboard/api/routes/inventory.php:1122`

GET /api/inventory/projects/{id}/rollup session

Returns a live project, optional tray and tray-subtree hardware units, and live allocations.

Path
- `id` positive integer — Project ID.

Success `200` · {project,tray,trayContents,allocations}

Errors
- `400 bad_request` — Invalid ID.
- `404 not_found` — No live project.

Tables `projects`, `locations`, `hardware_units`, `hardware_models`, `project_allocations`

Notes Tray contents use the subtree CTE capped at 16 descendant levels.

```
GET /api/inventory/projects/7/rollup
→ {"ok":true,"data":{"project":{"id":7,"name":"Lab"},"tray":null,"trayContents":[],"allocations":[]},"ts":"2026-11-03T12:00:00Z"}
```

Source: `dashboard/api/routes/inv_codes.php:610`

PRODUCT CATALOG · 8

GET /api/inventory/product-types session

Lists live product types with optional text and kind filters.

Query
- `q` string — Trimmed substring match against name, brand, model, category.
- `kind` string — hardware, peripheral, component, software, accessory, consumable, other.
- `limit` integer default `100` — 1–500.
- `offset` integer default `0` — 0–2147483647.

Success `200` · {items:[{id,kind,name,brand,model,category,specs,notes,image_url,lookup_source,lookup_ref,created_at,updated_at}],page:{limit,offset,total,hasMore,nextOffset}}

Errors
- `400 bad_request` — Filter type/value or pagination invalid.

Tables `product_types`

Notes Unknown query parameters are ignored.

```
GET /api/inventory/product-types?kind=hardware&limit=10
→ {"ok":true,"data":{"items":[{"id":4,"kind":"hardware","name":"Switch","brand":"Acme","model":"S1","category":null,"specs":
{},"notes":null,"image_url":null,"lookup_source":null,"lookup_ref":null,"created_at":"2026-01-01 00:00:00","updated_at":"2026-01-01 00:00:00"}],"page":
{"limit":10,"offset":0,"total":1,"hasMore":false,"nextOffset":null}},"ts":"2026-11-03T12:00:00Z"}
```

Source: `dashboard/api/routes/product_catalog.php:9`

GET /api/inventory/product-types/{id} session

Returns a live product type and its live barcodes.

Path
- `id` positive integer — Product type ID.

Success `200` · {id,kind,name,brand,model,category,specs,notes,image_url,lookup_source,lookup_ref,created_at,updated_at,barcodes:[{id,code,code_type}]}

Errors
- `400 bad_request` — Invalid path ID.
- `404 not_found` — No live product type.

Tables `product_types`, `product_barcodes`

Notes Specs JSON is decoded.

```
GET /api/inventory/product-types/4
→ {"ok":true,"data":{"id":4,"kind":"hardware","name":"Switch","brand":"Acme","model":"S1","category":null,"specs":
{},"notes":null,"image_url":null,"lookup_source":null,"lookup_ref":null,"created_at":"2026-01-01 00:00:00","updated_at":"2026-01-01 00:00:00","barcodes":
[]},"ts":"2026-11-03T12:00:00Z"}
```

Source: `dashboard/api/routes/product_catalog.php:31`

DELETE /api/inventory/product-types/{id} operator CSRF: Origin + X-Solari-CSRF

Soft-deletes a product type and soft-deletes its live barcodes.

Path
- `id` positive integer — Product type ID.

Success `200` · {id,status}

Errors
- `400 bad_request` — Invalid ID.
- `404 not_found` — No live product type.

Tables `product_types`, `product_barcodes`, `audit_log`

Locking Transactional soft-delete; no optimistic timestamp.

Notes In-use product types may be retired; existing item references remain.

```
DELETE /api/inventory/product-types/4
→ {"ok":true,"data":{"id":4,"status":"deleted"},"ts":"2026-11-03T12:00:00Z"}
```

Source: `dashboard/api/routes/product_catalog.php:42`

POST /api/inventory/product-types

operator | CSRF: Origin + X-Solari-CSRF

Creates a product type and optionally its first barcode.

Body JSON object

- `name` string **required** — Trimmed, 1–160 characters.
- `kind` string — Enum; defaults hardware.
- `brand` string|null — Max 128 bytes.
- `model` string|null — Max 128 bytes.
- `category` string|null — Max 128 bytes.
- `specs` object|null — Must be object; rejects nested secret/password/token/key/credential-like keys.
- `notes` string|null — Max 65535 bytes.
- `image_url` string|null — Max 1024 bytes; http(s) URL.
- `barcode` object — Optional {code,code_type}; normalized, max 128 bytes; code_type gtin/sku/other; GTIN check digit validated.

Success `201` · Product type row {id,kind,name,brand,model,category,specs,notes,image_url,lookup_source,lookup_ref,created_at,updated_at}

Errors

- `400 bad_request` — Unknown field, invalid field, specs shape/secret key, or barcode invalid.
- `409 duplicate_barcode` — Barcode unique key collision.
- `503 service_unavailable` — Caught PDO failure or SoR unavailable.

Tables `product_types`, `product_barcodes`, `audit_log`

Locking Transactional insert; no optimistic timestamp.

Notes Unsupported input keys are rejected.

```
POST /api/inventory/product-types {"name":"Switch","kind":"hardware"}
→ {"ok":true,"data":
{"id":4,"kind":"hardware","name":"Switch","brand":null,"model":null,"category":null,"specs":null,"notes":null,"image_url":null,"lookup_source":null,"lookup_ref":null,"
created_at":"2026-01-01 00:00:00","updated_at":"2026-01-01 00:00:00"},"ts":"2026-11-03T12:00:00Z"}
```

Source: `dashboard/api/routes/product_catalog.php:57`

PUT /api/inventory/product-types/{id}

operator | CSRF: Origin + X-Solari-CSRF

Updates supplied product-type fields and synchronizes a linked legacy hardware model.

Path

- `id` positive integer — Product type ID.

Body JSON object

- `updated_at` string — Optional; if supplied must equal raw current DB timestamp.
- `kind` string — Enum hardware, peripheral, component, software, accessory, consumable, other.
- `name` string — Trimmed, 1–160 characters.
- `brand` string|null — Max 128 bytes.
- `model` string|null — Max 128 bytes.
- `category` string|null — Max 128 bytes.
- `specs` object|null — Rejects non-object and secret-like nested keys.
- `notes` string|null — Max 65535 bytes.
- `image_url` string|null — Max 1024 bytes; http(s) URL.

Success `200` · Product type row {id,kind,name,brand,model,category,specs,notes,image_url,lookup_source,lookup_ref,created_at,updated_at}

Errors

- `400 bad_request` — Empty body, unsupported field, or field validation fails.
- `404 not_found` — No live product type.
- `409 conflict` — Expected timestamp mismatches or unique constraint collides.
- `503 service_unavailable` — Caught PDO failure or SoR unavailable.

Tables `product_types`, `hardware_models`, `audit_log`

Locking Optional updated_at comparison against locked row; mismatch conflict 409.

Notes PUT fails if no supported fields remain after updated_at is removed.

```
PUT /api/inventory/product-types/4 {"name":"Managed Switch","updated_at":"2026-01-01 00:00:00"}
→ {"ok":true,"data":{"id":4,"kind":"hardware","name":"Managed
Switch","brand":"Acme","model":"S1","category":null,"specs":null,"notes":null,"image_url":null,"lookup_source":null,"lookup_ref":null,"created_at":"2026-01-01
00:00:00","updated_at":"2026-02-01 00:00:00"},"ts":"2026-11-03T12:00:00Z"}
```

Source: `dashboard/api/routes/product_catalog.php:77`

GET /api/inventory/product-barcodes

session

Looks up a live product barcode by normalized code.

Query

- `code` string **required** — Whitespace stripped and uppercased; must not normalize to empty.

Success `200` · {found,productType,barcode?}

Errors

- `400 bad_request` — Code missing, wrong type, or empty after normalization.

Tables `product_barcodes`, `product_types`

Notes Not found returns {found:false,productType:null}.

```
GET /api/inventory/product-barcodes?code=abc
→ {"ok":true,"data":{"found":true,"productType":{"id":4,"kind":"hardware","name":"Switch"},"barcode":{"id":8,"code":"ABC","code_type":"sku"}},"ts":"2026-11-
03T12:00:00Z"}
```

Source: `dashboard/api/routes/product_catalog.php:123`

POST /api/inventory/product-barcodes operator CSRF: Origin + X-Solari-CSRF

Adds a barcode to a live product type.

Body JSON object

- `product_type_id` positive integer **required** — Must identify a live product type.
- `code` string **required** — Whitespace stripped, uppercase; 1–128 bytes.
- `code_type` string — gtin, sku, or other; defaults other; GTIN length/check digit validated.

Success `201` · {id,product_type_id,code,code_type}

Errors

- `400 bad_request` — Invalid ID, code, type, or GTIN.
- `404 not_found` — No live product type.
- `409 duplicate_barcode` — Unique barcode collision.
- `503 service_unavailable` — Caught PDO failure or SoR unavailable.

Tables `product_types`, `product_barcodes`, `audit_log`

Locking Transactional insert; no optimistic timestamp.

Notes Insert is audited.

```
POST /api/inventory/product-barcodes {"product_type_id":4,"code":"abc","code_type":"sku"}
→ {"ok":true,"data":{"id":8,"product_type_id":4,"code":"ABC","code_type":"sku"},"ts":"2026-11-03T12:00:00Z"}
```

Source: `dashboard/api/routes/product_catalog.php:130`

DELETE /api/inventory/product-barcodes/{id} operator CSRF: Origin + X-Solari-CSRF

Soft-deletes a live product barcode.

Path

- `id` positive integer — Barcode ID.

Success `200` · {id,status}

Errors

- `400 bad_request` — Invalid ID.
- `404 not_found` — No live barcode.

Tables `product_barcodes`, `audit_log`

Locking Transactional soft-delete; no optimistic timestamp.

Notes Writes an audit row.

```
DELETE /api/inventory/product-barcodes/8
→ {"ok":true,"data":{"id":8,"status":"deleted"},"ts":"2026-11-03T12:00:00Z"}
```

Source: `dashboard/api/routes/product_catalog.php:141`

CODES, SCAN & LABELS · 11

GET /api/inventory/codes session

Lists live inventory barcodes, newest first.

Query

- `subject_table` string — Whitelisted subject table.
- `subject_id` integer — Digits only.
- `code_type` enum — Validated against live barcodes.code_type enum.
- `q` string — Substring match on code.
- `limit` integer default `500` — Defaults for invalid/below 1; capped at 5000.

Success `200` · [{id,code,codeType,symbology,subjectTable,subjectId,labelPrintedAt,notes,sourceId,createdAt,updatedAt,labelUrl}]

Errors

- `400 bad_request` — Invalid subject table/id or code_type enum.

Tables `barcodes`, `information_schema`

Notes Deleted barcodes excluded; limit is not offset pagination.

```
GET /api/inventory/codes?subject_table=locations&limit=10
→ {"ok":true,"data":[{"id":10,"code":"LOC-
0001","codeType":"location","symbology":"both","subjectTable":"locations","subjectId":42,"labelPrintedAt":null,"notes":null,"sourceId":2,"createdAt":"2026-01-
01T00:00:00Z","updatedAt":"2026-01-01T00:00:00Z","labelUrl":"/api/inventory/codes/10/label"}],"ts":"2026-11-03T12:00:00Z"}
```

Source: `dashboard/api/routes/inv_codes.php:43`

POST /api/inventory/codes/lookup operator CSRF: Origin + X-Solari-CSRF

Resolves a scanned code to its live barcode and subject card, logging the lookup.

Body JSON object

- `code` string **required** — Trimmed required string, max 128 characters.

Success `200` · When found {found,barcode,subjectTable,subject}; otherwise {found:false,code,guessedType}.

Errors

- `400 bad_request` — Code missing or invalid.

Tables `barcodes`, `barcode_scans`, `hardware_units`, `locations`, `projects`, `interfaces`, `project_allocations`

Notes Every result logs lookup action; subject card may be null if subject was deleted.

```
POST /api/inventory/codes/lookup {"code":"AKO-0001"}
→ {"ok":true,"data":{"found":false,"code":"AKO-0001","guessedType":"asset_tag"},"ts":"2026-11-03T12:00:00Z"}
```

Source: `dashboard/api/routes/inv_codes.php:84`

POST /api/inventory/codes/receive

operator | CSRF: Origin + X-Solari-CSRF

Creates a hardware unit for an unknown scanned code and assigns an AKO asset tag.

Body JSON object

- `code` string **required** — Max 128 characters.
- `code_type` string — Live barcode enum; defaults guessed type, then coerces unsupported receive types to custom.
- `model_id` positive integer **required** — Must reference live hardware model.
- `serial` string|null — Trimmed, max 128.
- `location_id` positive integer|null — Must reference live location.
- `acquired_on` string|null — YYYY-MM-DD.
- `quantity` integer — Defaults 1; positive.
- `unit_cost_cents` integer|null — Non-negative.
- `receipt_id` positive integer|null — Must reference live receipt.
- `notes` string|null — Optional trimmed string.

Success `201` · {unit,assetTag,scannedCode}

Errors

- `400 bad_request` — Invalid fields, missing model_id, or FK validation.
- `409 conflict` — Scanned code already used or code insert collision.

Tables `barcodes`, `hardware_units`, `hardware_models`, `locations`, `receipts`, `barcode_scans`

Locking Unit, generated barcodes, and scan log written in one transaction.

Notes If scanned code equals minted tag, scannedCode is null.

```
POST /api/inventory/codes/receive {"code":"SN123","model_id":2}
→ {"ok":true,"data":{"unit":{"id":70},"assetTag":{"id":11,"code":"AKO-0002"},"scannedCode":{"id":12,"code":"SN123"}},"ts":"2026-11-03T12:00:00Z"}
```

Source: `dashboard/api/routes/inv_codes.php:114`

POST /api/inventory/codes/move

operator | CSRF: Origin + X-Solari-CSRF

Moves a live hardware unit to a live location and logs the move when location changes.

Body JSON object

- `hardware_unit_id` positive integer **required** — Must reference live hardware unit.
- `location_id` positive integer **required** — Must reference live location.

Success `200` · Hardware unit component row.

Errors

- `400 bad_request` — IDs missing, invalid, or reference no live row.

Tables `hardware_units`, `locations`, `barcodes`, `barcode_scans`, `hardware_models`

Locking Locks unit row in transaction; repeated same-location move is a no-op without log.

Notes Move log uses live asset tag when available, otherwise unit ID.

```
POST /api/inventory/codes/move {"hardware_unit_id":70,"location_id":42}
→ {"ok":true,"data":{"id":70,"locationId":42,"quantity":1},"ts":"2026-11-03T12:00:00Z"}
```

Source: `dashboard/api/routes/inv_codes.php:233`

POST /api/inventory/codes/associate

operator | CSRF: Origin + X-Solari-CSRF

Associates either a location as a project tray/bin or a hardware unit as a committed project allocation.

Body JSON object

- `project_id` positive integer **required** — Must reference live project.
- `location_id` positive integer — Exactly one of location_id or hardware_unit_id.
- `hardware_unit_id` positive integer — Exactly one of location_id or location_id.

Success `200` · Location mode {project,location,mode}; unit mode allocation row (201).

Errors

- `400 bad_request` — Missing/invalid IDs, both/neither subject supplied, or tray self-nesting.
- `409 conflict` — Cycle or allocation trigger/constraint conflict.

Tables `projects`, `locations`, `hardware_units`, `project_allocations`, `barcodes`, `barcode_scans`

Locking Transaction; project row locked for tray association; allocation trigger may park unit on tray.

Notes Unit-commit variant responds 201; location mode is idempotent for an already assigned tray/nested location.

```
POST /api/inventory/codes/associate {"project_id":7,"location_id":42}
→ {"ok":true,"data":{"project":{"id":7,"name":"Lab"},"location":{"id":42,"name":"Shelf"},"mode":"tray"},"ts":"2026-11-03T12:00:00Z"}
```

Source: `dashboard/api/routes/inv_codes.php:263`

GET /api/inventory/codes/history

session

Lists barcode scan/action history newest first with optional filters.

Query

- `code` string — Trimmed and truncated to 128 characters.
- `subject_table` string — Whitelisted subject table.
- `subject_id` integer — Digits only.
- `action` enum — Validated against barcode_scans.action enum.
- `limit` integer default `100` — Defaults invalid/below 1; capped at 1000.

Success `200` · [{id,code,barcodeId,subjectTable,subjectId,action,operator,detail,createdAt}]

Errors

- `400 bad_request` — Invalid subject table/id or action enum.

Tables `barcode_scans`, `information_schema`

Notes Unknown query parameters are ignored.

```
GET /api/inventory/codes/history?action=lookup
→ {"ok":true,"data":[{"id":5,"code":"AKO-0001","barcodeId":10,"subjectTable":"hardware_units","subjectId":70,"action":"lookup","operator":"operator","detail":
{"found":true},"createdAt":"2026-01-01T00:00:00Z"}],"ts":"2026-11-03T12:00:00Z"}
```

Source: `dashboard/api/routes/inv_codes.php:337`

POST /api/inventory/codes operator CSRF: Origin + X-Solari-CSRF

Creates or idempotently returns a barcode bound to a live subject.

Body — JSON object

- `code_type` enum **required** — Must be a live barcodes.code_type enum member.
- `symbology` enum — Live enum; defaults both.
- `subject_table` string **required** — hardware_units, locations, projects, interfaces, product_types.
- `subject_id` positive integer **required** — Must identify a live row.
- `code` string — Explicit code max 128; required for types without auto-prefix.
- `notes` string|null — Trimmed and truncated to 255.

Success — `201` · Barcode row {id,code,codeType,symbology,subjectTable,subjectId,labelPrintedAt,notes,sourceId,createdAt,updatedAt,labelUrl}

Errors

- `400 bad_request` — Missing/invalid type, subject, code or asset_tag exceeds 64.
- `409 conflict` — Code already bound elsewhere or auto-allocation exhausts three attempts.

Tables — `barcodes`, `hardware_units`, `locations`, `projects`, `interfaces`, `product_types`, `barcode_scans`

Locking — Transaction; locks subject for auto generation; existing exact supplied binding returns its ID.

Notes — Auto prefixes: AKO asset_tag, LOC location, PRJ project. Asset tags mirror to hardware_units.asset_tag.

```
POST /api/inventory/codes {"code_type":"asset_tag","subject_table":"hardware_units","subject_id":70}
→ {"ok":true,"data":{"id":11,"code":"AKO-
0002","codeType":"asset_tag","symbology":"both","subjectTable":"hardware_units","subjectId":70,"labelPrintedAt":null,"notes":null,"sourceId":2,"createdAt":"2026-01-
01T00:00:00Z","updatedAt":"2026-01-01T00:00:00Z","labelUrl":"/api/inventory/codes/11/label"},"ts":"2026-11-03T12:00:00Z"}
```

Source: `dashboard/api/routes/inv_codes.php:373`

GET /api/inventory/codes/{id}/label session

Renders and streams a barcode label as PNG or PDF bytes.

Path

- `id` positive integer — Barcode ID.

Query

- `format` string default `png` — Only exact pdf selects PDF; other values select PNG.
- `profile` string default `default` — default, makeid_small, nemonic; unknown uses default.
- `width_dots` integer default `profile width` — Invalid uses default; clamps 64–2048.
- `height_dots` integer default `profile height` — Invalid uses default; clamps 64–2048.
- `dpi` integer default `profile dpi` — Only 203 or 300 accepted, else default.

Success — `200` · Raw image/png or application/pdf bytes; no JSON envelope.

Errors

- `400 bad_request` — Invalid path ID.
- `404 not_found` — No live barcode.
- `503 label_renderer_unavailable` — Renderer reports missing dependencies.
- `500 internal_error` — Renderer fails or times out.

Tables — `barcodes`, `hardware_units`, `locations`, `projects`, `interfaces`, `hardware_models`

Notes — GET does not update label_printed_at.

```
GET /api/inventory/codes/10/label?format=png
→ raw PNG bytes
```

Source: `dashboard/api/routes/inv_codes.php:477`

POST /api/inventory/codes/{id}/printed operator CSRF: Origin + X-Solari-CSRF

Sets label_printed_at on a live barcode and logs the first print action.

Path

- `id` positive integer — Barcode ID.

Success — `200` · Barcode row {id,code,codeType,symbology,subjectTable,subjectId,labelPrintedAt,notes,sourceId,createdAt,updatedAt,labelUrl}

Errors

- `400 bad_request` — Invalid ID.
- `404 not_found` — No live barcode.

Tables — `barcodes`, `barcode_scans`

Locking — Transaction; update only when label_printed_at is null.

Notes — Repeated calls do not add another print log after first timestamp.

```
POST /api/inventory/codes/10/printed
→ {"ok":true,"data":{"id":10,"code":"LOC-0001","codeType":"location","symbology":"both","subjectTable":"locations","subjectId":42,"labelPrintedAt":"2026-11-
03T12:00:00Z","notes":null,"sourceId":2,"createdAt":"2026-01-01T00:00:00Z","updatedAt":"2026-01-01T00:00:00Z","labelUrl":"/api/inventory/codes/10/label"},"ts":"2026-
11-03T12:00:00Z"}
```

Source: `dashboard/api/routes/inv_codes.php:521`

GET /api/inventory/codes/{id} session

Returns a live barcode and its subject card.

Path

- `id` positive integer — Barcode ID.

Success — `200` · {barcode:{id,code,codeType,symbology,subjectTable,subjectId,labelPrintedAt,notes,sourceId,createdAt,updatedAt,labelUrl},subject}

Errors

- `400 bad_request` — Invalid ID.
- `404 not_found` — No live barcode.

Tables — `barcodes`, `hardware_units`, `locations`, `projects`, `interfaces`, `project_allocations`

Notes — Subject is null when the referenced subject has since been soft-deleted.

```
GET /api/inventory/codes/10
→ {"ok":true,"data":{"barcode":{"id":10,"code":"LOC-
0001","codeType":"location","symbology":"both","subjectTable":"locations","subjectId":42,"labelPrintedAt":null,"notes":null,"sourceId":2,"createdAt":"2026-01-
01T00:00:00Z","updatedAt":"2026-01-01T00:00:00Z","labelUrl":"/api/inventory/codes/10/label"},"subject":{"id":42,"name":"Shelf","path":"Lab ›
Shelf","unitCount":0}},"ts":"2026-11-03T12:00:00Z"}
```

Source: `dashboard/api/routes/inv_codes.php:540`

`DELETE` **`/api/inventory/codes/{id}`** `operator` `CSRF: Origin + X-Solari-CSRF`

Soft-deletes a live barcode and logs the deletion.

Path	• `id` positive integer — Barcode ID.
Success	`200` · {id,status}
Errors	• `400 bad_request` — Invalid ID. • `404 not_found` — No live barcode.
Tables	`barcodes`, `barcode_scans`
Locking	Transaction; no optimistic timestamp.
Notes	Barcode is soft-deleted; action log is written in same transaction.

```
DELETE /api/inventory/codes/10
→ {"ok":true,"data":{"id":10,"status":"deleted"},"ts":"2026-11-03T12:00:00Z"}
```

Source: `dashboard/api/routes/inv_codes.php:556`

3.10 RackWire · 12 routes

RackWire routes store rack and cable *plans* with version history, soft delete and restore, and a hardware library with port groups. `POST /api/rackwire/commit` resolves a plan's racks and cables into SoR locations and interconnects. It reports what was created, updated, skipped and committed; rerunning it is safe. Telemetry is a bounded key/value upload, truncated to `RW_TELEMETRY_MAX_KEYS` (256) keys.

`GET` **`/api/rackwire/plans`** `session`

Lists live plans as summaries without their plan JSON.

Success	`200` · [{id,name,slug,updatedAt,createdAt}]
Errors	Common errors only (§2.6).
Tables	`rw_plans`
Notes	Includes all live plans ordered by updated_at descending; no pagination.

```
GET /api/rackwire/plans
→ {"ok":true,"data":[{"id":12,"name":"Core rack","slug":"core-rack","updatedAt":"2026-11-01T12:00:00Z","createdAt":"2026-10-30T12:00:00Z"}],"ts":"2026-11-
03T00:00:00Z"}
```

Source: `dashboard/api/routes/rackwire.php:106`

`GET` **`/api/rackwire/plans/{id}`** `session`

Returns one live plan with its parsed plan, metadata, and plan hash.

Path	• `id` positive integer — Invalid values return bad_request/400.
Success	`200` · {id,name,slug,plan,planHash,thumbnail,notes,updatedAt,createdAt}
Errors	• `400 bad_request` — id is not a positive integer • `404 not_found` — no live plan has that id • `503 service_unavailable` — SoR storage is unconfigured or unreachable
Tables	`rw_plans`
Notes	The hash covers name, thumbnail, notes, and stored plan JSON.

```
GET /api/rackwire/plans/12
→ {"ok":true,"data":{"id":12,"name":"Core rack","slug":"core-rack","plan":{"devices":[],"cables":
[]},"planHash":"aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa","thumbnail":null,"notes":null,"updatedAt":"2026-11-01T12:00:00Z","createdAt":"2026-
10-30T12:00:00Z"},"ts":"2026-11-03T00:00:00Z"}
```

Source: `dashboard/api/routes/rackwire.php:117`

`GET` **`/api/rackwire/plans/{id}/versions`** `session`

Lists snapshots for a live plan, omitting each snapshot's plan JSON.

Path	• `id` positive integer — Invalid values return bad_request/400.
Success	`200` · [{id,planId,name,createdBy,createdAt}]
Errors	• `400 bad_request` — id is not a positive integer • `404 not_found` — no live plan has that id • `503 service_unavailable` — SoR storage is unconfigured or unreachable
Tables	`rw_plans`, `rw_plan_versions`
Notes	Sorted newest first by created_at then id; no pagination.

```
GET /api/rackwire/plans/12/versions
→ {"ok":true,"data":[{"id":41,"planId":12,"name":"before move","createdBy":"operator","createdAt":"2026-11-01T12:00:00Z"}],"ts":"2026-11-03T00:00:00Z"}
```

Source: `dashboard/api/routes/rackwire.php:131`

GET /api/rackwire/library session

Lists live hardware models with RackWire group definitions and live unit counts.

Success	`200` · [{id,vendor,name,model,kind,ru,drawW,note,budgetW,circuitA,circuitV,poeBudgetW,eff,isInternet,isBattery,groups,unitCount}]
Errors	• `503 service_unavailable` — SoR storage is unconfigured or unreachable
Tables	`hardware_models`, `rw_model_groups`, `hardware_units`
Notes	Missing or invalid stored groups become an empty array; device extras override model-derived name, kind, and note.

```
GET /api/rackwire/library
→ {"ok":true,"data":[{"id":"8","vendor":"ExampleCo","name":"Switch","model":"SW-
8","kind":"switch","ru":1,"drawW":20,"note":null,"budgetW":null,"circuitA":null,"circuitV":null,"poeBudgetW":null,"eff":null,"isInternet":null,"isBattery":null,"groups
":[],"unitCount":2}],"ts":"2026-11-03T00:00:00Z"}
```

Source: `dashboard/api/routes/rackwire.php:148`

GET /api/rackwire/telemetry session

Returns a monitoring frame for requested device keys, including available device and port metrics.

Query	• `keys` comma-separated strings — Trimmed, lowercased, deduplicated, empty values removed; only first 256 unique keys are used. Keys may match full host/gear identifiers, leading host labels, or gear names.
Success	`200` · {ts,devices:{key:{reachable,cpuPct?}},ports:{"key|ifName":{link,rxGbps?,txGbps?}}}
Errors	Common errors only (§2.6).
Tables	`local monitoring DB: node`, `hostCurrent`, `networkGear`, `gearInterfaceCurrent`
Notes	Releases the PHP session lock before querying. Gear reachability means lastSeenAt is within 300 seconds. CPU is averaged from cpuLoadMilli and clamped to 0–100; absent samples and unsupported uptime, load, temperature, speed, and error metrics are omitted. Empty keys return empty objects.

```
GET /api/rackwire/telemetry?keys=argon,sw-core
→ {"ok":true,"data":{"ts":1793664000000,"devices":{"argon":{"reachable":true,"cpuPct":12},"sw-core":{"reachable":true}},"ports":{"sw-core|eth0":
{"link":true,"rxGbps":0.012,"txGbps":0.004}}},"ts":"2026-11-03T00:00:00Z"}
```

Source: `dashboard/api/routes/rackwire.php:164`

GET /api/rackwire/sor/interconnects session

Lists up to 5000 live SoR interconnects with endpoint and assertion details.

Success	`200` · [{id,linkKind,label,lengthM,standard,jacket,notes,assertedKind,assertedAt,source,a:{entity,assetTag,ifName},b:{entity,assetTag,ifName}}]
Errors	• `503 service_unavailable` — SoR storage is unconfigured or unreachable
Tables	`interconnects`, `sources`, `entities`, `interfaces`, `hardware_units`
Notes	Ordered by interconnect id descending; rows are included regardless of source and expose source slug.

```
GET /api/rackwire/sor/interconnects
→ {"ok":true,"data":
[{"id":91,"linkKind":"copper","label":"uplink","lengthM":1.5,"standard":"cat6","jacket":"blue","notes":null,"assertedKind":"human","assertedAt":"2026-11-
01T12:00:00Z","source":"rackwire","a":{"entity":"Switch A","assetTag":"A-1","ifName":"eth0"},"b":{"entity":"Switch B","assetTag":"B-1","ifName":"eth0"}}],"ts":"2026-
11-03T00:00:00Z"}
```

Source: `dashboard/api/routes/rackwire.php:270`

POST /api/rackwire/plans operator CSRF: Origin + X-Solari-CSRF

Creates a plan or updates an existing RackWire-owned plan.

Body	JSON object • `string` string **required** — Trimmed, nonempty, at most 128 bytes. • `plan` object **required** — Must decode as an array/object; serialized as JSON. • `id` positive integer — When supplied selects update; invalid values return bad_request/400. • `expectedHash` 64-character hexadecimal string — Required for updates; otherwise plan_hash_required/409. • `thumbnail` string or null — If omitted on update, preserves stored value; empty or nonstring becomes null. • `notes` string or null — If omitted on update, preserves stored value; empty or nonstring becomes null.
Success	`201` · {id,name,slug,plan,planHash,thumbnail,notes,updatedAt,createdAt}
Errors	• `400 bad_request` — invalid id, missing/invalid name, missing/nonobject plan, or plan JSON serialization failure • `404 not_found` — update target is absent or no longer live • `409 plan_hash_required` — update lacks valid expectedHash • `409 conflict` — target belongs to another source or a constraint is violated • `409 plan_conflict` — expectedHash differs and requested state is not already applied • `503 busy` — row lock times out or deadlocks • `503 service_unavailable` — SoR unavailable or RackWire source is missing • `415 unsupported_media_type` — nonempty body lacks application/json • `400 bad_request` — nonempty body is not a JSON object
Tables	`rw_plans`, `sources`
Locking	Update locks the plan row; expectedHash is checked against name, thumbnail, notes, and plan JSON. Mismatch returns plan_conflict/409 unless the requested state is already present.
Notes	Updates preserve thumbnail/notes only when their keys are omitted; create and update return 201. Device and cable counts are derived from plan.devices and plan.cables.

```
POST /api/rackwire/plans with {"name":"Core rack","plan":{"devices":[],"cables":[]}}
→ {"ok":true,"data":{"id":12,"name":"Core rack","slug":"core-rack","plan":{"devices":[],"cables":
[]},"planHash":"aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa","thumbnail":null,"notes":null,"updatedAt":"2026-11-03T00:00:00Z","createdAt":"2026-
11-03T00:00:00Z"},"ts":"2026-11-03T00:00:00Z"}
```

Source: `dashboard/api/routes/rackwire.php:303`

POST /api/rackwire/plans/{id}/delete

operator | CSRF: Origin + X-Solari-CSRF

Soft-deletes a live plan owned by RackWire.

Path
- `id` positive integer — Invalid values return bad_request/400.

Success
`200` · {id,deleted}

Errors
- `400 bad_request` — id is not a positive integer
- `404 not_found` — plan is absent, deleted, or update affects no row
- `409 conflict` — plan belongs to another source
- `503 busy` — plan row lock times out or deadlocks
- `503 service_unavailable` — SoR unavailable or RackWire source is missing

Tables
`rw_plans`, `sources`

Notes
Sets deleted_at; does not physically remove the plan.

```
POST /api/rackwire/plans/12/delete
→ {"ok":true,"data":{"id":12,"deleted":true},"ts":"2026-11-03T00:00:00Z"}
```

Source: `dashboard/api/routes/rackwire.php:434`

POST /api/rackwire/plans/{id}/versions

operator | CSRF: Origin + X-Solari-CSRF

Saves a snapshot from the supplied plan or the plan's current stored JSON.

Path
- `id` positive integer — Invalid values return bad_request/400.

Body
JSON object
- `name` string — Nonempty strings are trimmed; otherwise stored as null.
- `plan` object — When present must be an array/object and serializable; otherwise snapshots persisted plan JSON.

Success
`201` · {id,planId,name,createdBy,createdAt}

Errors
- `400 bad_request` — invalid id, invalid inline plan, or malformed nonempty JSON body
- `404 not_found` — plan is absent/deleted
- `409 conflict` — plan belongs to another source or constraint violation
- `503 busy` — row lock times out or deadlocks
- `503 service_unavailable` — SoR unavailable or RackWire source is missing
- `415 unsupported_media_type` — nonempty body lacks application/json

Tables
`rw_plans`, `rw_plan_versions`, `sources`

Locking
Locks the live plan row during insert/count/prune to serialize saves.

Notes
Retains at most 25 snapshots per plan by deleting oldest first.

```
POST /api/rackwire/plans/12/versions with {"name":"before move"}
→ {"ok":true,"data":{"id":41,"planId":12,"name":"before move","createdBy":"operator","createdAt":"2026-11-03T00:00:00Z"},"ts":"2026-11-03T00:00:00Z"}
```

Source: `dashboard/api/routes/rackwire.php:458`

POST /api/rackwire/plans/{id}/restore

operator | CSRF: Origin + X-Solari-CSRF

Restores a snapshot's plan JSON onto a live RackWire-owned plan.

Path
- `id` positive integer — Invalid values return bad_request/400.

Body
JSON object
- `versionId` positive integer **required** — Missing value returns bad_request/400.
- `expectedHash` 64-character hexadecimal string **required** — Missing or invalid value returns plan_hash_required/409.

Success
`200` · {id,name,slug,plan,planHash,thumbnail,notes,updatedAt,createdAt}

Errors
- `400 bad_request` — invalid plan id/versionId or missing versionId
- `404 not_found` — plan or its requested version is absent
- `409 plan_hash_required` — expectedHash is invalid or missing
- `409 conflict` — plan belongs to another source or constraint violation
- `409 plan_conflict` — hash differs and snapshot is not already applied
- `503 busy` — row lock times out or deadlocks
- `503 service_unavailable` — SoR unavailable or RackWire source is missing
- `415 unsupported_media_type` — nonempty body lacks application/json
- `400 bad_request` — nonempty body is not a JSON object

Tables
`rw_plans`, `rw_plan_versions`, `sources`

Locking
Locks the plan row and compares expectedHash; mismatch returns plan_conflict/409 unless the snapshot JSON is already current.

Notes
Restore changes plan_json only; name, slug, thumbnail, notes, and denormalized counts are not updated by this handler.

```
POST /api/rackwire/plans/12/restore with {"versionId":41,"expectedHash":"aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa"}
→ {"ok":true,"data":{"id":12,"name":"Core rack","slug":"core-rack","plan":{"devices":[],"cables":
[]},"planHash":"bbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbb","thumbnail":null,"notes":null,"updatedAt":"2026-11-03T00:00:00Z","createdAt":"2026-
10-30T12:00:00Z"},"ts":"2026-11-03T00:00:00Z"}
```

Source: `dashboard/api/routes/rackwire.php:540`

POST /api/rackwire/library/{modelId}/groups

operator | CSRF: Origin + X-Solari-CSRF

Creates or updates RackWire group geometry and device extras for a hardware model.

Path

- `modelId` positive integer — Invalid values return bad_request/400.

Body

JSON object

- `groups` array — Defaults to empty array; nonarray returns bad_request/400. Elements are stored without per-element validation.
- `name` string — Copied to device_json when present; no type or length validation.
- `kind` string — Copied when present; no type or length validation.
- `ru` number — Copied when present; no type or range validation.
- `note` string — Copied when present; no type or length validation.
- `drawW` number — Copied when present; no type or range validation.
- `budgetW` number — Copied when present; no type or range validation.
- `circuitA` number — Copied when present; no type or range validation.
- `circuitV` number — Copied when present; no type or range validation.
- `poeBudgetW` number — Copied when present; no type or range validation.
- `eff` number — Copied when present; no type or range validation.
- `isInternet` boolean — Copied when present; no type validation.
- `isBattery` boolean — Copied when present; no type validation.

Success

`201` · {id,vendor,name,model,kind,ru,drawW,note,budgetW,circuitA,circuitV,poeBudgetW,eff,isInternet,isBattery,groups,unitCount}

Errors

- `400 bad_request` — invalid modelId, nonarray groups, group JSON serialization failure, or malformed nonempty JSON body
- `404 not_found` — hardware model is absent or deleted
- `409 conflict` — existing geometry belongs to another source or constraint violation
- `503 service_unavailable` — SoR unavailable or RackWire source is missing
- `415 unsupported_media_type` — nonempty body lacks application/json

Tables

`hardware_models`, `rw_model_groups`, `hardware_units`, `sources`

Notes

An omitted groups field overwrites geometry with an empty array; only the listed extras are persisted. Successful upsert returns 201.

```
POST /api/rackwire/library/8/groups with {"groups":[]}
→ {"ok":true,"data":{"id":"8","vendor":"ExampleCo","name":"Switch","model":"SW-
8","kind":"switch","ru":1,"drawW":null,"note":null,"budgetW":null,"circuitA":null,"circuitV":null,"poeBudgetW":null,"eff":null,"isInternet":null,"isBattery":null,"grou
ps":[],"unitCount":2},"ts":"2026-11-03T00:00:00Z"}
```

Source: `dashboard/api/routes/rackwire.php:624`

POST /api/rackwire/commit

operator | CSRF: Origin + X-Solari-CSRF

Resolves plan racks and cables into SoR locations and interconnects, reporting created, updated, skipped, and committed items.

Body

JSON object

- `planId` positive integer — Selects a stored live plan when inline plan is not supplied.
- `plan` object — Inline array/object takes precedence over planId.
- `dryRun` boolean-like — PHP empty() semantics; truthy value performs calculations and rolls back writes.

Success

`200` · {created,updated,skipped:[string],committed:[string],dryRun}

Errors

- `400 bad_request` — invalid planId, neither usable plan nor planId, or malformed nonempty JSON body
- `404 not_found` — referenced live plan is absent
- `503 service_unavailable` — SoR unavailable or RackWire source is missing
- `415 unsupported_media_type` — nonempty body lacks application/json

Tables

`rw_plans`, `sources`, `locations`, `hardware_units`, `entity_hardware_unit_links`, `entities`, `interfaces`, `interconnects`

Notes

One transaction covers the operation; dryRun rolls it back after computing the result. Racks are mapped beneath the first live property location or a new Rackwire property; cables with incomplete ends, unresolved entities, unsupported/disagreeing port domains, duplicate links, invalid lengthM (outside 0–9999.99), or oversized text are listed as skipped. Entity resolution prefers asset-tag associations, then unique entity name matches; interfaces resolve through ifMap or port id, with entity-level fallback. Cable text fields are limited to label 128, notes 255, standard 32, jacket 64 Unicode code points. No idempotency key is used.

```
POST /api/rackwire/commit with {"planId":12,"dryRun":true}
→ {"ok":true,"data":{"created":1,"updated":0,"skipped":[],"committed":["cable c1: copper — link_kind from port domain net, electrical connector/standard; port-level
identity"],"dryRun":true},"ts":"2026-11-03T00:00:00Z"}
```

Source: `dashboard/api/routes/rackwire.php:697`

4 Control socket & SCP wire protocol

Two binary-adjacent interfaces sit underneath the REST API. **solariCtl** is a local, line-oriented Unix socket. The dashboard uses it to ask the C server to act. **SCP** (SolariNet Control Protocol) is the mutual-TLS framed protocol between the server and every client and monitor agent. Neither is reachable from the network for integrators: build on the REST API (§2–3) and treat this chapter as the contract you must keep when you change the C tier.

4.1 solariCtl: socket and framing

Socket	`/run/solari/solariCtl.sock` (AF_UNIX). Configured by the server key `bind.server.ctlSocket`, with group from `ctl.socketGid`, mode `0660` (best effort). PHP reads the path from `SOLARI_CTL_SOCK`.
Framing	One request per connection: `VERB key=value key=value\n`. The reply is `OK [key=value …]\n` or `ERR <solariStatus> <message>\n`. Values containing spaces must be URL-encoded.
Authorization	`SO_PEERCRED` UID, never anything in the request. Root or the configured operator UID → **operator** (may call every verb). The configured dashboard UID (the PHP-FPM pool user) → **dashboard** (ordinary verbs plus the queue). Any other UID → **unknown**, denied.
Fail-closed default	The C allowlist names the dashboard-ordinary verbs explicitly (`src/server/solariCtl.c`). A new verb is privileged until it is added to that list.
Standby	On a standby server only `PING`, `REQUEST_GET` and `DISCOVER_STATUS` run.
PHP client	`dashboard/api/lib/SolariCtl.php`: `send()` (raw), `call()` (maps `ERR` to HTTP), `enqueue()`, `poll()`, `callQueued()`. The socket timeout is 5 s.

```
$ printf 'PING\n' | sudo socat - UNIX-CONNECT:/run/solari/solariCtl.sock
OK pong decommissionInline=1
```

This exchange was observed live on xenon on 2026-11-03. The socket there was `srw-rw---- jason:solari-ctl`. Operators can also use `/opt/solarinet/current/bin/solariCtlClient`. Run privileged verbs by hand only when following a runbook.

4.2 The privileged-work queue

The web server must never hold privilege. So the dashboard UID may not call privileged verbs directly; it **submits** them:

1. `REQUEST_SUBMIT verb=<PRIVILEGED_VERB> op=<operator> [idem=<key>] …args` → `OK request=<id>`. Only privileged inner verbs are accepted, and `op=` is mandatory. The row records the peer UID and the optional idempotency key, so a repeated key returns the existing request.
2. The server's queue worker claims the request and runs the inner verb with server privilege. The state moves `pending → claimed → done | failed`.
3. `REQUEST_GET request=<id>` returns state and result, scoped to the requester.

The PHP wrapper `callQueued()` does all three and returns early with `{request, status:"queued"}` once its wait budget (4 s by default) runs out (§2.10). Destructive verbs such as `DECOMMISSION` add a one-time confirmation token, consumed when the queue runs. Queued decommission tokens are redacted at rest (monitoring migration 033).

4.3 Verb table

Generated from `ctlVerbs[]` in `docs/authz/AUTHZ_MANIFEST.json`; there are 38 verbs.

- **Class** is the peer-ACL class: *ordinary*, *privileged*, *queue* or *public-enrollment*.
- **Operator UID**, **Destructive** and **Standby** are manifest metadata. The C ACL above is what enforces access.
- **PHP callers** lists the routes that invoke each verb.

Verb	Class	Operator UID	Destructive	Standby	PHP callers
`ADOPT`	ordinary	—	—	—	`POST /api/discovery/{discId}/adopt`
`ALERT_ACK`	ordinary	yes	—	—	`POST /api/alerts/ack` `POST /api/alerts/ack-all`
`APPROVE`	privileged	—	—	—	`POST /api/enrollments/{enrId}/approve`
`ASSET_PURGE`	privileged	yes	yes	—	`POST /api/assets/{assetId}/purge (callQueued poll)`
`ASSET_REMOVE`	privileged	yes	yes	—	`POST /api/assets/{assetId}/remove (callQueued poll)`
`ASSET_SET`	ordinary	—	—	—	`POST /api/assets/{assetId}`
`CONFIG_SET`	privileged	—	—	—	`POST /api/config (callQueued poll)`
`CONTROL`	privileged	—	—	—	none
`CRIT_SET`	ordinary	yes	—	—	`POST /api/nodes/{nodeId}/criticality` `POST /api/assets/{assetId}/criticality`
`DECOMMISSION`	privileged	yes	yes	—	`POST /api/control/decommission`
`DEPLOY`	privileged	yes	yes	—	`POST /api/control/deploy`
`DISCOVER`	privileged	yes	yes	—	`POST /api/discovery/scan`
`DISCOVER_STATUS`	ordinary	yes	—	yes	`GET /api/discovery/scans/{scanId}`

Verb	Class	Operator UID	Destructive	Standby	PHP callers
`ENROLL_ACK`	ordinary	—	—	—	`POST /api/enrollments/ack`
`ENROLL_SUBMIT`	ordinary	—	—	—	`POST /api/enrollments`
`ENROLL_TOKEN`	privileged	yes	yes	—	none
`FLEET_IMAGE`	privileged	yes	yes	—	`POST /api/control/fleet-image`
`FLEET_PROVISION`	privileged	yes	yes	—	`POST /api/control/fleet-provision`
`IGNORE`	ordinary	—	—	—	`POST /api/discovery/{discId}/ignore`
`IMAGE_REFRESH`	privileged	yes	yes	—	`POST /api/provisioning/image-refresh`
`LIFECYCLE_SET`	privileged	yes	yes	—	`POST /api/assets/{assetId}/lifecycle (callQueued poll)`
`PING`	ordinary	—	—	yes	`POST /api/control/decommission (capability probe)`
`POOL_DEL`	privileged	yes	yes	—	`POST /api/pools/{poolId}/delete (callQueued poll)`
`POOL_NEW`	ordinary	—	—	—	`POST /api/pools`
`POOL_SET`	ordinary	—	—	—	`POST /api/pools/{poolId}`
`PROVISION`	privileged	—	—	—	`POST /api/control/provision (callQueued poll)` `POST /api/nodes/{id}/config (callQueued poll)`
`PROV_STAGE`	privileged	yes	yes	—	`POST /api/provisioning/stage`
`REJECT`	privileged	—	—	—	`POST /api/enrollments/{enrId}/reject`
`REQUEST_GET`	enqueue	yes	—	yes	20 routes (queue envelope)
`REQUEST_SUBMIT`	enqueue	—	—	—	20 routes (queue envelope)
`RETIRE`	privileged	yes	yes	—	`POST /api/nodes/{nodeId}/retire`
`REVOKE_CERT`	privileged	yes	yes	—	none
`RULE_DEL`	privileged	yes	yes	—	`POST /api/rules/{ruleId}/delete (callQueued poll)`
`RULE_SET`	ordinary	—	—	—	`POST /api/rules/{ruleId}`
`SIGN`	privileged	yes	yes	—	`POST /api/enrollments/{enrId}/approve (server queue invokes SIGN during APPROVE)`
`STACK_APPLY`	privileged	yes	yes	—	`POST /api/provisioning/apply`
`SURVEY`	ordinary	—	—	—	`POST /api/control/survey`
`TARGET_REMOVE`	privileged	yes	yes	—	`POST /api/assets/{assetId}/targets/{targetId}/remove (callQueued poll)`

4.4 SCP transport

Library NNG 1.12 scalability protocols over TLS (Mbed TLS 3.6), wrapped in `lib/solari/solariNet.c`. Peer authentication is mandatory on both sides.

Server ports `7701` ingest (reports and ACKs) · `7702` survey and control (surveyor/respondent) · `7703` PUB (broadcast). Defaults are in `src/server/serverContext.c`.

Identity The certificate CN is `<role>.<fqdn>`, where role is `client`, `monitor` or `server`. Ingest derives the role from the leading label, resolves it against the enrolment record, and rejects any HELLO whose identity disagrees with the certificate. Inbound message types are then checked against a per-role matrix (`src/server/serverIngest.c`).

Enrolment Not an SCP message. A new agent POSTs a CSR to `/api/enrollments` using a 256-bit bearer token minted by `ENROLL_TOKEN`. The C bridge revalidates the token's digest, expiry, pre-bound host and role. An operator approves the request and the agent picks up its certificate with `/api/enrollments/ack`.

4.5 SCP frame format

Every integer is **big-endian**. `frameLen` counts the header plus payload, excluding itself and the CRC. The CRC-32 covers header plus payload. The maximum body is 1 MiB. The protocol version is `SCP_PROTO_VERSION = 1`.

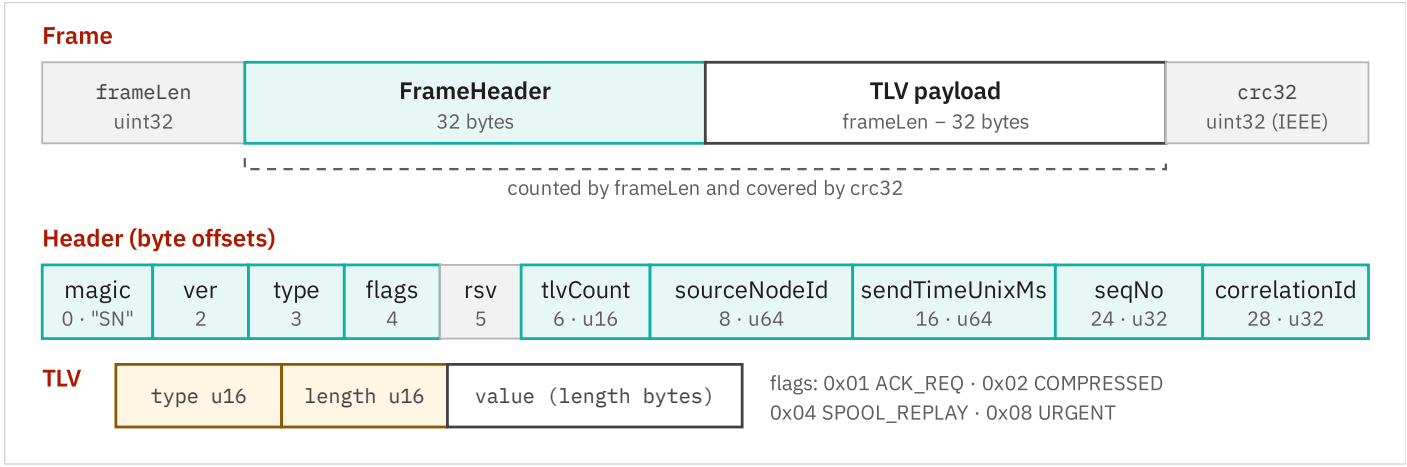


Figure 3. SCP wire frame (`include/solari/solariFrame.h`). All integers are big-endian, and the wire image is produced by the pack functions, not the C struct. Receivers skip TLV types they do not know.

Type	Value	Direction	Payload (TLVs)
`HELLO`	`0x01`	node → server	role, agent version, host FQDN, OS, arch, config epoch
`WELCOME`	`0x02`	server → node	session acknowledgement
`CLIENT_REPORT`	`0x10`	client → server	FQDN, OS/arch, per-core CPU, RAM/swap, disks, interfaces, USB, watched processes/logs, host-health counters
`MONITOR_REPORT`	`0x11`	monitor → server	FQDN, probe results, link stats
`WATCHDOG`	`0x12`	node → server	none; the header timestamp refreshes liveness
`DISCOVERY_ADVERT`	`0x13`	node → server	source, repeated `ip|mac|iface` entities, `proto|port|banner` services
`TOPOLOGY_REPORT`	`0x14`	node → server	uplink `localIf|gatewayIp|speedMbps`, LLDP `chassis|port|localIf`, segment `ifName|cidr`
`PEER_ALIVE`	`0x15`	monitor ↔ monitor	liveness between monitors
`SURVEY` / `SURVEY_RESP`	`0x20` / `0x21`	server ↔ fleet	the response is parsed as a monitor report
`CONTROL` / `CONTROL_RESULT`	`0x30` / `0x31`	server ↔ node	command; result correlated by `corrId`
`DECOMMISSION`	`0x32`	server → node	wipe scope
`WHO_IS_PRIMARY` / `PRIMARY_IS`	`0x40` / `0x41`	node ↔ server	failover lease query
`ACK` / `ERROR`	`0x50` / `0x51`	either	ERROR: `uint16` code + detail string

The TLV format is `uint16 type · uint16 length · value`, and receivers **skip unknown types**. That is what makes additive changes safe: add a new TLV type rather than changing the meaning of an existing one, and older servers will ignore it. The TLV registry lives in `include/solari/solariTlv.h`; message structs are in `include/solari/solariMsg.h`.

> **INFERRED** `CONTROL_RESULT`, `WHO_IS_PRIMARY` and `ACK` have no dedicated payload struct in `solariMsg.h`. Their shapes are defined by the handlers that build and parse them. Read `serverIngest.c` and the client control path before changing them.

5 Change data capture & messaging

SoR writes reach DNS, AD and other consumers through a transactional outbox, an emitter and RabbitMQ. If you want to *react* to SoR changes, add a consumer here; do not poll the SoR. Read `deploy/sorsync/README.md` together with this chapter.

5.1 Outbox

Migration 022 created two tables; 023, 024, 026 and 030 added the triggers.

`sor_outbox`

`id`	BIGINT auto-increment PK (the event's `outbox_id`)
`table_name`	source table
`row_id`	nullable row PK
`action`	`insert` | `update` | `delete`
`changed_at`	timestamp

`sor_sync_state`

`k`	PK: `emit_checkpoint`, `discovery_checkpoint`
`v`	last processed value
`updated_at`	timestamp

Each CDC table has `AFTER INSERT`, `AFTER UPDATE` and `AFTER DELETE` triggers that each insert one outbox row **in the same transaction** as the change. A rolled-back write therefore emits nothing. Insert and update use `NEW.id`; delete uses `OLD.id`.

Live, 2026-11-03. There are 69 outbox triggers across 23 tables. The other 15 triggers in the schema are validation and compatibility triggers, such as the 033/035 inventory bridge.

```
dhcp_ranges dns_records dns_zones entities entity_functions external_refs functions
function_incompatibility interfaces ip_addresses networks prov_images prov_runs
prov_software prov_software_artifacts prov_software_conflicts prov_software_requires
prov_stacks prov_stack_deps prov_stack_members sources subnets vlans
```

> **NOT IN CDC** Inventory tables (`hardware_units`, `product_types`, `locations`, ...) and RackWire tables do **not** emit outbox events. A consumer that needs inventory changes must first add triggers in a ledgered migration (§11.4).

5.2 Emitter (`sor_emitd`)

- Polls `sor_outbox WHERE id > emit_checkpoint ORDER BY id` in batches and publishes each row to the durable topic exchange **`sor.events`** with **publisher confirms**.
- Routing key `sor.<table>.<action>`, e.g. `sor.dns_records.update`. Messages use delivery mode 2 (persistent) and `content_type=application/json`, with no custom headers.
- Publishing is `mandatory`: an unroutable event (no bound queue) blocks the checkpoint. The exception is tables listed in `[amqp] optional_tables`, which may be published with no consumer.
- The checkpoint advances only after the **whole batch** is confirmed. Delivery is therefore **at-least-once**, and a consumer may see an event twice.
- Unit `sor-emit.service` runs on xenon. Its config is `SORSYNC_CONF` (default: the `sorsync.conf` next to the script), sections `[sor_emit]` / `[sor]` and `[amqp]`.

Event message

```
exchange:    sor.events   (topic, durable)
routing key: sor.ip_addresses.update
properties:  delivery_mode=2, content_type=application/json
body:
{"outbox_id": 160, "table": "ip_addresses", "row_id": 4211,
 "action": "update", "ts": 1762204750}
```

The event is a **change signal, not a row image**. `ts` is `UNIX_TIMESTAMP(changed_at)`. A consumer re-reads current SoR state, usually rendering the whole artefact it owns, so duplicate, reordered or missed events all converge to the same result.

> **OBSERVED** The live checkpoint read `emit_checkpoint=160` and `MAX(sor_outbox.id)=160`, so the emitter was caught up. The example above uses these values for illustration.

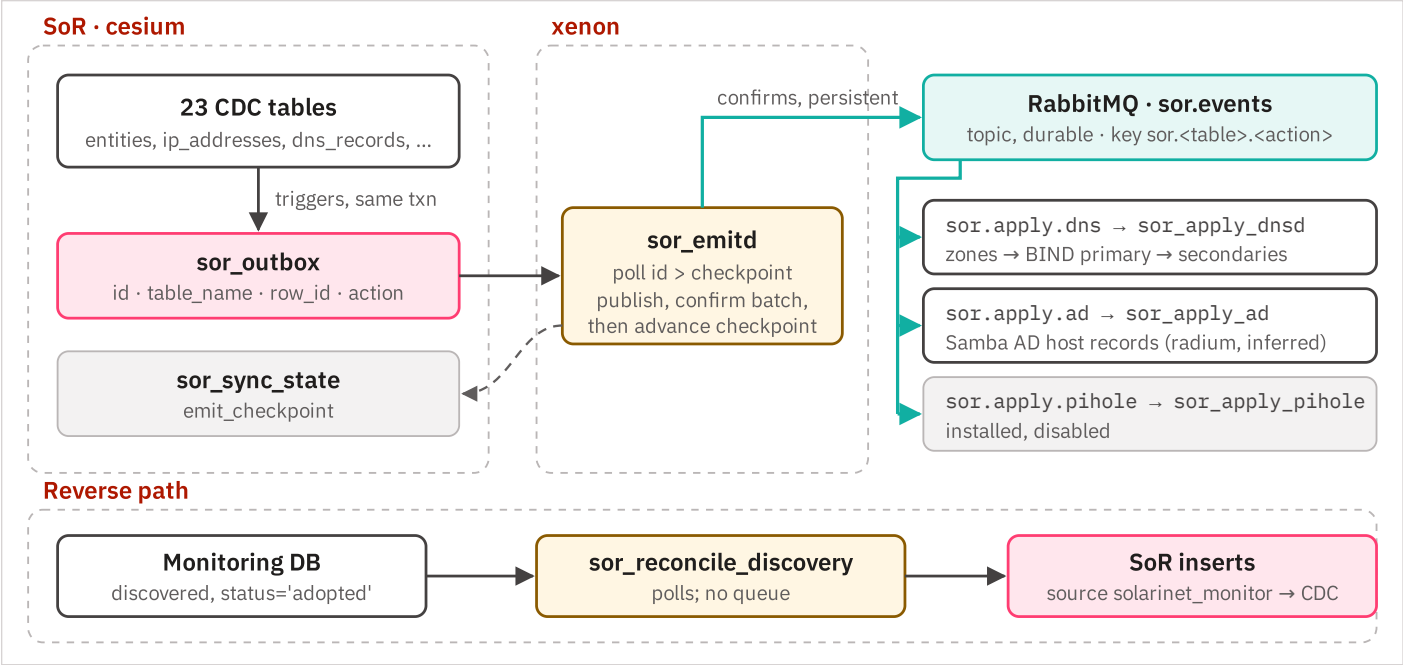


Figure 4. CDC forward path and the discovery reverse path. The outbox row is written in the same transaction as the change. The checkpoint advances only after a whole batch is confirmed, so delivery is at-least-once and consumers must re-read SoR state. Appliers ack only after a successful apply, and there is no dead-letter exchange.

5.3 Appliers

Applier	Queue	Binding keys	Effect	State	Live
`sor_apply_dnsd`	`sor.apply.dns`	`sor.{entities,ip_addresses,interfaces,dns_records,dns_zones,networks,subnets}.*`	Renders a full zone generation, releases it to the BIND primary on xenon and verifies it, then refreshes and SOA-verifies each `[dns.secondary.<name>]` (chlorine, steel)	JSON target-state file (`[dns] target_state_file`)	running on xenon
`sor_apply_ad`	`sor.apply.ad`	`sor.{entities,ip_addresses,interfaces}.*`	Reconciles host A records into Samba AD DNS via `samba-tool dns`. It deletes only records it created.	JSON owned-names file	`INFERRED` radium (not installed on xenon)
`sor_apply_pihole`	`sor.apply.pihole`	same as AD	Renders `custom.list` and reloads each target	none (compares remote file)	installed, **disabled**
`sor_reconcile_discovery`	*none*	—	Reverse path. Polls monitoring `discovered` rows (`status='adopted'`) and inserts the missing entities and IPs into the SoR with source `solarinet_monitor`. Its writes then flow through CDC.	reads rows directly; `discovery_checkpoint` unused	running on xenon (`sor-reconcile`)

Delivery contract (`deploy/sorsync/delivery.py`)

- `consume_after_apply()` sets prefetch, collects deliveries for `debounce_sec`, then calls the apply callback once for the batch.
- Acks happen **only if the callback returns exactly** `True`. An exception or `False` leaves the batch unacked; closing the connection requeues it and the loop reconnects after `reconnect_delay_sec`.
- Each applier reconciles once at start-up and every `reconcile_interval_sec`, which repairs anything missed while it was down.
- There is **no dead-letter exchange** and no retry queue. A permanently failing apply keeps redelivering, so watch the unit's journal.

> **BACKSTOP REFRESH** `solari-dns-secondary-refresh.service` re-pushes the secondary zone set from xenon to the secondaries. It reads `/etc/bind/solarinet-current/named.conf.managed` plus `/etc/bind/named.conf.rpz`, and its path unit watches `/etc/bind` so a release symlink swap triggers it. (Before 2026-11-03 it read the retired `named.conf.akoria` fragments and failed closed; fixed in `fbdb460`.)

5.4 Notification bus

A second exchange carries human-facing alerts (§9):

```
exchange:    notify.events   (topic, durable)
routing key: notify.<severity>         e.g. notify.critical
queue:       notify.dispatch  bound to notify.#
body:
{"severity":"critical","title":"cesium down","body":"no report for 300 s",
 "channel":"auto","to":null,"source":"alertbridge","ts":"2026-11-03T21:00:00Z"}
```

Only `title` or `body` is required. `alertbridge` publishes `{title, body, severity, source}`, and `notifyd` fills in defaults.

BROKER The live emitter points at AMQP vhost `solari` on `10.5.2.50:5672` (RabbitMQ on benzene), according to the `[amqp] url` in xenon's `sorsync.conf`. The URL contains credentials and is referenced by path only. The broker's own state was not inspected. Mosquitto (MQTT) is a separate broker that only the `mqtt` notify sender uses.

6 The SoR as a read surface

The System of Record is the MariaDB database `sor` on cesium (MariaDB 12.3.2). It is the canonical store for hosts, addressing, DNS, provisioning, inventory and RackWire. Reporting and integrations may **read** it directly. Every **write** goes through the REST API, because the API carries the provenance, locking and validation rules.

6.1 Access rules

1. **Read-only, always.** The only credential provisioned for tooling today is the broad application account `solari`, which has `ALL` on `sor.*` from `10.0.0.0/8`. Jason accepted this on 2026-11-02. Nothing in the database stops a write, so read-only is a discipline you must keep. A dedicated `SELECT`-only reporting account is a backlog item (§13).
2. **Never copy credentials.** Use the existing option file through a short-lived, owner-only copy, then shred it:

```
umask 077
sudo -n sed 's/^\[sor\]/[client]/' /etc/solarinet/prov-sor.cnf > /dev/shm/q.cnf
mariadb --defaults-file=/dev/shm/q.cnf -e "SELECT …"
shred -u /dev/shm/q.cnf
```

3. **Filter tombstones.** Nearly every business table soft-deletes. Add `WHERE deleted_at IS NULL` unless you want history.
4. **Never read `secrets`.** That table exists for legacy reasons. Licences and credentials are stored *by reference* only (remit decision 6); export nothing from it.
5. **Expect schema change.** Every migration is announced in `netdb/sor/migrations/`. Views (`v_*`) are the most stable read targets.

6.2 Data model at a glance

Live on 2026-11-03: 60 base tables and 4 views. Groupings follow `netdb/sor/SCHEMA.md`.

Area	Tables
Identity & provenance	`sources` (12 slugs: manual, dashboard, netdb, samba_ad, bind, forgejo, solarinet_monitor, unifi, pihole, rackwire, provisioner, scanner-ingest), `audit_log`, `users`, `groups`, `group_members`
Hosts & addressing	`entities`, `interfaces`, `ip_addresses`, `networks`, `subnets`, `vlans`, `dhcp_ranges`, `external_refs`, `operating_systems`, `relationships`, `services`, `service_endpoints`, `applications`, `monitoring_state`
DNS	`dns_zones`, `dns_records`; views `v_dns_hosts`, `v_dns_ifaces`, `v_dns_cnames`
Functions & provisioning	`functions`, `entity_functions`, `function_incompatibility`, `prov_images`, `prov_runs`, `prov_software` (+ `_artifacts`, `_requires`, `_conflicts`), `prov_stacks`, `prov_stack_members`, `prov_stack_deps`, `config_items`, `repositories`
PKI	`certificates`, `certificate_bindings`
Inventory	`product_types`, `product_barcodes`, `hardware_models` (legacy projection), `hardware_units`, `entity_hardware_unit_links`, `locations`, `container_types`, `receipts`, `projects`, `project_allocations`, `barcodes`, `barcode_scans`, `inventory_snapshots`; view `v_project_shopping`
RackWire	`rw_plans`, `rw_plan_versions`, `rw_model_groups`, `interconnects`, `power_circuits`
Plumbing	`sor_outbox`, `sor_sync_state` (§5), `solari_migration_ledger` (§12.3), `_solarinet_m036_location_baseline` (036 rollback baseline)

Provenance columns. Asserted rows carry `source_id → sources.id`, `asserted_kind` (`human`, `discovered`, …), `asserted_at`, `created_at` / `updated_at`, and `deleted_at` for soft delete. Filter by source to see who said what, e.g. rows the reconciler added with `source_id = (SELECT id FROM sources WHERE slug='solarinet_monitor')`.

> **SINCE 037** `hardware_units.model_id` is **nullable**. An item created from a catalogue product type carries `product_type_id` and may have no legacy `hardware_models` row. Reports must join on `product_type_id` first and treat `model_id` as optional. Software items also carry licence seats, expiry, a reference-only licence pointer and an "installed on" link to a non-software unit. Location `code` is unique among live locations across the whole tree.

6.3 IP addresses: the INET6 rule

Address columns (`ip_addresses.address` and others) use MariaDB's native `INET6` type. IPv4 values are stored as v4-mapped (`::ffff:10.0.0.1`). The legacy string functions silently produce wrong answers:

Expression (live result on 2026-11-03)	Use?
`WHERE address = CAST('10.0.0.1' AS INET6)` → 1 row	✔ preferred
`WHERE address = CAST('::ffff:10.0.0.1' AS INET6)` → 1 row	✔
`WHERE address = INET6_ATON('::ffff:10.0.0.1')` → 1 row	✔ (v4-mapped literal only)
`WHERE address = INET6_ATON('10.0.0.1')` → **0 rows**	✘ silent miss
`SELECT INET6_NTOA(address)` → **NULL**	✘ use `CAST(address AS CHAR)`

To display an address, take `CAST(address AS CHAR)` and strip a leading `::ffff:` for IPv4.

6.4 Useful read queries

```
-- live hosts with their primary IPv4 and OS
SELECT e.name, REPLACE(CAST(ip.address AS CHAR), '::ffff:', '') AS ipv4, os.name AS os
FROM entities e
LEFT JOIN ip_addresses ip ON ip.entity_id = e.id AND ip.deleted_at IS NULL
LEFT JOIN operating_systems os ON os.id = e.os_id
WHERE e.deleted_at IS NULL
ORDER BY e.name;

-- what DNS will publish (same view the DNS applier renders from)
SELECT * FROM v_dns_hosts ORDER BY 1 LIMIT 20;

-- migration head
SELECT migration_id, state, applied_at FROM solari_migration_ledger
WHERE migration_id <> 'schema' ORDER BY migration_id DESC LIMIT 1;
```

All three queries were run read-only against the live SoR on 2026-11-03. `v_dns_hosts` returns `host, ip, role, lifecycle`.

6.5 The monitoring database (separate)

Live telemetry lives in a **different** database: MariaDB 11.8.6 on xenon, configured through `SOLARI_DB_*`. It holds `hostCurrent` / `hostHistory`, `probeCurrent`, `alertEvent`, `discovered`, `networkGear`, `lldpEdge`, enrolment and queue tables. It has its own migration stream in `db/migrations/`, whose numbers **overlap** the SoR's. For example, monitoring 037 (`node_certcn_role_prefix`) is unrelated to SoR 037. Read it through the API (§3, Monitoring) rather than directly; its schema changes more often.

7 Status panel interface

The status panel is a Galactic Unicorn LED matrix (RP2350). It shows fleet health and sounds an alarm for critical episodes. Three contracts meet here: four REST routes under `/api/panel`, the `solariPanel` C daemon that bridges REST to the device, and a framed binary protocol over USB-CDC serial or a mutual-TLS socket. The authoritative wire definition is `status-panel/protocol.h`. This chapter summarises it, and the header wins if the two disagree.

> **THE ALARM TEST IS LOUD.** A critical alert or a tier-threshold breach arms the panel's audible alarm. Do not raise synthetic critical alerts to test an integration unless someone is beside the panel and ready to acknowledge it.

7.1 The chain

1. The dashboard composes a snapshot from systems, pools, alerts, probes, gear, the reported panel state, pending commands and the screen configuration: `GET /api/panel`.
2. `solariPanel` (systemd `solari-panel.service` on lithium) logs in as the local principal `panel`. It polls every `pollSec` (default 5 s) and re-sends the latest snapshot to the device every `snapshotSec` (default 2 s). If the data is more than 35 s old at send time it sets `dataStale`.
3. Frames go to the device over USB-CDC (`serialDev`, default glob `/dev/serial/by-id/*`, first match wins). They can also go over an optional raw-TLS listener for the standalone Wi-Fi panel. Both transports can run together.
4. The device answers with STATE and CONFIG frames. The daemon relays them back through `POST /api/panel/state` and `/api/panel/config`. Operator commands queued by `POST /api/panel/command` ride down as CONTROL frames.

7.2 REST contract

Route	Policy	Body	Response
`GET /api/panel`	session	—	Snapshot payload (systems, pools, alerts, probes, gear, `panelState`, pending commands, screen config).
`POST /api/panel/command`	operator + CSRF	`{kind, arg}`	`{cmdId}`. `409 queue_full` when 16 or more commands are pending. `403` for the panel principal itself.
`POST /api/panel/state`	session, panel principal only, `csrf: panel-service`	Device STATE fields	`{panelState:{theme, screen, brightnessPct, autoBright, sleeping, alarmArmed, alarmAcked, dwellSec, ackedEpisodeId, lastCmdId, reportedAt}}`. Commands up to `lastCmdId` are marked applied.
`POST /api/panel/config`	session, panel principal only, `csrf: panel-service`	Device CONFIG fields	`{panelScreenConfig:{screenCfg, flags, reportedAt} | null}`

Command kinds

Every kind is idempotent on the device, so a re-sent command is harmless.

kind	Name	arg
1	SETTHEME	0–3
2	SETSCREEN	0–2, within the current theme
3	BRIGHTNESS	0–100; latches manual brightness
4	AUTOBRIGHT	ignored; re-enables the light sensor
5	ACKALARM	ignored; same path as the physical acknowledge button
6	SETDWELL	seconds, one of 3, 6, 30
7	SLEEP	0 wake, 1 sleep
8	SCREENEN	`(screenIdx << 1) | enabled`, ≤ 23
9	SCREENWT	`(screenIdx << 3) | weightCode`; screenIdx ≤ 11, weightCode ≤ 5

The panel service principal and the CSRF bridge

The daemon authenticates as the local account `panel` with role **viewer**. Its password is read from `passFile`. The daemon refuses to start unless that file is owned by the service user and has mode `0600`.

The state and config routes declare `['csrf' => 'panel-service']`. `Policy::isCsrfBridged()` skips the CSRF check only when two things are true. First, the principal is exactly `source=local, username=panel, role=viewer`. Second, the request has no `Origin`, `Referer` or `Sec-Fetch-Site` header. Any browser-originated request still gets the full check.

The bridge exists for deployed daemons that predate `X-Solari-CSRF`. The code comment says it should be removed once the daemon echoes the login `csrfToken`. See the roadmap in §13.

7.3 Wire framing

Header 6 bytes: magic `A5 53`, version (`0x01`), type, then u16 `payloadLen`.

Trailer 2-byte CRC-16/CCITT (poly `0x1021`, init `0xFFFF`). It covers version, type, length and payload, but not the magic.

Parser The parser scans byte by byte for the magic. It rejects an oversize length before buffering anything. A frame with a bad CRC is dropped, and scanning resumes inside it. Frames with an unknown type or version are skipped without losing sync. Snapshots with a duplicate or older `seq` are ignored; the comparison is RFC 1982 wraparound-aware.

Limits Maximum payload 2048 bytes. A partial frame is abandoned after 500 ms.

Byte order Little-endian throughout.

Host → panel

Type	Name	Payload
`0x01`	SNAPSHOT	Fixed layout, below
`0x02`	PING	—
`0x03`	HELLOREQ	—
`0x04`	CONTROL	8 B: u32 cmdId, u8 kind, u8 arg, 2 reserved
`0x05`	PROVISION	TLV, 192-byte chunks

Panel → host

Type	Name	Payload
`0x81`	HELLO	Identity / version
`0x82`	EVENT	BUTTON 1, ACK 2 (+8-byte episode), THEMECHANGE 3, LINKLOST 4, LINKBACK 5
`0x83`	LOG	≤ 128 ASCII bytes
`0x84`	STATE	16 B
`0x85`	CONFIG	16 B, including `u8[12] screenCfg`
`0x86`	PROVACK	5 B

SNAPSHOT layout

Block	Offset / stride	Contents
globals	0, 40 B	u32 ts, u16 seq (duplicates ignored), u16 score, alarmActive, dataStale, stateRoll[5] (ok, degraded, down, unknown, maint), alertCounts[3] (info, warn, crit), meanLoadPct, poolCount, systemCount, hasTopAlert, u32 rx/tx kbps, u16 rttTenthMs, u16 lossPermille, 6 reserved
pools	40, stride 20, ≤ 8	name char[8], tier 0–3, per-state counts, total, loadPct, 4 reserved
systems	after pools, stride 16, ≤ 64	name char[12], state, pool index (`0xFF` = unpooled), tier, loadPct
topAlert	after systems, 84 B, only if `hasTopAlert`	u32 alertId, u32 episodeId, severity, 3 reserved, subject char[24], detail char[48]

States are OK 0, DEGRADED 1, DOWN 2, UNKNOWN 3 and MAINT 4. MAINT is reserved and is always 0 today. Severities are INFO 0, WARN 1 and CRIT 2. Names use the uppercase charset `[A-Z0-9 .:-]`.

A new `episodeId` re-arms the tone, and acknowledged episodes are never reused. History such as load ribbons and RTT sparklines is not sent on the wire. The firmware builds it up from successive snapshots.

PROVISION limits

SSID ≤ 32 bytes, PSK 8–63, host ≤ 253, CA certificate ≤ 2048, client certificate ≤ 4096, client key ≤ 2048. These are used for the standalone Wi-Fi panel.

7.4 Daemon configuration

The config file is `/etc/solari-panel/solari-panel.conf`. Each line is `key=value`, and `#` starts a comment. The template is `status-panel/daemon/solari-panel.conf.example`.

Key	Meaning
`apiBase`	Dashboard base URL, e.g. `https://dashboard.akoria.net` (required)
`user`, `passFile`	Panel principal and its password file (required; file mode 0600, owned by the service user)
`pollSec`, `snapshotSec`	API poll interval (default 5 s) and frame resend interval (default 2 s); both must be positive
`serialDev`	USB-CDC device path or glob; set it empty for listener-only mode
`listen`, `listenAddr`	Optional TLS listener port and address. Wildcard addresses are refused. If the address is omitted, the daemon uses the default-route address.

Key	Meaning
`tlsCert`, `tlsKey`, `clientCa`	Listener chain, key and client CA (e.g. chlorine root). Required when `listen` is set. Files must be owned by root or the daemon user; key mode 0600, certificate and CA 0644 or stricter.
`allowedSan`	The single client SAN accepted, e.g. `panel-01.panel.akoria.net`. The client certificate also needs the clientAuth EKU.
`denylist`	File of hex certificate serials, one per line, loaded at startup

The listener requires TLS 1.2 or later and uses an AEAD-only cipher list. Deployment, flashing and certificate issuance are covered by the panel runbooks in `docs/panel/` and the firmware developer reference `docs/panel/SolariNet_Panel_Developer_Reference.html`. This manual does not repeat them.

7.5 Changing the panel contract

- Change `protocol.h` / `protocol.c` once. Both the daemon and the firmware compile it. Then rebuild both and refresh the golden and parity fixtures in `status-panel/fixtures/`.
- Reserved bytes must be zero on send and ignored on receive. Prefer additive changes. The gear list is the model: a same-version trailer after the base SNAPSHOT layout, detected by presence, so older receivers ignore it. Bump the version byte only for a breaking change, because receivers skip frames whose version they do not know.
- A new command kind needs four changes: the `PanelControlKind` enum, firmware handling, the argument validation in `dashboard/api/routes/panel.php`, and the panel tests.

8 Provisioning & Ansible hooks

Provisioning turns SoR catalogue rows (images, software items, stacks) into running machines. The REST layer validates the request and queues a privileged ctl verb. The C server runs a Python helper. The helper reaches the provisioning host, benzene, over SSH and runs Ansible from a published tree there. The design record is `docs/Provisioning_UI_Design.md`, phases P1–P7.

8.1 Catalogue API and validation

The routes are `/api/provisioning/{images, software, stacks}`, with sub-resources for artifacts, requires, conflicts, members and deps. Reads need a session and writes need operator (§3). Validation lives in `dashboard/api/lib/ProvCatalog.php`. It mirrors the CHECK constraints of migration 029 one for one, so a bad request returns a field-specific `400` instead of a database error.

A few rules are deliberately stricter than 029 for newly supplied values. Stored values are never re-validated, so seeded rows stay editable.

- `release_name`, `install_template` and `ansible_role` must each be one safe path token.
- Column URLs must be http(s) and form a single token.
- Spec keys are limited to the sets `deploy/fleet/stackgen.py` accepts. Item types are pkg, repo, artifact, script, build and role. A repo spec needs at least one family block (suse, debian, fedora).
- An empty `applies_*` list is refused. Use `null` to mean "all".
- Regexes anchor with `\z`, never `$`, so a trailing newline is rejected.

Enumerations Arches: x86_64, arm64, armhf, riscv64, i386 (artifacts may also use `any`). Boot methods: ipxe, pi_netboot, media. Refresh: manual, weekly, point_release. Run modes: reimage, apply, refresh. Run statuses: queued, running, succeeded, failed, cancelled.

DB error mapping Duplicate key or FK → `409`. CHECK → `400`. SIGNAL → `409`. Lock wait or deadlock (1205, 1213, 40001) → `503 busy`. Anything else → `500`. SQL text and driver messages are never returned.

8.2 The three enqueue verbs

Route → verb	Body	Helper (in the release)
`POST /api/provisioning/image-refresh` → `IMAGE_REFRESH`	`{image}`	`deploy/fleet/image-refresh.py`: fetches and verifies an image and updates the netboot menu
`POST /api/provisioning/stage` → `PROV_STAGE`	`{entity, image, stacks[], mac}`	`deploy/fleet/prov-stage.py`: stages a reimage plus first-boot stack work; `prov-finish.py` completes it on first boot
`POST /api/provisioning/apply` → `STACK_APPLY`	`{entity, stacks[], dry_run}`	`deploy/fleet/stack-apply.py`: applies SoR stacks to a running machine through Ansible

IDs match `^[1-9][0-9]{0,9}$` and must not exceed 4294967295. `stacks` holds 1–32 distinct slugs matching `^[a-z0-9][a-z0-9-]{0,47}$`. `dry_run` must be a JSON boolean. The MAC is normalised to lowercase with colons.

Each route calls `SolariCtl::callQueued()` with the operator and the `Idempotency-Key` header. It returns `status: "queued"` or `"started"` plus a `log` handle, and a `request` id when the call outlasts the synchronous wait (§4.2). Track progress with `GET /api/provisioning/runs[/{id}]` and `GET /api/provisioning/log?kind=image-refresh|prov-stage|stack-apply&id=N`.

Helpers write `<ctl socket dir>/<kind>-<id>.log` with mode 0640, group-readable by the ctl group. The log route returns the last 64 KiB.

8.3 The Ansible tree

Source `deploy/fleet/ansible/`: `ansible.cfg`, `playbooks/site.yml`, `inventory/hosts.yml` (example only), `group_vars/all.yml`, and roles `base-os`, `common-stack`, `solarinet-client`, `stack-items`, `dev-tools`, `directory-join`, `dns-secondary` and `shared-home`. `stack-items` dispatches on item type, with one `step-*.yml` per type (artifact, build, pkg, repo, role, script) plus `guard.yml`.

Rendering `deploy/fleet/stackgen.py` turns SoR stacks into Ansible variables. `stack-apply.py` writes a temporary vars file per run. Host-specific values live in `inventory/host_vars/<host>.yml` on benzene.

Where it runs On benzene, from `${PROV_ROOT}/ansible` (default `PROV_ROOT=/srv/solari-provision`). It never runs from the release on xenon. Helpers reach benzene as `${BENZENE}` (default `jason@benzene.akoria.net`) with `ssh -o BatchMode=yes`.

Image store `${PROV_ROOT}/store` is NAS-backed and guarded by `prov-store-guard.sh`. Store health is written to `<ctl socket dir>/prov-store.json` and served by `GET /api/provisioning/store`: `{known, store, fstype, source, reachable, mounted, linked, ok, size_bytes, avail_bytes, reason, checked_at, stale}`.

Publishing the tree: `sync-ansible.sh`

Every release that changes `deploy/fleet/ansible` must be published to benzene after `install-release.py`:

```
/opt/solarinet/current/deploy/fleet/sync-ansible.sh          # publish
/opt/solarinet/current/deploy/fleet/sync-ansible.sh --check  # diff only: 0 in sync, 1 drift, 2 check failed
```

- Each publish extracts into its own directory under `${PROV_ROOT}/ansible.releases/`. Directory names sort in publish order.
- `${PROV_ROOT}/ansible` is a symlink, switched with one atomic rename. A playbook that is already running keeps resolving files in the tree it started with.
- The newest three trees are kept, and `ansible.prev` points at the previous one. To roll back, repoint the symlink.

- benzene's own `inventory/` is carried forward.
- Each tree carries a `.release` marker holding the release version.

> **INFERRED: THE "(UNKNOWN)" STAMP.** The script takes the marker's value from `RELEASE.json` two levels above the script (`"version"`). When there is no such file, or the value fails `^[A-Za-z0-9._-]+$` , it writes `unknown` . A git checkout has no `RELEASE.json` , but an installed release does: `/opt/solarinet/current/RELEASE.json` holds `rc-20261103-c39f565-io` . So an "(unknown)" stamp most likely means the publish ran from a checkout rather than from `/opt/solarinet/current` . The marker on benzene was not read for this manual.

8.4 Extending provisioning

1. Add the SoR shape: a migration (§11.3), plus CDC triggers if appliers must see it (030 is the model).
2. Teach `ProvCatalog` the new fields and keys. Its `SPEC_KEYS` must match `stackgen.py` 's `*_KEYS` exactly.
3. Render it in `stackgen.py` , and add the role or task file under `deploy/fleet/ansible/roles/` .
4. Add tests: `tests/dashboard/test_provisioning_api.php` (it uses the fake ctl client in `_prov_worker.php` , so no socket is needed) and `tests/deploy/test_prov_helpers.py` and its neighbours for the helpers.
5. Release, then run `sync-ansible.sh` . Without that step the new role exists on xenon but not where playbooks run.

9 Notifications

Notifications are carried on the message bus. Any process can publish a small JSON message to the RabbitMQ topic exchange `notify.events`. `notifyd` consumes it and fans it out to pluggable senders. `alertbridge` turns the C server's `alertEvent` rows into such messages, and it pages when a reporting node goes silent. Publishers never need to know which transport delivers the message.

9.1 Message contract

Publish to exchange `notify.events` (durable topic, vhost `solari`) with any routing key under `notify.`. The queue is bound with `notify.#`, so every key reaches it. alertbridge uses `notify.<severity>`.

```
{
  "severity": "crit",                // crit | warn | info (default info)
  "title": "xenon: disk /var 94% full",
  "body": "Free space on /var dropped below 10% …",
  "channel": "auto",                 // auto = route by severity, or a sender name
  "to": ["…"],                       // optional; else [defaults] default_<sender>_to
  "source": "monitoring",            // free-form provenance tag
  "ts": 1783483080                   // unix seconds; default = receipt time
}
```

At least one of `title` or `body` is required. Values that are supplied are validated. A message that cannot be parsed is logged and dropped.

For one-off use from a shell or an agent, run `deploy/notify/notify-send.py --severity warn --title … --body … [--source …]`. It reads the AMQP URL from `notify.conf`, or from `--config`.

9.2 Dispatch and delivery semantics

1. **Choose senders.** If `channel` names an enabled sender, only that sender runs. If it names an unknown or disabled sender, a warning is logged and routing falls back to severity. Otherwise `[routing] <severity>` lists the senders. If that is empty, `[defaults] default_channel` is used.
2. **Resolve recipients.** The message's `to` wins. Otherwise the sender's `default_sms_to`, `default_push_to`, `default_apple_to` or `default_email_to` is used.
3. **Send.** Each sender receives `message` plus `_to` and `_routing_key`, and its own `[sender.<name>]` section.
4. **Acknowledge.** The message is acked if any sender succeeded. It is also acked, after an error is logged, if all senders failed. There is no requeue and no dead-letter exchange, so delivery is **at most once per sender**. A TTL or DLX on the broker is the documented path if replay is ever wanted.

9.3 Senders

Config name	Module	State	Transport
`log`	`senders/log.py`	working	Appends a line to `/var/log/solarinet/notify.log` and logs to journald. This is the durable fallback.
`apple`	`senders/apple.py`	working	SSH to an always-on Mac and send an iMessage with `osascript`. iCloud then fans it out to every Apple device.
`email`	`senders/mail.py`	implemented	iCloud SMTP from a dedicated address, using an app-specific password held in `notify.conf`. The file is named `mail.py` so it does not shadow the stdlib `email` module.
`webpush`	`senders/webpush.py`	implemented	Web Push to installed dashboards. Subscriptions are read from the monitoring DB, and messages are signed with a VAPID key file.
`mqtt`	`senders/mqtt.py`	implemented	Republishes to Mosquitto under `notify/<severity>` for the Tab5 display.
`sms`	`senders/sms_tachyon.py`	stub	Tachyon cellular SBC. The `send_sms()` seam has two candidate backends (`mmcli_ssh`, AT), and both only log what they would run.
`push`	`senders/push.py`	stub	Interface-only placeholder, so routing that names `push` does not crash.

The "State" column comes from each module's own docstring. On 2026-11-03, `notify.service` and `alertbridge.service` were both `active` on xenon. notifyd runs as `/opt/solarinet/current/apps/notify/notifyd.py --config …/deploy/notify/notify.conf`, so its config path is still in a checkout rather than `/etc`.

> The Proton mail migration means the iCloud-based `email` and `apple` paths may change. See §13.

9.4 alertbridge: bridge and dead-man's switch

Bridge Polls the monitoring DB with `alertEvent WHERE eventId > checkpoint`, joined to `node` and `alertRule`. It publishes each new row to `notify.events` as `notify.<severity>`.

Dead-man's switch A node that was reporting is declared silent after max(120 s, 3 × `sample_interval_sec`). The client default for `sampleIntervalSec` is 15. A silent node raises a **crit** notification.

Config Path from `ALERTBRIDGE_CONF`. Sections `[db]`, `[amqp]` (url, exchange, reconnect_delay_sec), `[bridge]` (poll_interval_sec, batch, source), `[deadman]` (enabled, sample_interval_sec) and `[state]` (file; defaults to `alertbridge.state.json` next to the config, holding the checkpoint and the silent set). The DB credentials come from an environment file that the unit sources.

Contract `docs/design/HOST_HEALTH_CONTRACT.md` §6 (Unit D); operator view in `docs/SolariNet_Host_Health_Monitoring.md`.

9.5 notify.conf reference

The template is `deploy/notify/notify.conf.example`. Real files are gitignored. Never commit them.

Section	Keys
`[amqp]`	`url`, `exchange` (notify.events), `queue` (notify.dispatch), `binding_keys` (notify.#), `reconnect_delay_sec`
`[defaults]`	`default_sms_to`, `default_push_to`, `default_apple_to`, `default_email_to`, `default_channel`
`[routing]`	`crit`, `warn`, `info`, each a comma-separated sender list
`[senders]`	`enabled`. Unknown names are logged and skipped, never fatal.
`[sender.log]`	`path`
`[sender.sms]`	`method`, `host`, `ssh_user`, `ssh_key`, `modem_index`, `at_device`, `at_baud`
`[sender.push]`	`endpoint`
`[sender.webpush]`	`db_host`, `db_port`, `db_name`, `db_user`, `db_pass`, `vapid_private_key`, `vapid_subject`
`[sender.mqtt]`	`host`, `port`, `username`, `password`, `topic_prefix`, `tls`, `client_id`
`[sender.email]`	`smtp_host`, `smtp_port`, `smtp_user`, `smtp_pass`, `from_addr` (per the `mail.py` docstring; not present in the example file)
`[sender.apple]`	`ssh`, `ssh_opts`

The procedure for adding a sender is recipe §11.8.

10 Configuration reference

This chapter lists key and variable **names**, never values. Every file named here has a checked-in `.example` or `.sample` template. The real files are gitignored, or live outside the repo with restrictive modes. Never commit or paste their contents.

> **LIVE PATHS ARE NOT YET `/etc`-CLEAN.** On xenon (2026-11-03) several units still read config from a working checkout rather than `/etc/solarinet`:

Unit	Config path
`solarinet-server`	`--config /home/jason/Code/Solarinet/run/server.conf`
`notify`	`…/deploy/notify/notify.conf`
`alertbridge`	`ALERTBRIDGE_CONF=…/deploy/alertbridge/alertbridge.conf`
`sor-emit`	`SORSYNC_CONF=…/deploy/sorsync/sorsync.conf`

The hardened template expects `/etc/solarinet/server.conf`. The binaries do run from the immutable release. Moving the configs is a follow-up (§13).

10.1 C daemons (server, client, monitor)

The format is INI with `[section]` and `key = value`. Comments start with `;` or `#`. A key may repeat where it is a list (`probe.target`, `peer.peer`). Templates: `deploy/dashboard/hardened/server.conf.example`, `deploy/dashboard/client.conf.sample`, `deploy/monitor.conf.sample`, and the Ansible template `roles/solarinet-client/templates/client.conf.j2`.

Section	Keys	Used by
`[identity]`	hostFqdn, nodeId	all
`[bind]`	ingestUrl, surveyUrl, controlUrl, pubUrl, ctlSocket, useTls	server
`[server]`	primaryUrl, failoverUrl, subUrl, reportAckUrl	client, monitor
`[tls]`	caFile, certFile, keyFile, bind	all
`[ca]`	mode, url, certFile, keyFile	server
`[db]`	host, port, name, user, socket, poolSize. The password comes from `SOLARI_DB_PASS` in an env file.	server
`[lease]`	leaseRenewSec, leaseTtlSec	server (HA)
`[discovery]`	autoDiscover, autoEnroll	server
`[ingest]`	allowLegacyReports: a rolling-upgrade bridge for pre-session agents. Default and recommendation: `false`.	server
`[ctl]`	enforcePeer, operatorUid, dashboardUid, socketGid, helperRoot (default: the release root, pinned with realpath), logDir (default: the socket directory)	server
`[control]`	stateFile	client
`[schedule]`	sampleIntervalSec (sample default 15), watchdogIntervalSec	client
`[watch]`	spoolDb, process, logfile	client
`[probe]`	roundIntervalSec, probesPerRound, probeTimeoutMs, replFactorHint, spoolDb, target (`icmp:HOST`, `tcp:HOST:PORT`; repeatable)	monitor
`[peer]`	gossipUrl, gossipIntervalSec, peer (repeatable), peerTtlSec	monitor

The keys were extracted from the `solariConfigGet*` call sites under `src/` and the templates. Template URLs use `tls+tcp://host:7701` for ingest and 7702–7703 for survey, control and pub.

10.2 Dashboard (PHP-FPM environment)

The dashboard reads configuration only through `getenv()`. Values are set in the FPM pool (`deploy/dashboard/php-fpm-solarinet.conf`, hardened variant under `deploy/dashboard/hardened/`). No dashboard INI file exists.

Area	Variables
Monitoring DB	`SOLARI_DB_DSN`, `SOLARI_DB_HOST`, `SOLARI_DB_PORT`, `SOLARI_DB_NAME`, `SOLARI_DB_SOCK`, `SOLARI_DB_USER`, `SOLARI_DB_PASS`, `SOLARI_DB_CHARSET`
SoR DB	`SOR_DB_HOST`, `SOR_DB_PORT`, `SOR_DB_NAME`, `SOR_DB_USER`, `SOR_DB_PASS`
Auth	`SOLARI_AUTH_FILE` (default: project-root `solari-auth.json`), `SOLARI_LOCAL_LOGIN`, `SOLARI_AUTH_RATE_DIR`
Control	`SOLARI_CTL_SOCK` (default `/run/solari/solariCtl.sock`; provisioning logs and `prov-store.json` are read from its directory)
Policy / DNS	`SOLARI_ALLOWED_ORIGINS`, `SOLARI_DNS_EXTERNAL`, `SOLARI_RPZ_ZONE_FILE`
CA / identity	`STEPCA_URL`, `SOLARINET_CA_PEM`, `STEPCA_CA_FILE`, `KC_TOKEN_URL`, `KC_ADMIN_URL`, `KC_SVC_CLIENT`, `KC_SVC_SECRET`
Forgejo	`FORGEJO_URL`, `FORGEJO_TOKEN`
Inventory	`SOLARI_UPLOAD_DIR`

The auth file (`solari-auth.json`)

The file holds local users, with `password_hash` hashes managed by `dashboard/api/tools/solari-passwd.php`, and an `oidc` block. These are the `oidc` keys and their defaults (`Oidc::config()`):

Key	Default	Notes
`enabled`	false	SSO is offered only if this is true and issuer, clientId, clientSecret and redirectUri are all set. A half-filled block reads as off.
`issuer` , `allowedEndpointOrigins`	"", []	Explicit HTTPS origins for split-host IdPs
`clientId` , `clientSecret` , `redirectUri`	""	Keycloak client
`caBundle` , `insecureSkipVerify`	"", false	`insecureSkipVerify` is for development only
`scopes` , `buttonLabel`	"openid profile email", "Sign in with SSO"	
`adminGroups` , `operatorGroups` , `defaultRole`	["domain admins"], [], "viewer"	Group-to-role mapping
`postLoginRedirect`	"/"	

Request inputs (not configuration)

Handlers also read these request headers: `Idempotency-Key` , `X-Solari-CSRF` , `Origin` , `Referer` , `Host` , `Content-Type` , `Content-Length` and `User-Agent` . They also read the server variables `HTTPS` , `SERVER_PORT` , `REMOTE_ADDR` , `REMOTE_USER` and `SOLARI_ROLE` . Behaviour is described in §2.

10.3 Python services

Service	Locator	Sections / keys
sorsync (`sor_emitd` , `sor_apply_dnsd` , reconciler, UniFi)	`SORSYNC_CONF` , default: `sorsync.conf` next to the script	`[sor]` , `[sor_emit]` , `[sor_discovery]` , `[sor_unifi]` : host, port, user, password, name. `[amqp]` : url, exchange, poll_interval_sec, reconnect_delay_sec. `[dns]` : gen_zones, render_python, domain, incoming_root, target_state_file, applied_zone_dir, runtime_zone_dir, release_helper, release_config, gen_bare, secondary_refresh_helper, primary_address, primary_config, primary_additional_configs, secondary_refresh_timeout_sec, debounce_sec, reconcile_interval_sec. `[dns.secondary.<name>]` : address, ssh_host, update_helper. `[discovery]` : mon_host, mon_port, mon_user, mon_password, mon_name, interval_sec, source_label. `[pihole]` and `[pihole.<host>]` : targets, domain, ssh, custom_list, reload. `[ad]` : dc, zone, admin_user, admin_pw_file, protect, state_file.
notifyd	`--config` , default: adjacent `notify.conf`	See §9.5
alertbridge	`ALERTBRIDGE_CONF` , default: adjacent; `SOLARI_DB_PASS` is a fallback for `[db] password`	See §9.4
panel daemon (C)	`/etc/solari-panel/solari-panel.conf`	See §7.4
DNS release	`/etc/solarinet/dns-release.json`	Live zone-transfer and secondary lists (provisioning P7)
SoR read access	`/etc/solarinet/prov-sor.cnf`	MariaDB option file, section `[sor]` (§6.1)

Other templates in the tree: `deploy/backups/backup.conf.example` , `deploy/authbroker/authbroker.conf.example` , `deploy/discovery/*.conf.example` , `deploy/mosquitto/mosquitto.conf.example` , `deploy/opie/opie.conf.example` , and the PKI and CRL Apache snippets.

11 Extension recipes

Each recipe lists, in order, every file a change of that kind must touch, the test that proves it, and the trap most likely to catch you. They assume the contracts in §2–§10. The map below shows where each recipe lands.

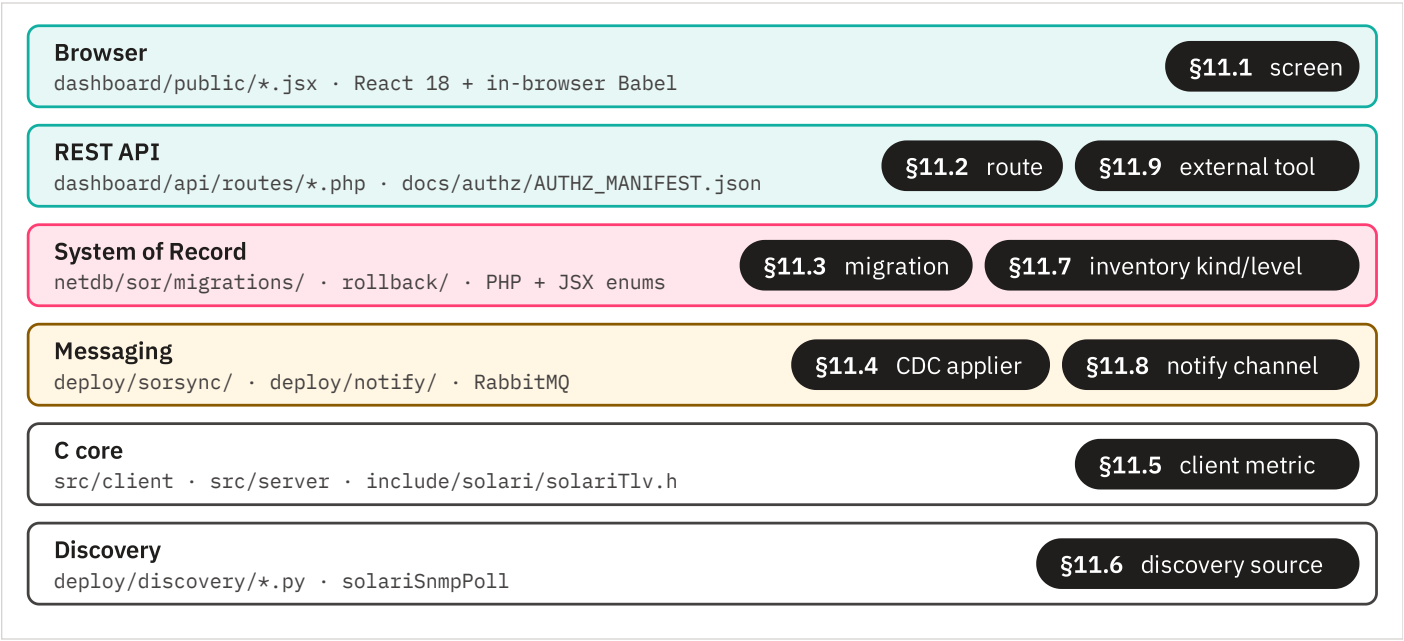


Figure 5. The six layers a change can touch, with the recipe for each. Many changes cross layers: a new inventory kind (§11.7) touches the SoR enum, the PHP validator and the JSX list, and a new route (§11.2) often needs a migration (§11.3) first.

COMMON RULES

- Core daemons are standard C99 (`CMAKE_C_EXTENSIONS OFF`). The dashboard back end is PHP. Ops glue is Python under `deploy/`.
- No server-side JavaScript, and no build step or bundler. The dashboard's JSX is compiled in the browser (§11.1).
- Every SoR schema change is a ledgered migration with a rollback script and a rehearsal (§11.3).
- Every new route or ctl verb gets an `AUTHZ_MANIFEST.json` entry. CI fails if the manifest and the code disagree.

11.1 Add a dashboard screen

The dashboard is a static single-page app in `dashboard/public/`. `index.html` loads each source as `<script type="application/x-solarinet-jsx" src="X.jsx?v=…">`. `boot.js` compiles those scripts in the browser with the vendored Babel 7.29.0 and caches the output in Cache Storage under `solari-jsx-transform-v2`. Nothing is compiled ahead of time.

1. **Write the component.** Put it in a new `screensN.jsx` (the current files run `screens.jsx` to `screens8.jsx`) or in the screen file for that area. Export it on `window` at the end of the file, for example `Object.assign(window, { ProvImagesScreen })` (`screens-prov.jsx:2205`). There are no modules or imports; globals are the linkage.
2. **Load it.** Add a `<script type="application/x-solarinet-jsx" src="screensN.jsx?v=…">` tag to `index.html` after the files it depends on (`api.jsx`, `components.jsx`, `data.jsx`). Bump the `?v=` on every file you change, because it is the cache key.
3. **Precache it.** Add the file to the precache list in `sw.js` and bump `CACHE_VERSION` (currently `"solari-v15"`). If you skip this, offline and iPad home-screen users keep the old bundle.
4. **Route it.** In `app.jsx`, add the component to the destructure from `window` (line 8). Add a render line next to the others (around line 291): `{route.name === "x" && XScreen && <XScreen toast={toast} onNavigate={go} />}`. Add a command-palette entry near line 227.
5. **Navigate to it.** Add a nav item to `components.jsx` (around line 185), for example `{ id: "x", label: "…", icon: "…" }`. Icons come from `icons.jsx`.
6. **Declare its maturity.** Map the route to a capability in `data.jsx` (lines 643–657). `test_capability_maturity.js` fails on a route with no mapping.
7. **Fetch data** only through `window.SolariAPI` (`api.jsx`). Add the endpoint to the `EP` map (from line 262) and call `getJSON` / `post`. `unwrap()` captures `csrfToken` and unwraps the `{ok,data}` envelope. Never call `fetch` directly; you would lose the CSRF header and error mapping.
8. **Test.** Run `node tests/dashboard/test_jsx_parse.js`, `test_layout.js`, `test_capability_maturity.js` and any screen-specific `test_*_ui.js`. CI runs these with Node (`ci.yml` lines 723–738). Node is a CI tool only; nothing in the repo depends on it at runtime or build time.

TRAP A JSX syntax error does not fail any build, because there is none. The screen silently fails to register in the browser. `test_jsx_parse.js` is the only gate, so run it before pushing. Design and test for iPad Safari first.

11.2 Add an API route

1. **Choose the file.** SoR-backed CRUD goes in a file under `dashboard/api/routes/`. Each file returns `static function (Router $router): void { … }`. Bridges to solariCtl verbs go in `routes_mutations.php`.
2. **Register the file** in the `$groups` array in `routes.php`. The loader calls `$register = require $file; $register($router);` in array order. Order matters where files share helpers: `inv_codes.php` and `location_admin.php` must load after `inventory.php`.
3. **Declare the policy** as the fourth argument: `$router->post('/api/x', $handler, ['auth' => 'operator'])`. The values are `public | session | operator | admin`. With no declaration, GET/HEAD/OPTIONS default to `session` and every write defaults to `operator`. Writes also get the Origin check and `X-Solari-CSRF` check automatically (§2.4). Do not declare `csrf: 'panel-service'` on a new route; that bridge is scheduled for removal (§13).
4. **Write the handler** with the shared helpers:
   - Bodies: `solari_json_body($max)`.
   - Output: `Response::ok($data, $status)` and `Response::error($code, $message, $status)`. These produce the envelope in §2.
   - Normalising values: `Coerce::{iso,json,int,float,bool,id}`.
   - Re-checking the role inside a handler: `Operator::requireOperator()`.

   Map database errors to named codes and never return SQL text. For optimistic locking, copy the `expected_updated_at` pattern (§2.9).
5. **Bind each placeholder once.** PDO runs with native prepares, so a repeated `:name` raises `HY093`, which surfaces as a 500. Use `:a1`, `:a2`, and so on. `tests/dashboard/test_pdo_placeholders.php` fails CI on a reuse.
6. **Declare constants above the file-scope `return`.** Constants are not hoisted, so a `const` written below the `return` never runs. See `rackwire.php:83-99`, which declares `RW_TELEMETRY_MAX_KEYS` before the `return` on line 100.
7. **Update the manifest.** Add a `phpRoutes` entry to `docs/authz/AUTHZ_MANIFEST.json`, for example `{"method":"POST","pattern":"/api/x","csrf":"origin+session-token","roleRequired":"operator"}`. Reads use `"csrf":"none"`. In the baseline, 100 entries use `origin+session-token`, 92 use `none` and 2 use `panel-service`. A route that calls a ctl verb must also be listed in that verb's `phpCallers` under `ctlVerbs`.
8. **Queue privileged verbs.** A verb whose `class` is `privileged` must be called through `callQueued` (`REQUEST_SUBMIT`, §4), never directly. `test_authz_manifest.php` fails if a privileged verb is called directly or `privilegedDirectCallSites` is non-empty.
9. **Test.** Run `php tests/dashboard/test_authz_manifest.php` and `test_pdo_placeholders.php`, plus a route test modelled on the nearest `tests/dashboard/test_*.php` for the same area.
10. **Document it.** Appendix B is generated from the router table when this manual is rebuilt, so the new route appears there automatically. The description in §3 and any new error codes in §2 are written by hand; add them.

11.3 Add a SoR migration

1. Create `netdb/sor/migrations/NNN_name.sql` with the next free three-digit number. `tools/migrate.py --database sor` applies files in order and records each one in `solari_migration_ledger`, with a SHA-256 checksum. **Never edit an applied file**: the checksum check will refuse the whole stream.
2. Write the reverse in `netdb/sor/rollback/NNN-down.sql` and mention it in the migration's header comment. The runner has no rollback command; rollback scripts are applied by hand or in rehearsal. If the rollback needs data the forward step destroys, snapshot that data into a table inside the migration. 036 does this with `_solarinet_m036_location_baseline`.
3. Add a rehearsal script, `tests/netdb/test_<name>_NNN.sh`, modelled on `test_prov_images_os_034.sh`. It applies forward, checks the result, rolls back, and applies forward again against a scratch database. It must refuse to run unless the target is a disposable database. In CI it runs with `SOLARI_TEST_SOR_DB=1` and `MYSQL_HOST=127.0.0.1`.
4. Add the rehearsal as a step in **both** CI workflows, Forgejo and GitHub.
5. Regenerate the schema snapshot with `tools/export_schema.py` into `netdb/sor/schema.snapshot.sql`, and run `tools/audit_schema.py`. Commit the snapshot. `migrate.py --verify-live` compares the live schema against it.
6. If the new table should emit CDC events, add its trigger in the same migration and follow §11.4 step 5.
7. Add grants for any new runtime identity in `deploy/sorsync/sor-runtime-grants.sql`.

> **INET6** SoR IP columns are native `INET6`. `INET6_NTOA(col)` and a bare `INET6_ATON('10.0.0.1')` silently return the wrong value against that type. Use `CAST(col AS CHAR)` and `CAST('…' AS INET6)`, or the v4-mapped `::ffff:` form (§6).

Applying a migration in production, and backing out of it, is covered in §12.3.

11.4 Add a CDC applier

An applier consumes the `sor.events` topic exchange and makes some external system match the SoR (§5).

1. **Write the consumer** as `deploy/sorsync/sor_apply_<name>.py`. It loads config through `db_config.py`, connects to the broker, and declares a durable queue bound to `sor.events` with the keys it cares about, for example `sor.dns_hosts.*`. It reconciles once from current SoR state, then consumes through `delivery.consume_after_apply()`. A message is acknowledged only when the apply callback returns exactly `True`. Copy `sor_apply_dnsd.py:276`.
2. **Write the unit** as `deploy/sorsync/sor-apply-<name>.service`, following `sor-apply-dns.service` for restart policy and config path.
3. **Grant least privilege** in `sor-runtime-grants.sql`: SELECT on exactly the tables the applier renders.
4. **Test** with unit and behaviour tests in `deploy/sorsync/test_*.py`. CI's sorsync job discovers them automatically and runs them against a disposable MariaDB and RabbitMQ.

5 **Coordinate `optional_tables`.** The emitter publishes as mandatory unless a table appears in `[amqp] optional_tables`. An unroutable mandatory message stalls the checkpoint. For a new CDC table: deploy the trigger with the table in the optional list, then start the consumer, and only then remove the table from the list (see the hazard note in 030).

> **DESIGN RULE** Treat the event as a hint. Apply the *current* SoR state, idempotently, for the whole artefact you own. Duplicate, reordered and missed events then all converge. A periodic reconcile catches anything missed.

11.5 Add a client metric (C)

1 **Collect it in the PAL.** Declare the function in `src/client/platOS.h` and implement it in `src/client/plat/platLinux.c`, plus every other `plat/<os>.c` that exists. Exactly one `plat` file is compiled. PAL functions return `solariStatus`, write into caller-owned buffers, and never allocate. An OS that cannot supply the value returns a "not supported" status rather than inventing a value.

2 **Carry it.** Add the field to `solariClientReport` in `include/solari/solariMsg.h`. Assign a TLV id in `include/solari/solariTlv.h` from the host-metrics range `0x10xx`. `0x1001` – `0x100D` and `0x1010` – `0x1014` are taken, so check the header for the next free id. All integers are big-endian.

3 **Encode and decode** in `lib/solari/solariMsg.c`, with the writer alongside `TLV_CR_HEALTH_SCALARS` (line 133) and the reader case near line 251. Fill the field in `src/client/clientCollect.c` and `clientReport.c`.

4 **Persist it** in `src/server/serverDb.c` (prototypes are in `server.h`). Put the column in a new **monitoring** migration, `db/migrations/NNN_name.sql`. That stream is separate from the SoR and currently ends at 037. Apply it with `tools/migrate.py --database monitoring`.

5 **Test** with Unity suites in `tests/unit/`: extend `test_msg.c` (round trip), `test_tlv.c`, `test_platlinux.c` and `test_server_db.c`. New suites are added to the list in `tests/CMakeLists.txt` and run under CTest.

> **COMPATIBILITY** Readers skip unknown TLVs. Old servers therefore accept reports from new clients, and new servers must accept reports with the TLV missing: default the field and store NULL. Never change the meaning or width of an existing TLV; add a new id instead. This is what lets the protocol grow without a version bump (§4).

11.6 Add an SNMP or discovery source

There are two layers. **Monitoring-side discovery** fills the monitoring database's `discovered` table. The **reverse reconciler**, `deploy/sorsync/sor_reconcile_discovery.py`, then proposes or merges those rows into the SoR.

- **Client-observed source** (seen from a host). Add a value to the `TLV_DISC_SOURCE` enum (`0x1203`: 1 mDNS, 2 ARP, 3 advert, 4 portscan, 5 LLDP). Emit it from `src/client/clientDiscovery.c` and handle it in `src/server/serverDiscovery.c` and `serverDiscoveryEnrich.c`. Test with `test_server_discovery*.c`.
- **SNMP data.** Build with `-DSOLARI_WITH_SNMP=ON`. The code shells out to net-snmp's `snmpbulkwalk` at runtime; there is no libnetsnmp build dependency. Interface counters belong in `serverSnmp.c`, run by the standalone `solariSnmpPoll` (systemd-timer friendly, kept out of the failover-critical server loop). LLDP neighbours belong in `serverTopology.c`. Test with `test_server_snmp.c` and `test_server_topo_snmp.c`.
- **Out-of-band enricher** (Python). Copy the pattern in `deploy/discovery/`: a `<name>.py`, a `<name>.conf.example` and a `solari-<name>.service`. The enricher UPDATEs existing `discovered` rows by IP and never overwrites `via`, the first-discovery method. The existing enrichers are `mdns_inspect.py`, `avahi_import.py`, `oui_enrich.py` and `nmap_enrich.py`.
- **Provenance in the SoR.** SoR rows record who asserted them through `source_id` → `sources` and `asserted_kind`. A new automated source gets its own `sources` slug through a migration (§11.3), and writes `asserted_kind='machine'`.

> **SCOPE** Active scanning is capped at about 4096 hosts, a /20, per discovery scan. Keep new active sources inside the private ranges the existing enrichers use.

11.7 Add an inventory product kind or location level

Both lists are closed enumerations enforced in three places: the SoR `ENUM`, PHP validation, and the UI. Change all three in one change set.

Layer	Product kind	Location level
SoR	`product_types.kind` ENUM (033): hardware, peripheral, component, software, accessory, consumable, other	`locations.loc_type` ENUM (036): property, floor, room, section, storage_unit, plus triggers `trg_locations_level_ins` and `trg_locations_level_upd`
PHP	`routes/product_catalog.php`, lines 19 and 157 (inline lists)	`routes/location_admin.php`: `LOC_CHILD_LEVEL` (property → floor → room → section)
UI	`screens4.jsx`: `CAT_KINDS` and `CAT_KIND_LABEL` (line 1587)	`screens8.jsx`: `LOC_ICON` and `TAG_LOC_TYPES` (line 53)

1 Write a migration (§11.3) that widens the ENUM. When renaming or removing a value, follow 036's widen → remap → narrow sequence. Update the level triggers if the hierarchy changes.

2 Update the PHP list and the hierarchy map, then the UI constants and labels.

3 For a new location level, decide whether it can hold storage units. A `storage_unit` may sit under anything, including another storage unit.

4 Test with the inventory PHP route tests and `test_inventory_*_ui.js`.

> **PREFER DATA TO ENUM** If the new "kind" is really a *sort of container*, add a row to `container_types` through the UI or API. It is an editable table, and no migration is needed.

> **SINCE 037** The Components picker hides retired (soft-deleted) product types, so retiring a type needs no picker change. Software-kind types carry licence fields: seats, expiry, a reference-only licence pointer (never the key itself), and installed-on links. A new kind that needs per-item fields should follow that pattern rather than overloading notes. Live location codes are globally unique, so a new level's code scheme must not collide with existing codes.

11.8 Add a notification channel

1. Create `deploy/notify/senders/<name>.py` exporting `send(message, cfg) -> bool`. `message` is the decoded event plus `_to` and `_routing_key`. `cfg` is that sender's `[sender.<name>]` section. Return `False` on an expected failure rather than raising.
2. Add it to `REGISTRY` in `notifyd.py` and to `[senders] enabled`.
3. Add a `[sender.<name>]` block to `notify.conf.example`, with keys only and no values. Reference secrets by file path, not inline.
4. Route to it with `[routing] <severity> = …`, or let a publisher pick it explicitly with `channel` (§9.2).
5. Test with `deploy/notify/test_notifyd.py` (dispatch, routing) and `test_notify_send.py` (CLI).

> **DELIVERY GUARANTEE** notifyd acknowledges a message even when every sender fails. There is no requeue or dead-letter queue, so a channel that must not lose alerts needs its own retry and spool (§9.2, §13).

11.9 Drive the API from an external tool

A script uses the same session, Origin and CSRF rules as the browser. This complete curl session reads a paged list and makes one write. Run it as an operator account. The password comes from a file, never from the command line.

```
B=https://dashboard.akoria.net
J=$(mktemp); trap 'rm -f "$J"' EXIT
# 1. Log in (local accounts; needs SOLARI_LOCAL_LOGIN=1 on the server)
jq -n --arg u ops --rawfile p ~/.config/solari/ops.pass '{username:$u,password:($p|rtrimstr("\n"))}' |
  curl -sS -c "$J" -H 'Content-Type: application/json' -H "Origin: $B" --data @- "$B/api/auth/login" >/dev/null
# 2. Fetch the session's CSRF token
TOK=$(curl -sS -b "$J" "$B/api/auth/whoami" | jq -r .data.csrfToken)
# 3. Read every page
off=0
while :; do
  r=$(curl -sS -b "$J" "$B/api/inventory/components?limit=500&offset=$off")
  echo "$r" | jq -c '.data.items[]'
  [ "$(echo "$r" | jq .data.page.hasMore)" = true ] || break
  off=$(echo "$r" | jq .data.page.nextOffset)
done
# 4. Write: cookie + Origin + X-Solari-CSRF
curl -sS -b "$J" -H "Origin: $B" -H "X-Solari-CSRF: $TOK" -H 'Content-Type: application/json' \
  --data '{"code":"SN-000123"}' "$B/api/inventory/codes/lookup" | jq .
```

- **Check** `.ok` on every response. On failure, branch on `.error.code`, never on the message text (Appendix A).
- **Writes without** `Origin` are rejected with `403 forbidden` even when the token is right.
- **SSO-only accounts** cannot use this flow, because OIDC needs a browser. Create a dedicated local operator account for automation.
- **Retry** `503 busy` with back-off. On `409` (edit conflict), re-read the row first.
- **Prefer the SoR** (§6, read-only) for bulk reporting. Use the API when you need its validation or business rules.

> **INFERENCE** The account name `ops` and the path `~/.config/solari/ops.pass` are placeholders. The code-lookup body shape was read from `inv_codes.php:84`.

12 Maintenance

This chapter covers what keeps the interfaces in §2–§10 stable over time: pinned dependencies, immutable releases, ledgered migrations with compensating rollbacks, backups, a proposed deprecation policy, and the test matrix that guards all of it.

12.1 Dependencies

Layer	Dependency	Version / pin	Where pinned
C core	Mbed TLS	`mbedtls-3.6.7` (`068ff08`); `framework` submodule `dde0c4a`	Git submodules under `third_party/`; `third_party/README.md`
	NNG	`v1.12.5` (`437a5f7`)	
	cJSON	`v1.7.19` (`c859b25`)	
	SQLite	Minimal header only (`sqlite3_min.h`). With `SOLARI_WITH_SQLITE=ON` the system `libsqlite3` is linked (`lib/solari/CMakeLists.txt:46`)	`third_party/sqlite/`
	Unity	Vendored source (test only)	`third_party/unity/`
Runtime CLIs	net-snmp (`snmpbulkwalk`), avahi-utils, nmap	Distribution packages; optional	Only when `SOLARI_WITH_SNMP` or the discovery tools are enabled
Dashboard UI	React / ReactDOM	18.3.1, production builds	Vendored under `dashboard/public/`; no package manager
	Babel standalone	7.29.0 (in-browser JSX)	
	IBM Plex fonts	Vendored `.woff2` in `dashboard/public/fonts/`	
Dashboard API	PHP	8.5.4 on xenon (observed)	Host package. PDO with native prepares; no Composer
Database	MariaDB	11.8.6 client on xenon (observed)	Host package. Native `INET6` is required
Python daemons	Released daemons	`pika==1.4.1`, `PyMySQL==1.2.0`, `PyYAML==6.0.3`, `paho-mqtt==2.1.0`, `pywebpush==2.3.0`	`deploy/release/requirements.txt`. The installer builds a `.venv` inside each release from it
	sorsync, unifi, authbroker	Exact pins	Their own `requirements.txt`
	alertbridge, opie, mcp	Unpinned or range pins	Their own `requirements.txt`, used only for checkout runs
Broker	RabbitMQ	4.3.7 image	`deploy/sorsync/RabbitMQ-4.3.7.Dockerfile`; CI tests a version matrix

- **Upgrading a C dependency.** Move the submodule to a release tag, update the table in `third_party/README.md`, and rebuild the pinned I/O prefix with `deploy/build-pinned-io-deps.sh`. Release binaries link against that prefix (`SOLARI_VENDORED_DEPS=OFF`), so a submodule bump has no effect on releases until the prefix is rebuilt. Run the full CTest suite and the cross builds. Clones used for releases need `--recursive`.
- **Upgrading a vendored JS file.** Replace the file, bump its `?v=` and the `sw.js` `CACHE_VERSION`. If Babel changed, also bump the transform cache name in `boot.js` (`solari-jsx-transform-v2`) so stale compiled output is discarded.
- **Python.** Change pins in `deploy/release/requirements.txt`; the next release installs them. Range-pinned components should move to exact pins (§13).

12.2 Versioning and release

SolariNet has no semantic version. The runnable unit is an **immutable release** named `rc-YYYYMMDD-<sha7>-io`. The SHA is the source commit and `-io` marks a build with the I/O feature set. A milestone on `main` gets a dated tag, currently `release-20261030`.

1. **Build** from a clean, committed checkout. `deploy/build-release-binaries.sh <prefix> <build-dir>` compiles against the pinned I/O prefix with `-DSOLARI_WITH_IO=ON -DSOLARI_WITH_SQLITE=ON -DSOLARI_WITH_JSON=ON -DSOLARI_WITH_DISCOVERY_TOOLS=ON`.
2. **Package** with `deploy/dashboard/package-release.py`. It produces an allowlisted tarball holding the server, scan worker, ctl client, client, monitor and SNMP binaries; `dashboard/api` and `public`; the fleet and deploy helpers; and the Python daemon sources. It writes `RELEASE.json` with the version, `sourceRevision`, `sourceTreeDirty`, the RackWire submodule pin and a SHA-256 manifest. It refuses a dirty tree unless given `--allow-dirty`, and the installer then refuses that bundle without the same opt-in.
3. **Install** with `deploy/dashboard/install-release.py`. It rejects unsafe paths, symlinks, undeclared files and hash mismatches. It stages the tree under `/opt/solarinet/releases/<version>`, builds `.venv`, and atomically switches `/opt/solarinet/current`. It does not restart services.
4. **Cut over** with `deploy/host-configs/xenon/cutover-immutable.sh`, then check with `verify-immutable-processes.py`. Every daemon must run from `/opt/solarinet/current`. Procedure: `docs/immutable-daemon-release-runbook.md`.
5. **Publish Ansible** to benzene with `deploy/fleet/sync-ansible.sh`, run from `/opt/solarinet/current` so the tree is stamped with the release version (§8.3).
6. **Roll back** by re-activating a retained release. That is a symlink switch plus a service restart. Xenon held 35 retained releases on 2026-11-03; `current` pointed at `rc-20261103-c39f565-io`.

CONTRACT VERSIONS The wire contracts version themselves independently of releases:

- SCP uses TLV extensibility (§4), so no version bump is needed.
- Panel frames carry `PANEL_PROTO_VERSION` = 1 and grow through additive, presence-detected trailers (§7.3).
- The authz manifest has `schemaVersion`.
- The SoR evolves through ledgered migrations (§12.3).

The REST API is not versioned; its stability rule is §12.5.

12.3 Migrations and rollback

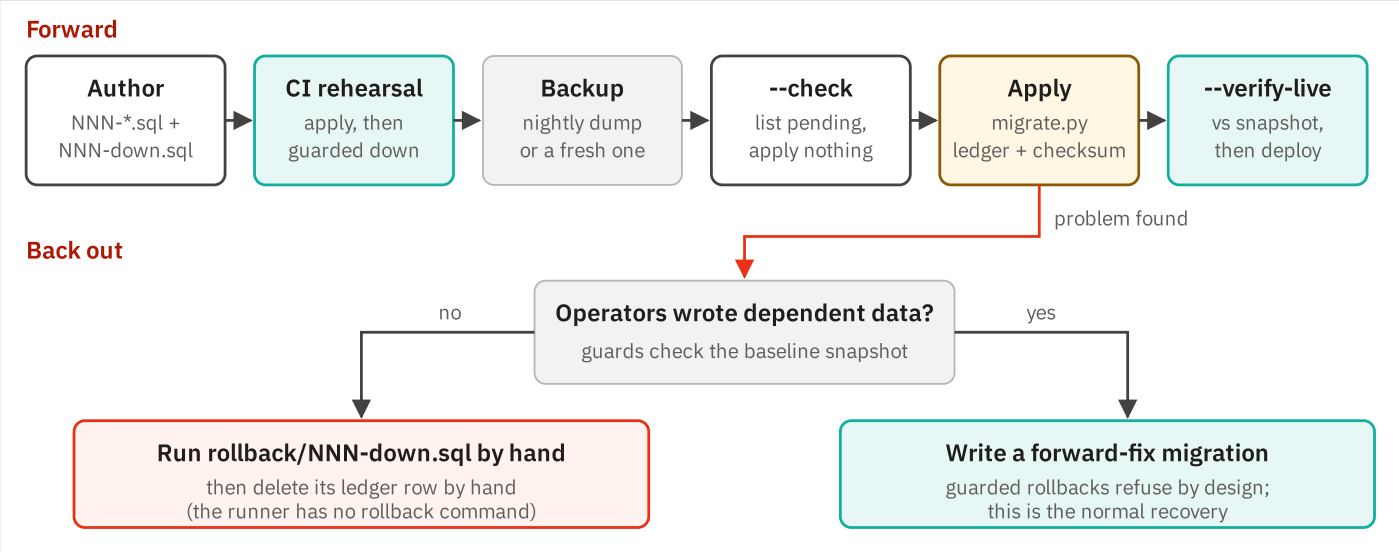


Figure 6. Applying a SoR migration with `tools/migrate.py`, and the two ways back. The runner refuses changed checksums, out-of-order history and unrecognised non-empty databases.

There are two ledgered streams. The **monitoring** stream lives in `db/migrations/`, currently ending at 037. The **SoR** stream is `netdb/sor/schema.sql` plus `netdb/sor/migrations/`; source and the live ledger both end at 037 (§6). Both are applied by `tools/migrate.py`. The runner keeps `solari_migration_ledger(migration_id, checksum, state, started_at, applied_at)` and refuses:

- a changed checksum on any applied file;
- an interrupted or out-of-order history;
- a non-empty database it does not recognise.

Applying in production

1. Confirm last night's backup exists (§12.4), or take a fresh dump of `sor` on cesium.
2. `tools/migrate.py --database sor --db-name sor --defaults-file <option file> --check` lists what is pending. It is the non-applying mode; there is no `--dry-run`.
3. Apply by running the same command without `--check`.
4. Run `--verify-live` to compare the live schema with `schema.snapshot.sql`.
5. Deploy the release whose code expects the new schema. Order matters: migrations are written to be additive first, so the old code keeps working against the new schema during the gap.

Backing out

The runner has **no rollback command**. Each migration has a compensating script, `netdb/sor/rollback/NNN-down.sql` (or a grouped one such as `024-028-down.sql`), which lives outside `migrations/` so the runner cannot discover it. The scripts are defensive. 036's rollback runs a guard procedure that `SIGNAL`s and refuses if any location changed since the migration's baseline snapshot. It never deletes operator data. After a successful rollback, **delete that migration's ledger row by hand**; every script header says so. DDL auto-commits, so a rollback that fails halfway must be finished by hand, not re-run blindly.

PREFER FORWARD FIXES Once operators have written data that depends on a migration, the guarded rollbacks will (correctly) refuse. The normal recovery is then a new forward migration. Rollback scripts exist for the window between apply and first use, and for CI rehearsal (`tests/netdb/test_sor_migration_rollback.sh`).

12.4 Backups

`deploy/backups/solari-backup.sh` runs from the templated `solari-backup@<host>.timer` nightly at 03:30 with jitter. Each host's config is `/etc/solari-backup-<host>.conf`.

Host	What	Target
cesium	MariaDB `sor`, plus Forgejo once relocated	`/data/backups/` on a separate spindle (sdb1) from the live datadir
xenon	MariaDB `solarinet` (monitoring)	An off-host NAS target is documented but not yet provisioned (§13)

- **Safety.** Before publishing a dump, the runner decodes the whole gzip stream and rejects an empty payload. The target directory must be root-owned with mode exactly `0700`, and artefacts are `0600`. Retention is 14 days, pruning only files that match the backup name pattern.
- **Failure alerting.** On failure, `solari-notify-failure@.service` publishes to `notify.events` (§9).
- **Restore** is documented in `docs/SolariNet_Backup_and_Restore_Runbook_2026-10-20.html`.

> **OBSERVED** On xenon, `solari-backup@xenon.timer` last ran at 2026-11-03 03:31 UTC and was next due 2026-11-04 03:38 UTC. The cesium timer was not checked from xenon.

What backups do not cover. Configuration under `/etc/solarinet`, the live configs in the xenon checkout (§10), and benzene's provisioning store are not in the nightly dump. Releases are reproducible from git.

12.5 Deprecation policy PROPOSED

No written policy exists today. The rules below are proposed. They codify what the code already does and are for the Lead and operator to adopt or amend.

1. **Additive by default.** New response fields, new optional request fields, new routes, new TLVs and new panel trailers do not need notice. Clients must ignore unknown fields. The code already does this for TLVs and panel frames.
2. **Breaking changes need a compatibility window.** A breaking change is removing or renaming a field or route, changing a type or meaning, tightening validation on existing data, or changing a default. It ships behind a compatibility mode for at least one release, as pagination did: omitting `limit` and `offset` still returns the bare array.
3. **Mark it.** A deprecated route or field is listed in §13 with its replacement and its earliest removal release. Where practical, the response carries a `Deprecation: true` header.
4. **Schema.** Columns are dropped only in a migration that follows the release which stopped reading them. Compatibility triggers bridge legacy writers in the meantime, as 033 and 035 do for `hardware_models`.
5. **Security exceptions.** A change needed to close a vulnerability may skip the window. It is recorded in §13 after the fact.

The live deprecations are the panel CSRF bridge (§7.2) and the legacy `hardware_models` writers (§6). Both are tracked in §13.

12.6 Testing matrix

Two workflow files define the same twelve jobs, including the SoR migration rehearsals: `.forgejo/workflows/ci.yml` (canonical) and `.github/workflows/ci.yml`. GitHub adds a thirteenth, `macos`, which builds the portable core and Unity suites as a universal binary. Run the suite for any layer you touched before pushing.

Suite	Run locally	Guards	CI job
C / Unity	`make test` or `ctest --test-dir build --output-on-failure`	SCP framing, TLV, messages, config, crypto, client/monitor/server logic, SNMP and topology parsers. Fault and fuzz suites are separate.	`core`, `server`, `protocol-fuzz`
Cross builds	`deploy/cross-build-client.sh`	Linux ARM64 and ARM32, Windows targets	`cross`
Dashboard PHP	`php tests/dashboard/<test>.php`	Auth, policy, CSRF, authz manifest, PDO placeholders, route contracts, provisioning, inventory	`dashboard`
Dashboard UI	`node tests/dashboard/test_jsx_parse.js` and the other `test_*.js`	JSX parses, layout, capability maturity, inventory screens	`dashboard`
Browser	Playwright steps in the workflow	RackWire and dashboard packaged flows	`rackwire-browser`
sorsync	`python -m unittest discover -s deploy/sorsync -p 'test_*.py'`	Emitter, appliers, delivery; disposable MariaDB plus RabbitMQ version matrix; grants; rollback/forward recovery	`sorsync`
SoR migrations	`SOLARI_TEST_SOR_DB=1 bash tests/netdb/test_*_NNN.sh`	Forward, rollback and forward again on a scratch database: 030, 031 and 033–036, plus the grouped `test_sor_migration_rollback.sh`	`sorsync`; 033–036 also in `dashboard`
Python services	`python3 -m unittest discover -s tests/deploy`, `deploy/notify`, `deploy/alertbridge`, `deploy/backups`	Fleet, provisioning helpers, notify dispatch, backups, alertbridge	`python-integrations`
Release	`python3 -m unittest discover -s deploy/dashboard -p 'test_release.py'`, `deploy/release/test_immutable_release.py`	Packaging allowlist, tamper and traversal rejection, install and rollback	`dashboard` runs `test_release.py`. `test_immutable_release.py` is not wired into either workflow (§13)
Panel	`make -C status-panel/firmware/test`, `make -C status-panel/daemon test`, `make -C status-panel/tools smoke`	Firmware frame codec (host build) against the golden fixtures, the daemon, and the tooling smoke test	`panel`

> **NOT RUN HERE** This table was compiled from the workflow files and test sources. Writing this manual did not run any of these suites.

13 Roadmap

This chapter lists planned interface changes, live deprecations, and the backlog that came out of writing this manual. Status chips: DONE shipped and live; PLANNED scoped in an approved plan; BACKLOG identified but not scheduled; PARKED deliberately deferred. Status is as of 2026-11-03.

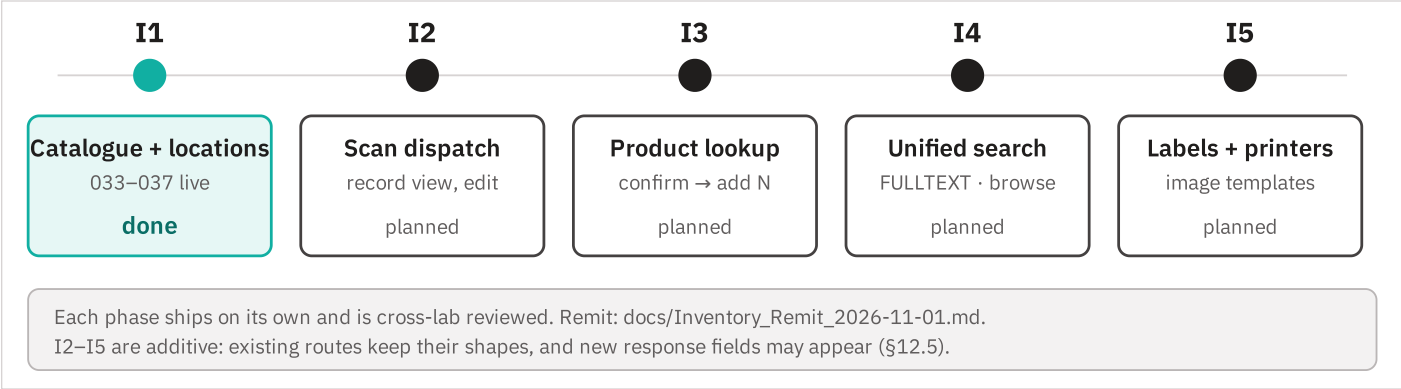


Figure 7. Inventory phases as of 2026-11-03. I1 is complete; I2–I5 are planned but not yet designed at route level.

13.1 Inventory phases I1–I5

The plan of record is `docs/Inventory_Remit_2026-11-01.md`. Each phase is shippable on its own and cross-lab reviewed. The "API impact" column is INFERRED from the remit's design section (§4 of that document); none of I2–I5 has been designed at route level yet.

Phase	Status	Delivers	API impact
I1	DONE	Broadened catalogue (`product_types` + barcodes + backfill), location levels with uniqueness, move and merge, bulk storage units, container types, project assignment. Close-out migration 037 is live.	Shipped: catalogue and location routes in §3. Migrations 033–037.
I2	PLANNED	Scan dispatch, and a record view with edit and delete for items, locations and projects (R3)	A dispatch endpoint that resolves one scanned string, in order: exact registry match → system-format prefix (`AKO-`, `LOC-`, `PRJ-`, `SI-`) → GTIN with check digit → manual. Soft-delete routes with audit.
I3	PLANNED	Product lookup, and the "scan → confirm → add N items" flow (R1)	A server-side PHP provider adapter (UPCitemdb free tier, then Open Products Facts) with a cache table keyed by normalised GTIN. The browser never calls the providers. Results are candidates the operator confirms.
I4	PLANNED	Unified search and editable browse views (R6)	One search endpoint over product types, items, locations, codes and receipts: MariaDB FULLTEXT plus exact-code match. Likely a new FULLTEXT-index migration.
I5	PLANNED	Label templates, verified printer profiles, share-to-vendor-app (R5)	Rendered label *images* sized per printer (MakeID EP53, MakeID E1, Nemonic MIP-201W). There is no direct-print bridge, so this is probably a render route or client-side canvas rather than a device API.

FOR INTEGRATORS I2–I5 are additive under §12.5. Existing catalogue, location and code routes keep their shapes. New fields may appear in responses; clients must ignore unknown fields.

13.2 Deprecations

What	Status	Replacement	Removal condition
Panel service CSRF bridge (`csrf: 'panel-service'`, §7.2), used by two routes	BACKLOG	`solariPanel` echoes the session's `csrfToken` in `X-Solari-CSRF` like any other client	The daemon change ships and is deployed on lithium. Then drop the policy flag, `Policy::isCsrfBridged()`, and the manifest value.
Legacy `hardware_models` writers, bridged into `product_types` by triggers (033, 035)	BACKLOG	Write `product_types` directly (§3)	The writers in the baseline must go first: the `/api/inventory/models` create and update handlers (`inventory.php:489`, `:521`), the catalogue mirror (`product_catalog.php:101`), and the reconciler's arch fill (`sor_reconcile_discovery.py:113`). Then a migration drops the compatibility triggers.
Bare-array responses on page-envelope routes (compatibility mode, §2.8)	PARKED	Send `limit` and `offset`, then read `{items, page}`	None scheduled. It stays until every in-repo caller pages explicitly.

13.3 Interface and operations backlog

These items were found while this manual was verified against the code and the live system. Each is a proposal, not a commitment. The *Evidence* column says how it was found.

Item	Status	Area	Evidence
Optimistic lock (`updated_at`) on component `PUT` / `DELETE`	BACKLOG	Inventory (§2.9)	Source: the component handlers do not check `updated_at`; catalogue, location and container-type edits do.
Move live xenon configs from the `/home/jason/Code/Solarinet` checkout to `/etc/solarinet`: `run/server.conf`, `run/db.env`, and the notify and alertbridge configs	BACKLOG	Config (§10)	Observed in the running units. The immutable-release runbook keeps them in the checkout "for this cutover".
Run `sync-ansible.sh` from `/opt/solarinet/current` so the benzene tree is stamped with the release version instead of "unknown"	BACKLOG	Provisioning (§8.3)	Source: REV comes from `RELEASE.json`, which only exists in a release. The cause of the stamp is `INFERRED`.
Notify: add a retry queue or dead-letter exchange. Messages are acked even when every sender fails (at-most-once delivery).	BACKLOG	Notify (§9)	Source: `notifyd.py` dispatch.
Notify: the `sms` and `push` senders are stubs that only log	BACKLOG	Notify (§9)	Source: `senders/sms_tachyon.py`, `senders/push.py`.
Notify: replace the Gmail email path with Proton (Bridge or hydroxide plus SMTP on a fleet host), or another phone-delivery channel	PLANNED	Notify (§9)	Operator decision on record; not started.
Pin `deploy/alertbridge/requirements.txt` (and the range pins in opie and mcp) exactly	BACKLOG	Dependencies (§12.1)	Source. Released daemons already use the exact pins in `deploy/release/requirements.txt`.
Wire `deploy/release/test_immutable_release.py` into CI	BACKLOG	Release (§12.2)	Source: neither workflow references it.
Provision the xenon off-host backup target (NAS)	BACKLOG	Backups (§12.4)	Source: `deploy/backups/README.md` says not yet provisioned.
A narrow read-only SoR reporting account for external tools, instead of the broad `solari` account	BACKLOG	SoR (§6)	The broad account was accepted by the operator for now. A read-only account is cheap and lets §6 integrations avoid write grants.
Drop `_solarinet_m036_location_baseline` once 036 rollback is no longer wanted	BACKLOG	SoR (§12.3)	Observed in the live SoR. Dropping it makes `036-down.sql` refuse, by design.
A checker that compares this manual's route index (Appendix B) with the router table and `docs/authz/AUTHZ_MANIFEST.json`, so new or removed routes are flagged	BACKLOG	Docs	This HTML file is the maintained source; edit it directly. The catalogue was first drafted from a one-off extraction, so drift has to be caught by review.
Keycloak's LDAP bind uses the AD Administrator account; move it to a dedicated service bind account	BACKLOG	Auth (§2)	Posture debt recorded after the 2026-08-17 SSO outage.
Standalone Wi-Fi status panel: battery-pack record, flash, certificate, DNS and cutover	PLANNED	Panel (§7)	Merged; waiting on operator gates in the panel runbook. The TLS listener contract in §7.4 is already in the daemon.
macOS PAL (`plat/platMacos.c`) so the client and monitor build on macOS	BACKLOG	Client (§11.5)	Source: the GitHub `macos` job notes that it is a future step.
Static-musl client build matrix for heterogeneous hosts	BACKLOG	Client	Recorded in `docs/SolariNet_Handoff_2026-06-26.html`; not started.
A hosted MCP connector exposing this API to Claude	PARKED	External tools (§11.9)	Parked 2026-09-18. A custom connector is fetched from the vendor's servers, so it would need public exposure, which is not acceptable on xenon.

A Error code index

Every error uses the envelope `{ok:false, error:{code, message}}` (§2.6). Clients should branch on the HTTP status and the `code`. The `message` is for people and may change.

A.1 Raised before a handler runs

The router, the policy layer and authentication raise these for any route. They are omitted from the per-route lists in §3.

HTTP	Code	Condition	Source
401	unauthorized	No valid session on a route that needs one	Auth.php
403	forbidden	Role too low: viewer on an operator route, or not admin on an admin route	Operator.php via Policy
403	forbidden	"Cross-origin or missing Origin on a state-changing request."	Policy::enforceCsrf
403	forbidden	"Missing or invalid CSRF token." The X-Solari-CSRF header does not match the session token.	Policy::enforceCsrf
403	forbidden	"Unrecognized policy for ...". The route declares an unknown auth level, which is a server bug; the request fails closed.	Policy.php
404	not_found	"No route for ..."	Router.php
405	method_not_allowed	The path exists but not for this method	Router.php

A.2 Raised by route handlers

This table is generated from the router table, like Appendix B. It lists each (status, code) pair a handler can return, how many routes return it, and up to three of those routes. The same code may appear under more than one status. For example, `control_error` is `400` when solariCtl rejects the arguments as invalid, `502` when the bridge replies badly, and `503` when this server is not the active primary ("active server required", "not primary", "lease lost"). A role or confirm-token refusal from the bridge becomes `403` (`SolariCtl::statusForCode`).

HTTP	Code	Routes	Examples
400	bad_request	168	POST /api/auth/login, POST /api/panel/command, POST /api/panel/state (+165 more)
400	confirmation_required	1	POST /api/networks/{id}/allocator-eligibility
400	control_bad_request	1	POST /api/discovery/scan
400	control_error	20	GET /api/discovery/scans/{scanId}, POST /api/discovery/{discId}/adopt, POST /api/discovery/{discId}/ignore (+17 more)
400	unsupported_media_type	1	POST /api/discovery/{discId}/adopt
401	invalid_credentials	1	POST /api/auth/login
401	unauthorized	2	GET /api/auth/whoami, POST /api/enrollments/{enrId}/reject
403	forbidden	36	POST /api/panel/command, POST /api/panel/state, POST /api/panel/config (+33 more)
403	local_login_disabled	1	POST /api/auth/login
404	not_found	89	GET /api/nodes/{id}, POST /api/alerts/ack, POST /api/maintenance/{id}/cancel (+86 more)
409	bad_request	1	POST /api/maintenance/{id}/cancel
409	confirm_required	9	POST /api/enrollments/{enrId}/approve, POST /api/control/decommission, POST /api/nodes/{nodeId}/retire (+6 more)
409	conflict	55	GET /api/provisioning/images, GET /api/provisioning/images/{id}, POST /api/provisioning/images (+52 more)
409	container_not_empty	1	PUT /api/inventory/locations/{id}/project
409	duplicate_barcode	2	POST /api/inventory/product-types, POST /api/inventory/product-barcodes
409	duplicate_location	6	POST /api/inventory/locations/{id}/move, POST /api/inventory/locations/{id}/merge, POST /api/inventory/locations/bulk-storage-units (+3 more)
409	entity_already_linked	1	POST /api/inventory/components/{id}/entity-link
409	location_cycle	3	POST /api/inventory/locations/{id}/move, POST /api/inventory/locations/{id}/merge, POST /api/inventory/locations/bulk-storage-units
409	merge_conflict	1	POST /api/inventory/locations/{id}/merge
409	plan_conflict	2	POST /api/rackwire/plans, POST /api/rackwire/plans/{id}/restore
409	plan_hash_required	2	POST /api/rackwire/plans, POST /api/rackwire/plans/{id}/restore
409	queue_full	1	POST /api/panel/command
409	unit_already_linked	1	POST /api/inventory/components/{id}/entity-link
413	inventory_page_required	1	GET /api/inventory

HTTP	Code	Routes	Examples
413	payload_too_large	13	POST /api/provisioning/software/{id}/artifacts, POST /api/provisioning/software/{id}/requires, POST /api/provisioning/software/{id}/conflicts (+10 more)
415	unsupported_media_type	44	POST /api/auth/login, POST /api/panel/command, POST /api/panel/state (+41 more)
500	db_error	29	GET /api/provisioning/images, GET /api/provisioning/images/{id}, POST /api/provisioning/images (+26 more)
500	entity_link_failed	1	POST /api/inventory/components/{id}/entity-link
500	entity_unlink_failed	1	DELETE /api/inventory/components/{id}/entity-link
500	internal_error	14	POST /api/inventory/receipts/{id}/scan, GET /api/inventory/codes/{id}/label, POST /api/control/deploy (+11 more)
502	control_error	7	POST /api/provisioning/image-refresh, POST /api/provisioning/stage, POST /api/provisioning/apply (+4 more)
502	control_unavailable	30	POST /api/discovery/scan, GET /api/discovery/scan-requests/{requestId}, GET /api/discovery/scans/{scanId} (+27 more)
503	busy	31	GET /api/provisioning/images, GET /api/provisioning/images/{id}, POST /api/provisioning/images (+28 more)
503	control_error	10	POST /api/provisioning/image-refresh, POST /api/provisioning/stage, POST /api/provisioning/apply (+7 more)
503	control_unavailable	1	POST /api/control/decommission
503	entity_link_failed	1	POST /api/inventory/components/{id}/entity-link
503	label_renderer_unavailable	1	GET /api/inventory/codes/{id}/label
503	service_unavailable	22	GET /api/networks/allocator-eligibility, GET /api/networks/allocator-eligibility, POST /api/networks/{id}/allocator-eligibility (+19 more)

A.3 Retry guidance

Codes	Retry?	How
503 busy, 503 service_unavailable, 502/503 control_unavailable, 503 control_error	Yes	Back off and retry. For queued control verbs, resend the same Idempotency-Key so a retry cannot run the action twice (§4.2).
409 conflict (stale updated_at)	After re-read	Fetch the record again, re-apply the change, and send the new updated_at (§2.9).
409 merge_conflict	After re-preview	Call merge-preview again and send the new fingerprint.
409 queue_full	Later	The panel command queue already holds 16 pending commands. Wait for the daemon to drain it.
409 confirm_required, 400 confirmation_required	With confirmation	Resend with the explicit confirmation field the route documents.
413 inventory_page_required	Differently	Switch to the paged collection routes.
Other 400, 404, 409, 413, 415	No	Fix the request.
500 db_error, 500 internal_error	Once, then report	Details are in the PHP-FPM log, not in the response.

SINCE 037 Inventory writes that hit a database deadlock return 409 with a "try again" message. They are safe to retry as they are.

B Route index

All routes in the order of the §3 catalogue. Each path links to its route card. This table is generated from the router table at build time. The CSRF column shows *yes* where the Origin check and `X-Solari-CSRF` are required, and *panel* where the panel service bridge applies (§7.2). Use the method filter in the sidebar to narrow it.

B.1 All routes

Method	Path	Role	CSRF	Domain
POST	`/api/auth/login`	public	—	Authentication
POST	`/api/auth/logout`	session	yes	Authentication
GET	`/api/auth/config`	public	—	Authentication
GET	`/api/auth/oidc/login`	public	—	Authentication
GET	`/api/auth/oidc/callback`	public	—	Authentication
GET	`/api/auth/whoami`	session	—	Authentication
GET	`/api/summary`	session	—	Monitoring
GET	`/api/nodes`	session	—	Monitoring
GET	`/api/nodes/{id}`	session	—	Monitoring
GET	`/api/nodes/{id}/history`	session	—	Monitoring
GET	`/api/probes`	session	—	Monitoring
GET	`/api/alerts`	session	—	Monitoring
GET	`/api/rules`	session	—	Monitoring
POST	`/api/alerts/ack`	operator	yes	Monitoring
POST	`/api/alerts/ack-all`	operator	yes	Monitoring
GET	`/api/maintenance`	session	—	Monitoring
POST	`/api/maintenance`	operator	yes	Monitoring
POST	`/api/maintenance/{id}/cancel`	operator	yes	Monitoring
GET	`/api/opie`	session	—	Monitoring
GET	`/api/opie/{reportId}`	session	—	Monitoring
GET	`/api/segments`	session	—	Monitoring
GET	`/api/netgear`	session	—	Monitoring
GET	`/api/topology`	session	—	Monitoring
GET	`/api/gear`	session	—	Monitoring
GET	`/api/gear/{id}/interfaces`	session	—	Monitoring
GET	`/api/gear/{id}/history`	session	—	Monitoring
GET	`/api/push/vapid`	session	—	Monitoring
POST	`/api/push/subscribe`	operator	yes	Monitoring
DELETE	`/api/push/subscribe`	operator	yes	Monitoring
GET	`/api/stream`	session	—	Monitoring
GET	`/api/panel`	session	—	Status panel
POST	`/api/panel/command`	operator	yes	Status panel
POST	`/api/panel/state`	session	panel	Status panel
POST	`/api/panel/config`	session	panel	Status panel
GET	`/api/discovery`	session	—	Fleet control
GET	`/api/enrollments`	session	—	Fleet control
POST	`/api/discovery/scan`	operator	yes	Fleet control

Method	Path	Role	CSRF	Domain
GET	/api/discovery/scan-requests/{requestId}	operator	—	Fleet control
GET	/api/discovery/scan-jobs	operator	—	Fleet control
GET	/api/discovery/scans/{scanId}	operator	—	Fleet control
POST	/api/discovery/{discId}/adopt	operator	yes	Fleet control
POST	/api/discovery/{discId}/ignore	operator	yes	Fleet control
POST	/api/enrollments	public	—	Fleet control
POST	/api/enrollments/ack	public	—	Fleet control
POST	/api/enrollments/{enrId}/approve	operator	yes	Fleet control
POST	/api/enrollments/{enrId}/reject	operator	yes	Fleet control
POST	/api/control/provision	operator	yes	Fleet control
POST	/api/control/decommission	operator	yes	Fleet control
POST	/api/nodes/{nodeId}/retire	operator	yes	Fleet control
POST	/api/nodes/{nodeId}/criticality	operator	yes	Fleet control
POST	/api/control/survey	operator	yes	Fleet control
POST	/api/control/deploy	operator	yes	Fleet control
GET	/api/control/deploy	session	—	Fleet control
POST	/api/control/fleet-provision	operator	yes	Fleet control
POST	/api/control/fleet-image	operator	yes	Fleet control
GET	/api/control/fleet-provision	session	—	Fleet control
GET	/api/config	session	—	Fleet control
POST	/api/config	operator	yes	Fleet control
GET	/api/nodes/{id}/config	session	—	Fleet control
POST	/api/nodes/{id}/config	operator	yes	Fleet control
POST	/api/rules/{ruleId}	operator	yes	Fleet control
POST	/api/rules/{ruleId}/delete	operator	yes	Fleet control
GET	/api/pools	session	—	Fleet control
POST	/api/pools	operator	yes	Fleet control
POST	/api/pools/{poolId}	operator	yes	Fleet control
POST	/api/pools/{poolId}/delete	operator	yes	Fleet control
GET	/api/assets	session	—	Fleet control
GET	/api/assets/{assetId}	session	—	Fleet control
POST	/api/assets/{assetId}	operator	yes	Fleet control
POST	/api/assets/{assetId}/remove	operator	yes	Fleet control
POST	/api/assets/{assetId}/lifecycle	operator	yes	Fleet control
POST	/api/assets/{assetId}/purge	operator	yes	Fleet control
POST	/api/assets/{assetId}/criticality	operator	yes	Fleet control
POST	/api/assets/{assetId}/targets/{targetId}/remove	operator	yes	Fleet control
GET	/api/networks/allocator-eligibility	session	—	DNS
POST	/api/networks/{id}/allocator-eligibility	operator	yes	DNS
GET	/api/dns/query	session	—	DNS
GET	/api/dns/health	session	—	DNS
GET	/api/dns/zones	session	—	DNS
GET	/api/dns/zones/{id}/records	session	—	DNS
GET	/api/dns/hosts	session	—	DNS

Method	Path	Role	CSRF	Domain
GET	/api/dns/rpz	session	—	DNS
GET	/api/forgejo	session	—	Identity
GET	/api/ca	session	—	Identity
GET	/api/identity	session	—	Identity
GET	/api/builds	session	—	Provisioning
GET	/api/provisioning/images	session	—	Provisioning
GET	/api/provisioning/images/{id}	session	—	Provisioning
POST	/api/provisioning/images	operator	yes	Provisioning
PUT	/api/provisioning/images/{id}	operator	yes	Provisioning
DELETE	/api/provisioning/images/{id}	operator	yes	Provisioning
GET	/api/provisioning/software	session	—	Provisioning
GET	/api/provisioning/software/{id}	session	—	Provisioning
POST	/api/provisioning/software	operator	yes	Provisioning
PUT	/api/provisioning/software/{id}	operator	yes	Provisioning
DELETE	/api/provisioning/software/{id}	operator	yes	Provisioning
POST	/api/provisioning/software/{id}/artifacts	operator	yes	Provisioning
DELETE	/api/provisioning/software/{id}/artifacts/{artifactId}	operator	yes	Provisioning
POST	/api/provisioning/software/{id}/requires	operator	yes	Provisioning
DELETE	/api/provisioning/software/{id}/requires/{requiresId}	operator	yes	Provisioning
POST	/api/provisioning/software/{id}/conflicts	operator	yes	Provisioning
DELETE	/api/provisioning/software/{id}/conflicts/{otherId}	operator	yes	Provisioning
GET	/api/provisioning/stacks	session	—	Provisioning
GET	/api/provisioning/stacks/{id}	session	—	Provisioning
POST	/api/provisioning/stacks	operator	yes	Provisioning
PUT	/api/provisioning/stacks/{id}	operator	yes	Provisioning
DELETE	/api/provisioning/stacks/{id}	operator	yes	Provisioning
POST	/api/provisioning/stacks/{id}/members	operator	yes	Provisioning
PUT	/api/provisioning/stacks/{id}/members/{softwareId}	operator	yes	Provisioning
DELETE	/api/provisioning/stacks/{id}/members/{softwareId}	operator	yes	Provisioning
POST	/api/provisioning/stacks/{id}/deps	operator	yes	Provisioning
DELETE	/api/provisioning/stacks/{id}/deps/{dependsOnId}	operator	yes	Provisioning
GET	/api/provisioning/targets	session	—	Provisioning
GET	/api/provisioning/log	session	—	Provisioning
GET	/api/provisioning/store	session	—	Provisioning
GET	/api/provisioning/runs	session	—	Provisioning
GET	/api/provisioning/runs/{id}	session	—	Provisioning
POST	/api/provisioning/image-refresh	operator	yes	Provisioning
POST	/api/provisioning/stage	operator	yes	Provisioning
POST	/api/provisioning/apply	operator	yes	Provisioning
GET	/api/inventory	session	—	Inventory
GET	/api/inventory/components	session	—	Inventory
GET	/api/inventory/components/{id}	session	—	Inventory
GET	/api/inventory/components/{id}/entity-link	session	—	Inventory
POST	/api/inventory/components/{id}/entity-link	operator	yes	Inventory

Method	Path	Role	CSRF	Domain
DELETE	`/api/inventory/components/{id}/entity-link`	operator	yes	Inventory
POST	`/api/inventory/components`	operator	yes	Inventory
PUT	`/api/inventory/components/{id}`	operator	yes	Inventory
DELETE	`/api/inventory/components/{id}`	operator	yes	Inventory
GET	`/api/inventory/models`	session	—	Inventory
GET	`/api/inventory/models/{id}`	session	—	Inventory
POST	`/api/inventory/models`	operator	yes	Inventory
PUT	`/api/inventory/models/{id}`	operator	yes	Inventory
GET	`/api/inventory/locations`	session	—	Inventory
GET	`/api/inventory/locations/{id}`	session	—	Inventory
POST	`/api/inventory/locations`	operator	yes	Inventory
PUT	`/api/inventory/locations/{id}`	operator	yes	Inventory
GET	`/api/inventory/receipts`	session	—	Inventory
GET	`/api/inventory/receipts/{id}`	session	—	Inventory
POST	`/api/inventory/receipts`	operator	yes	Inventory
POST	`/api/inventory/receipts/{id}/scan`	operator	yes	Inventory
GET	`/api/inventory/receipts/{id}/scan`	session	—	Inventory
GET	`/api/inventory/projects`	session	—	Inventory
GET	`/api/inventory/projects/{id}`	session	—	Inventory
GET	`/api/inventory/allocations`	session	—	Inventory
POST	`/api/inventory/projects`	operator	yes	Inventory
PUT	`/api/inventory/projects/{id}`	operator	yes	Inventory
DELETE	`/api/inventory/projects/{id}`	operator	yes	Inventory
POST	`/api/inventory/projects/{id}/allocations`	operator	yes	Inventory
PUT	`/api/inventory/projects/{id}/allocations/{allocId}`	operator	yes	Inventory
DELETE	`/api/inventory/projects/{id}/allocations/{allocId}`	operator	yes	Inventory
GET	`/api/inventory/shopping`	session	—	Inventory
POST	`/api/inventory/locations/{id}/move`	operator	yes	Inventory
GET	`/api/inventory/locations/{id}/merge-preview`	session	—	Inventory
POST	`/api/inventory/locations/{id}/merge`	operator	yes	Inventory
POST	`/api/inventory/locations/bulk-storage-units`	operator	yes	Inventory
PUT	`/api/inventory/locations/{id}/project`	operator	yes	Inventory
GET	`/api/inventory/container-types`	session	—	Inventory
POST	`/api/inventory/container-types`	operator	yes	Inventory
PUT	`/api/inventory/container-types/{id}`	operator	yes	Inventory
DELETE	`/api/inventory/container-types/{id}`	operator	yes	Inventory
GET	`/api/inventory/product-types`	session	—	Inventory
GET	`/api/inventory/product-types/{id}`	session	—	Inventory
DELETE	`/api/inventory/product-types/{id}`	operator	yes	Inventory
POST	`/api/inventory/product-types`	operator	yes	Inventory
PUT	`/api/inventory/product-types/{id}`	operator	yes	Inventory
GET	`/api/inventory/product-barcodes`	session	—	Inventory
POST	`/api/inventory/product-barcodes`	operator	yes	Inventory
DELETE	`/api/inventory/product-barcodes/{id}`	operator	yes	Inventory

Method	Path	Role	CSRF	Domain
GET	`/api/inventory/codes`	session	—	Inventory
POST	`/api/inventory/codes/lookup`	operator	yes	Inventory
POST	`/api/inventory/codes/receive`	operator	yes	Inventory
POST	`/api/inventory/codes/move`	operator	yes	Inventory
POST	`/api/inventory/codes/associate`	operator	yes	Inventory
GET	`/api/inventory/codes/history`	session	—	Inventory
POST	`/api/inventory/codes`	operator	yes	Inventory
GET	`/api/inventory/codes/{id}/label`	session	—	Inventory
POST	`/api/inventory/codes/{id}/printed`	operator	yes	Inventory
GET	`/api/inventory/codes/{id}`	session	—	Inventory
DELETE	`/api/inventory/codes/{id}`	operator	yes	Inventory
GET	`/api/inventory/locations/{id}/contents`	session	—	Inventory
GET	`/api/inventory/projects/{id}/rollup`	session	—	Inventory
GET	`/api/rackwire/plans`	session	—	RackWire
GET	`/api/rackwire/plans/{id}`	session	—	RackWire
GET	`/api/rackwire/plans/{id}/versions`	session	—	RackWire
GET	`/api/rackwire/library`	session	—	RackWire
GET	`/api/rackwire/telemetry`	session	—	RackWire
GET	`/api/rackwire/sor/interconnects`	session	—	RackWire
POST	`/api/rackwire/plans`	operator	yes	RackWire
POST	`/api/rackwire/plans/{id}/delete`	operator	yes	RackWire
POST	`/api/rackwire/plans/{id}/versions`	operator	yes	RackWire
POST	`/api/rackwire/plans/{id}/restore`	operator	yes	RackWire
POST	`/api/rackwire/library/{modelId}/groups`	operator	yes	RackWire
POST	`/api/rackwire/commit`	operator	yes	RackWire